CS604 Computer Vision · Team 06

Presentation 考前学习手册

从零开始理解：用 ViT-B/16 做 10,000 类细粒度图片分类

这本手册是什么?

一本给零基础同学准备的“考前教材”。它按幻灯片的顺序，从最基础的概念讲起，解释我们用到的每一个方法、模型、图表和公式，最后附上 Q&A 准备和复习清单。

依据的材料

幻灯片 `cs604_team06_slides_v2.pptx`（共 20 页：正文 15 页 + 备用 5 页）、幻灯片的讲稿备注、课程报告 `report/main.pdf` 和方法 notebook 的说明文档。书中的数字都来自这些文件。

Presentation：2026 年 11 月 7 日（15 分钟讲解 + 5 分钟提问）

讲者分工：A 第 1–3 页　B 第 4–6 页　C 第 7–9 页　D 第 10–15 页　备用页第 16–20 页

目录

阅读指南 **1**

第 0 章　零基础预备知识 **3**

0.1　图像分类：让电脑给图片“贴标签” 3
0.2　神经网络：一个有很多“旋钮”的函数 3
0.3　怎么知道答得好不好：损失函数和准确率 4
0.4　怎么拧旋钮：梯度下降 5
0.5　死记硬背的风险：过拟合 6
0.6　站在巨人肩膀上：预训练和迁移学习 7
0.7　ViT 的直觉：把图片切块，当成句子来读 8
0.8　算力的语言 8
0.9　我们要回答的三个问题，和这本手册的路线 8

第 1 章　逐页讲解 Slides **9**

1.1　第 1 页：封面 10
1.2　第 2 页：类别多、照片少、预算固定（问题分析） 11
1.3　第 3 页：先复现基线，再比较七种方法 13
1.4　第 4 页：我们怎么迭代——一次只问一个问题 14
1.5　第 5 页：把小改动分散到每一层（七种方法的比较） 16
1.6　第 6 页：线性探针 head——起点高，终点低 17
1.7　第 7 页：我们的解决办法——从类别原型出发 19
1.8　第 8 页：训练 loss 更低，准确率反而更低（为什么有效） 21
1.9　第 9 页：每个改动都单独测试（消融实验） 23
1.10　第 10 页：全量微调训练所有参数（最终模型 1） 24
1.11　第 11 页：LoRA 只训练 1.38%，训练完再合并（最终模型 2） 26
1.12　第 12 页：所有指标上的最终结果 28
1.13　第 13 页：输入分辨率——我们找到的最大提升（截止之后） 29
1.14　第 14 页：总结（Takeaways） 31
1.15　第 15 页：Thank you（以及 Q&A 分工） 32
1.16　备用页总说明（第 16–20 页） 33
1.17　第 16 页（备用）：线性探针——只训练 head 34

1.18 第 17 页（备用）：部分微调——只训练最后 K 个 block 35
1.19 第 18 页（备用）：BitFit——只训练偏置 36
1.20 第 19 页（备用）：Adapter——在每个 MLP 后面加一个瓶颈 37
1.21 第 20 页（备用）：AdaptFormer——带缩放的旁路 38

第 2 章 方法与模型详解 39

2.1 我们的模型：ViT-B/16 40
2.2 分类头（head）和它的三种初始化 45
2.3 七种 adaptation（适配）方法 50
2.4 训练是怎么进行的 61
2.5 怎么评估：怎么判断模型好不好 66
2.6 测试时增强（TTA）和输入分辨率 67
2.7 一张总对比表 69

第 3 章 重要技术概念（术语表） 71

3.1 数据与评估 71
3.2 模型与结构 73
3.3 训练 74
3.4 微调方法 76
3.5 实验与工程 77

第 4 章 图表、公式与架构图 79

4.1 第 2 页：数据示例图和四个数字 80
4.2 第 3 页：基线表和两类方法卡片 82
4.3 第 4 页：迭代时间表 83
4.4 第 5 页：七种方法对比表和“准确率–参数量”图 85
4.5 第 6 页：head 初始化曲线 87
4.6 第 7 页：原型 head 的流程和三个数字 88
4.7 第 8 页：训练 loss 曲线、三步解释和对照实验 90
4.8 第 9 页：消融实验条形图 92
4.9 架构图逐个讲解（第 10、11、16 到 20 页） 93
4.10 第 12 页：最终结果表 98
4.11 第 13 页：分辨率表 99
4.12 公式汇总 101

第 5 章 Presentation Q&A 108

5.1 回答问题的方法 . . . 109
5.2 基础概念 . . . 110
5.3 技术原理 . . . 113
5.4 模型与 backbone 选择 . . . 118
5.5 方法比较 . . . 120
5.6 实验结果与数据解读 . . . 124
5.7 实际应用 . . . 129
5.8 过程、预算与诚信 . . . 130
5.9 最可能被问到的 10 个问题 . . . 132

第 6 章 最后总结 133

6.1 整个 Presentation 的核心逻辑 . . . 133
6.2 最重要的 10 个知识点 . . . 134
6.3 必须理解的专业术语 . . . 135
6.4 每个模型和方法的核心区别 . . . 136
6.5 最容易被老师问到的技术问题 . . . 136
6.6 Presentation 前应该重点复习什么 . . . 138

附录 A 关键数字速查表 139

A.1 任务、数据和预算 . . . 139
A.2 模型（ViT-B/16） . . . 139
A.3 参考基线（holdout） . . . 139
A.4 七种方法（原型 head、3 轮、holdout） . . . 140
A.5 Head 初始化 . . . 140
A.6 消融实验（一次改一个） . . . 140
A.7 两个最终模型 . . . 141
A.8 分辨率（截止后，holdout，3 轮） . . . 141

阅读指南

一句话讲清我们的项目

我们拿一个已经在上千万张图片上“学过看图”的模型 ViT-B/16，教它认出新数据集里的一万个类别（每类只有 45 张训练照片），而且每次训练只能用大约 1.2 个 GPU 小时。我们比较了 7 种“教”的方法，发现最关键的不是选哪种方法，而是**新加的分类器（head）一开始怎么初始化**：用“类别原型”（prototype）开始，比常见的“线性探针”（linear probe）开始高 2.1 到 3.1 分。

一页速览：最重要的数字

项目	数字和含义
任务	10,000 个细粒度类别；训练集 450,000 张（每类 45 张）；验证集 50,000 张、测试集 100,000 张，标签都看不到，只能在 CodaBench 网站上打分
我们的“小考”	从训练集每类拿出 2 张不参与训练，共 20,000 张，叫 holdout；它和 CodaBench 的分数相差不到 0.3 分
预算	每次训练 2 小时 × 48 GB 显存 = 96 GB·h，在 80 GB 的 A100 上就是 1.2 小时（允许 ±10%）
模型（backbone）	ViT-B/16，先在 ImageNet-21k 上预训练、再在 ImageNet-1k 上微调；8,580 万个参数；课程推荐的模型，我们没有换
最终模型 1：全量微调	训练所有参数；验证集 76.22%，测试集 **76.09%**（比参考答案 70.8% 高 5.4 分）；1.03 GPU 小时
最终模型 2：LoRA	只训练 1.375% 的参数；验证集 71.90%，测试集 **71.67%**（比参考 LoRA 68.3% 高 3.6 分）；0.54 GPU 小时；推理和原模型一样快
最大的发现	常见的“线性探针 head”在我们这里反而有害；“原型 head”在 3 种方法上都多出 2.1–3.1 分
截止后的发现	把图片从 224 像素放大到 288/320 像素，在 holdout 上又多 4.4 分（没有测试集分数，不算在正式结果里）

怎么读这本手册

- **只有 1 小时**：读这一页、第 6 章“最后总结”、第 5 章里你负责部分的问题，以及附录 A 的数字表。
- **有 3 小时**：再加上第 1 章“逐页讲解”中你负责的几页，以及第 2 章里 LoRA、原型 head 两节。
- **有一整天**：按顺序读。第 0 章先补最基础的概念；第 1 章逐页讲幻灯片；第 2 章把每个方法讲透；第 3 章是术语表（遇到不懂的词就查）；第 4 章逐个讲图表、公式和架构图；第 5 章是 Q&A；第 6 章是总结和复习清单。

书里的几种彩色框

重点

重点：一定要记住的结论或数字。

生活类比

生活类比：用日常生活里的事情来理解技术概念。

例子

例子：用简单的数字一步一步算给你看。

容易搞错

容易搞错：初学者常见的误解，老师追问时容易出错的地方。

公式

公式：每个符号是什么意思，整个公式想表达什么。

English answer（可以直接说）

English answer：Q&A 时可以直接说的英文回答。

讲者分工

讲者	页码	时长（估计）	负责回答的问题
A	第 1–3 页	约 3:26	预算和 GPU 小时、backbone 选择、数据和 holdout
B	第 4–6 页	约 3:36	方法细节、参数量、延迟、LoRA 和 adapter 的比较
C	第 7–9 页	约 3:47	原型 head 为什么比线性探针 head 好，以及证据
D	第 10–15 页	约 3:51	全量微调和 PEFT 的比较、公平性、局限、分辨率

时长按每分钟 130 个英文单词估算，全场约 14 分 40 秒。第 16–20 页是备用页，只在被问到具体方法时翻出来用。

第 0 章　零基础预备知识

这一章不讲我们项目的细节，只补最基础的概念。读完它，后面的每一页幻灯片你都能看懂“它在说什么”。每个概念在第 2 章、第 3 章和第 4 章里还会讲得更细。

0.1　图像分类：让电脑给图片“贴标签”

图像分类（image classification）就是：给电脑一张图片，让它说出这张图属于哪一类。

- **输入**：一张图片。电脑眼里的图片就是一大堆数字。我们的图片会被缩放成 224×224 像素，每个像素有红、绿、蓝 3 个数字，所以一张图是 $224 \times 224 \times 3 = 150{,}528$ 个数字。
- **输出**：模型给 10,000 个类别各打一个分数，分数最高的那个类别就是它的答案。

我们的数据集是**细粒度**（fine-grained）分类：类别之间长得很像，比如不同种类的鸟、蛇、蛾子和植物。类别的名字被隐藏了（只知道编号），所以我们没法借助文字信息，只能靠图片本身。

生活类比

普通人能分清“猫”和“狗”，这是普通分类；鸟类专家能分清两种只在翅膀花纹上有细微差别的麻雀，这是细粒度分类。我们要训练的是一个能认 10,000 种东西的“专家”。

0.2　神经网络：一个有很多“旋钮”的函数

神经网络（neural network）可以看成一个很复杂的函数：输入图片的数字，经过很多层计算，输出 10,000 个分数。每一层计算里都有很多可以调整的数字，叫**参数**（parameters）或**权重**（weights）。我们用的 ViT-B/16 有 85,798,656 个参数，约 8,580 万个。

训练（training）就是不断调整这些参数，让模型的输出越来越接近正确答案。

生活类比

想象一个有 8,580 万个旋钮的调音台。一开始声音（模型的答案）很乱。训练就是根据“听起来哪里不对”，一点一点地拧这些旋钮，直到声音变好听。

0.3　怎么知道答得好不好：损失函数和准确率

0.3.1　准确率（accuracy）：给人看的分数

Top-1 准确率：模型的第一个答案猜对的图片占多少比例。比如 20,000 张图里猜对 14,676 张，准确率就是 $14{,}676/20{,}000 = 73.38\%$。我们报告的所有准确率默认都是 top-1。

Top-5 准确率：正确答案只要出现在模型的前 5 个答案里就算对，所以总是比 top-1 高。

0.3.2　损失（loss）：给模型看的分数

准确率只看“对或错”，太粗糙，模型没法从中知道该往哪个方向改。所以训练时用的是**损失函数**（loss function），它衡量“模型错得有多离谱”，越小越好。我们用的是**交叉熵**（cross-entropy）。它只看一件事：模型给**正确答案**分了多少概率。

交叉熵的直觉

$$\text{loss} = -\log(p_{\text{正确类}})$$

$p_{\text{正确类}}$ 是模型认为“这张图属于正确类别”的概率（0 到 1 之间）。log 是自然对数。

- 模型很有把握且答对：$p = 0.9$，$\text{loss} = -\log 0.9 \approx 0.105$，很小。
- 模型犹豫：$p = 0.1$，$\text{loss} = -\log 0.1 \approx 2.30$。
- 模型完全瞎猜（10,000 类平均分）：$p = 1/10{,}000$，$\text{loss} = \log 10{,}000 \approx 9.21$。

所以 loss 从 9.21 往下降，就说明模型越来越有把握。

容易搞错

训练 loss 低，不一定代表模型好！loss 是在**训练照片**上算的。如果模型只是把训练照片“背”下来了，训练 loss 会很低，但遇到新照片照样认不出。这正是我们第 8 页讲的现象。

0.4　怎么拧旋钮：梯度下降

梯度（gradient）告诉我们：每个参数往哪个方向变一点点，loss 会下降得最快。**梯度下降**（gradient descent）就是每次沿着这个方向走一小步。

- **学习率**（learning rate，LR）：每一步走多大。太大会“冲过头”，太小会走得很慢。我们全量微调用 10^{-4}（0.0001），LoRA 用 10^{-3}（0.001）。
- **批次**（batch）：每次拿多少张图片一起算梯度。我们用 256 张。
- **步**（step）：每更新一次参数叫一步。
- **轮**（epoch）：把所有训练图片完整看一遍叫一轮。

生活类比

在大雾中下山：你看不到山脚，只能用脚感受哪边往下（梯度），然后迈一步（步长 = 学习率）。迈太大可能摔下悬崖，迈太小天黑了还没到。

算一算我们训练了多少步

全量微调用全部 450,000 张图，每批 256 张：每轮 $450{,}000/256 \approx 1{,}757$ 步；8 轮就是 $1{,}757 \times 8 = 14{,}056$ 步，正好是第 10 页写的步数。

LoRA 每类留出 2 张，只用 $43 \times 10{,}000 = 430{,}000$ 张：每轮 $430{,}000/256 \approx 1{,}679$ 步；3 轮就是 $1{,}679 \times 3 = 5{,}037$ 步，正好是第 11 页写的步数。

0.5　死记硬背的风险：过拟合

过拟合（overfitting）就是模型在训练数据上表现很好，在新数据上表现变差。它把训练照片“背”了下来（memorization，记忆），而不是学到能推广到新照片的规律（generalization，泛化）。

我们每类只有 45 张照片，模型参数却有几千万个，特别容易背答案。所以我们的原则是：**永远用模型没见过的照片来判断好坏，不看训练 loss**。

0.5.1　四种数据集，各有用途

数据集	大小	用途
训练集（train）	450,000 张，每类 45 张	用来调参数
holdout（我们自己留的）	20,000 张，每类 2 张	从训练集里拿出来、不参与训练，用来比较方法、选设置
验证集（validation）	50,000 张，每类 5 张	标签隐藏，提交到 CodaBench 才能看到分数
测试集（test）	100,000 张，每类 10 张	标签隐藏，最终排名用；提交次数有限

生活类比

训练集是练习题，holdout 是自己出的模拟考，验证集是学校的月考（能看分数，但不知道每题答案），测试集是期末考（只考一次）。如果模拟考和月考的分数总是差不多，就说明模拟考出得好，可以放心用它来做决定。我们的 holdout 和 CodaBench 验证集只差 0.3 分以内（例如全量微调 72.31% 对 72.57%）。

0.6　站在巨人肩膀上：预训练和迁移学习

从零开始训练一个认图模型需要海量数据。我们每类只有 45 张，远远不够。所以我们用**预训练**（pretraining）好的模型：它已经在大数据集上学会了“怎么看图”。

- 我们的模型先在 **ImageNet-21k**（约 1,400 万张图、两万多个类别）上预训练，再在 **ImageNet-1k**（约 128 万张图、1,000 个类别）上微调。这正是课程文档推荐的版本。
- **迁移学习**（transfer learning）：把在一个任务上学到的知识用到另一个任务上。
- **微调**（fine-tuning）：拿预训练模型在新数据上继续训练，让它适应新任务。

0.6.1　Backbone 和 head

- **Backbone**（主干网络）：模型的主体，负责把图片变成一串有意义的特征数字（我们的是 768 个数字）。它是预训练得来的。
- **Head**（分类头）：最后一层，把 768 个特征数字变成 10,000 个类别分数。原来的 head 只认 ImageNet 的 1,000 类，所以我们必须**换一个新的 10,000 类 head**。新 head 有 $768\times10{,}000+10{,}000 = 7{,}690{,}000$ 个参数。

生活类比

一个学过很多年美术的学生（backbone）去鸟类研究所工作。他不需要从“什么是颜色和形状”学起，只需要学会“这些特征对应哪种鸟”（head），再顺便把观察方法往鸟类细节上调一调（微调 backbone）。

重点

新 head 怎么初始化（一开始的参数设成什么），是我们整个项目最重要的发现。第 6–8 页专门讲它。

0.6.2　“教”模型的几种方式

微调可以训练所有参数（**全量微调**，full fine-tuning），也可以只训练很少一部分参数（**参数高效微调**，parameter-efficient fine-tuning，PEFT），比如 LoRA。我们一共比较了 7 种方法，第 2 章会逐个讲。

0.7　ViT 的直觉：把图片切块，当成句子来读

ViT（Vision Transformer）原来是处理文字的 **Transformer** 模型，后来被用在图片上。它的思路是：

1. 把 224 × 224 的图片切成 16 × 16 的小块（patch），一共 14 × 14 = 196 块。
2. 每一块变成一串 768 个数字，叫一个 **token**，就像句子里的一个词。再加一个特殊的 **[CLS] token**，专门负责“总结全图”，一共 197 个 token。
3. 经过 12 层 Transformer **block**（模块）。每一层里，所有小块互相“交流”（self-attention，自注意力），然后各自“思考”一下（MLP）。
4. 最后取 [CLS] token 的 768 个数字，交给 head 打分。

“ViT-B/16”里，B 表示 Base（中等大小），16 表示每块是 16 × 16 像素。

生活类比

把一张照片撕成 196 块拼图，每块拼图派一个代表去开会。开 12 轮会，每轮大家都听听别人的信息（比如“我这里有羽毛”“我这里有喙”），最后会议记录员（[CLS]）写出总结：“这是一只鸟，翅膀有白色斑点……”。

0.8　算力的语言

- **GPU**：专门做大量并行计算的芯片。我们用 NVIDIA A100，显存 80 GB。
- **显存**（GPU memory，VRAM）：GPU 上的“桌面空间”。训练时模型、数据和中间结果都要放在上面。
- **GPU 小时**（GPU-hours）：用了多少小时的 GPU。课程规定每次训练最多“2 小时 × 48 GB”，即 96 GB·h；我们的 GPU 有 80 GB，所以是 96/80 = 1.2 小时。
- **延迟**（latency）：一次处理 1 张图要多久，单位毫秒（ms）。**吞吐量**（throughput）：每秒能处理多少张图。
- **bf16**：用 16 位的数字代替 32 位的数字来算，省一半显存、算得更快，准确率几乎不变。

0.9　我们要回答的三个问题，和这本手册的路线

报告里把整个项目归结为三个问题：

1. 新的 10,000 类 head 应该怎么初始化？（第 6–8 页）
2. 用哪种适配方法？可训练的参数应该放在哪里？（第 3、5、10、11 页，以及备用页 16–20）
3. 固定的预算怎么分配给训练轮数、数据量和测试时的计算？（第 9、12、13 页）

接下来怎么读

第 1 章按页码逐页讲幻灯片；第 2 章把每个方法和模型讲透；第 3 章是术语表；第 4 章逐个讲图表、公式和架构图；第 5 章是 Q&A；第 6 章是总结和复习清单。

第 1 章　逐页讲解 Slides

这一章按页码顺序讲解全部 20 页幻灯片（正文第 1–15 页，备用第 16–20 页）。每一页都回答同样几个问题：这一页讲什么？想解决什么问题？关键概念是什么？图表怎么看？和前后页有什么联系？最后用一个框写出“讲这一页时要抓住的一句话”，英文是讲稿里的原意，可以直接用。

整个演讲的故事线（先记住这条线，再看每一页）

问题分析（第 2 页）→ 复现基线、介绍 7 种方法（第 3 页）→ 我们怎么一轮一轮做实验（第 4 页）→ 7 种方法比一比（第 5 页）→ 发现一个问题：线性探针 head（第 6 页）→ 我们的解决办法：原型 head（第 7 页）→ 为什么有效（第 8 页）→ 其他每个设置为什么这样选（第 9 页）→ 两个最终模型（第 10、11 页）→ 最终成绩（第 12 页）→ 截止后的发现：分辨率（第 13 页）→ 总结（第 14 页）。

1.1　第 1 页：封面

主讲：A



第 1 页缩略图

这一页主要讲什么

介绍题目、团队，以及两个最终模型在**隐藏测试集**上的成绩。题目很长，我们逐个词拆开：

- Prototype-initialized：新的分类 head 用“类别原型”来初始化（第 7 页详细讲）。这是我们方法的核心。
- adaptation：适配，就是把一个预训练好的模型改造成适合新任务的模型。微调是一种适配方式。
- ViT-B/16：我们用的模型，Vision Transformer，Base 大小，每个小块 16 × 16 像素。
- 10,000-class fine-grained classification：10,000 个类别的细粒度分类。

副标题有两句话。第一句：Seven adaptation methods, one budget, and the one choice that mattered most: how the new head starts，意思是我们比较了 7 种适配方法，预算固定，而最重要的一个选择是新 head 怎么开始。第二句是成绩：全量微调 76.09%，LoRA 71.67%（测试集 top-1 准确率）。

想解决什么问题

开场 30 秒内让听众知道三件事：我们做了什么（比较 7 种方法），最重要的发现是什么（head 的起点），结果有多好（两个测试成绩）。

关键概念

测试集（只能提交有限次、最终排名用）；全量微调和 LoRA（第 10、11 页讲）；head（模型的最后一层分类器）。

与后面的联系

“prototype”在第 7 页解释；“seven methods”在第 3、5 页；76.09% 和 71.67% 在第 12 页展开。

讲这一页时要抓住的一句话

One design choice mattered more than the choice of method: how the new 10,000-class head starts.

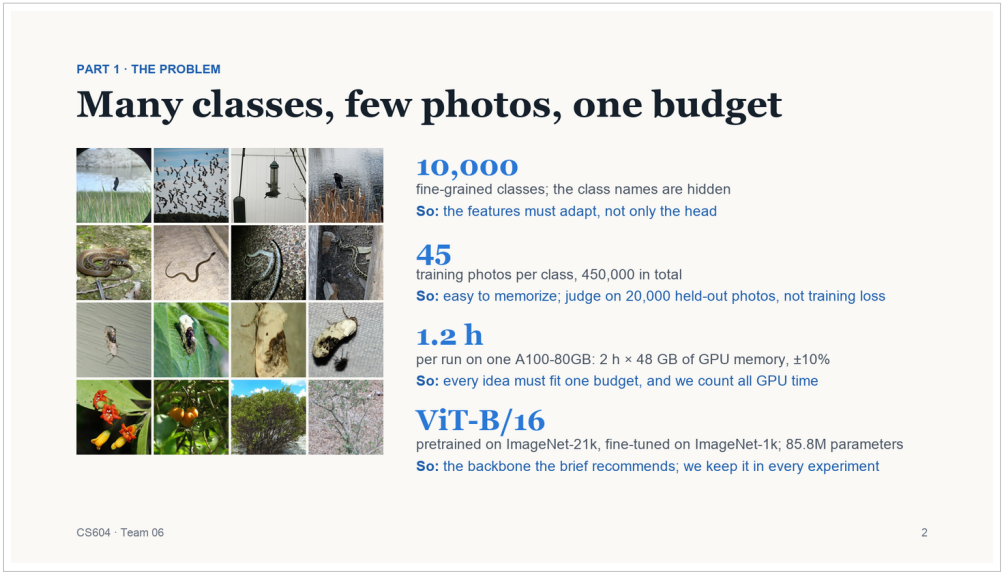
（比起选哪种方法，有一个设计选择更重要：新的 10,000 类 head 怎么开始。）

容易搞错

封面和最后一页上的名字目前还是 [Name A]–[Name D] 占位符，讲之前一定要换成真名。

1.2　第 2 页：类别多、照片少、预算固定（问题分析）

主讲：A



第 2 页缩略图

这一页主要讲什么

在选任何方法之前，先分析这个任务难在哪里。页面右边列了 4 个“难点”，每个后面都跟一句“So:”（所以我们要怎么做）：

1. **10,000 个细粒度类别，类名被隐藏**。So：特征本身也要调整，不能只换 head。原因是预训练模型学的是 ImageNet 那种“大类”的区别（猫、狗、汽车），没见过这么细的区别。
2. **每类只有 45 张训练照片，共 450,000 张**。So：很容易“背答案”，所以用 20,000 张留出的照片来判断好坏，而不看训练 loss。
3. **每次训练 1.2 小时**（一块 80 GB 的 A100；规则是 2 小时 × 48 GB 显存，允许 ±10%）。So：每个想法都要在同一个预算里完成，并且所有 GPU 时间都要算进去。
4. **ViT-B/16**：先在 ImageNet-21k 上预训练、再在 ImageNet-1k 上微调，8,580 万个参数。So：用课程文档推荐的 backbone，所有实验都不换。

想解决什么问题，为什么需要

这是老师邮件里第一条评分标准：“你们怎么分析问题”。每一个难点都直接引出一个设计决定，后面的实验都是围绕这些决定展开的。

关键概念

- **细粒度**（fine-grained）：类别之间差别很细微。
- **背答案 / 过拟合**（memorization / overfitting）：只记住训练照片，不会举一反三。
- **Holdout**：从训练集里每类留出 2 张，共 20,000 张，不参与训练，专门用来打分。它和 CodaBench 的分数相差约 0.3 分以内，所以整个演讲里的比较都用它。
- **预算**：$2\,\text{h} \times 48\,\text{GB} = 96\,\text{GB}\cdot\text{h}$；我们的 GPU 是 80 GB，所以 $96/80 = 1.2$ 小时。计时包括：给 head 提取特征、训练、训练中在 holdout 上评估。
- **Backbone**：模型主体。课程推荐的版本，我们保留了它，并且检查过“只在 ImageNet-21k 上训练的版本”并不更好（第 4 页最后一行）。

图怎么看

左边是 16 张训练照片，每一行是一个类别的 4 张照片：第一行是一种鸟（望远镜里看到的、一群、在喂食器上、在芦苇上），第二行是一种蛇，第三行是一种蛾子，第四行是一种植物（花、果实、整棵灌木、树枝）。它说明**同一类的照片差别很大**：同一个类别可能是一只鸟，也可能是一群鸟；可能是一朵花，也可能是一整棵灌木。只有 45 张这么不一样的照片，模型很难学，也很容易背答案。

与前后页的联系

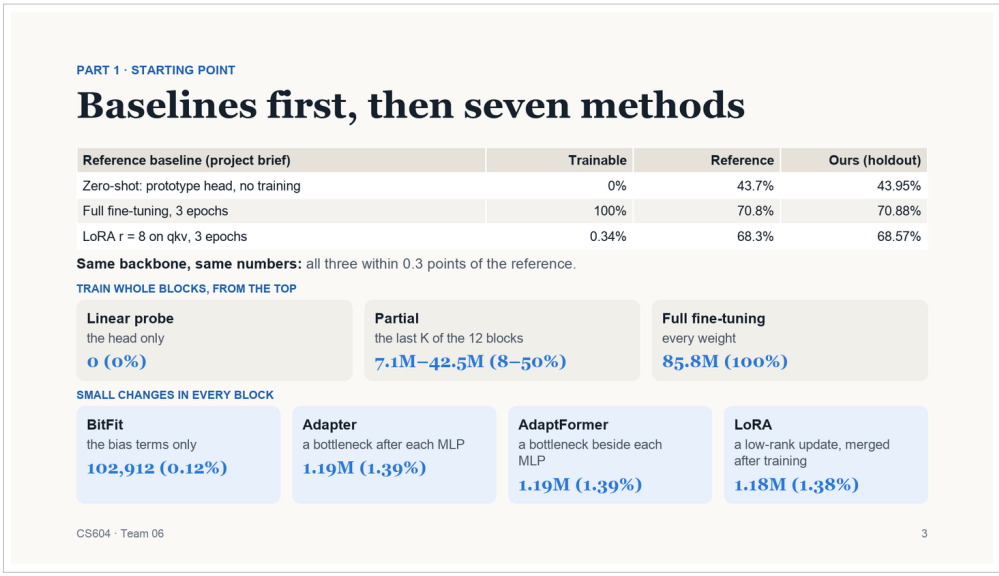
“容易背答案”这个风险，在第 6–8 页变成了我们最重要的发现；“预算”决定了第 9 页训练几轮、第 13 页分辨率能不能用；“backbone 不换”在第 4 页有检查结果。下一页先复现基线，证明我们的代码是对的。

讲这一页时要抓住的一句话

Before choosing any method, we asked what makes this problem hard, because each difficulty points to a design choice.
（在选方法之前，我们先问这个问题难在哪里，因为每一个难点都指向一个设计选择。）

1.3　第 3 页：先复现基线，再比较七种方法

主讲：A



第 3 页缩略图

这一页主要讲什么

两件事。

第一，复现参考结果。项目文档里给了三个参考数字（一位博士生跑的）。我们用自己的代码重新跑一遍，看能不能得到同样的结果：

基线	训练的参数比例	参考结果	我们（holdout）
Zero-shot：原型 head，完全不训练	0%	43.7%	43.95%
全量微调，3 轮	100%	70.8%	70.88%
LoRA（r = 8，只加在 qkv 上），3 轮	0.34%	68.3%	68.57%

三个都在 0.3 分以内（Same backbone, same numbers）。

为什么全量微调在这里是 70.88%，到第 5 页变成 73.38%?

复现基线时（9 月），全量微调和 LoRA 用的还是**线性探针 head**，这是当时的默认做法；zero-shot 用的是不训练的原型 head。第 5 页的七种方法比较全部换成了**原型 head**，所以全量微调从 70.88% 升到 73.38%。这 2.5 分的差距本身就是第 6–8 页要讲的发现。

第二，介绍七种适配方法，分成两大类：

- **从顶层开始训练整块**（Train whole blocks, from the top）：线性探针（只训练 head，backbone 参数 0 个）、部分微调（训练最后 K 个 block，710 万到 4,250 万个参数，即 8%–50%）、全量微调（训练所有参数，8,580 万个，100%）。
- **在每一层做小改动**（Small changes in every block）：BitFit（只训练偏置，102,912 个，0.12%）、Adapter（在每个 MLP 后面加一个“瓶颈”小模块，119 万，1.39%）、AdaptFormer（瓶颈小模块放在 MLP 旁边，119 万，1.39%）、LoRA（学一个低秩的更新，训练完合并回原权重，118 万，1.38%）。

想解决什么问题，为什么需要

- **复现基线**是为了证明“我们的流程是对的”。如果连参考结果都跑不出来，后面的任何提升都可能只是 bug。三个结果都差不到 0.3 分，说明代码、数据、评估都没问题。
- **分成两类**是为了后面（第 5 页）回答一个问题：可训练的参数应该“集中放在顶层”，还是“分散到每一层”？

关键概念

- **Zero-shot**（零样本）：完全不训练，直接用原型 head 预测。这里 43.95% 是“什么都不训练”的起点。
- **可训练参数比例**：训练的参数占 backbone 8,580 万个参数的百分比。比例越小，越“参数高效”。注意：10,000 类的 head（769 万个参数）在所有方法里都要训练，这个比例不算 head。
- **七种方法**：一句话版本见上面；详细原理见第 2 章，备用页第 16–20 页有其中 5 种方法的结构图。

与前后页的联系

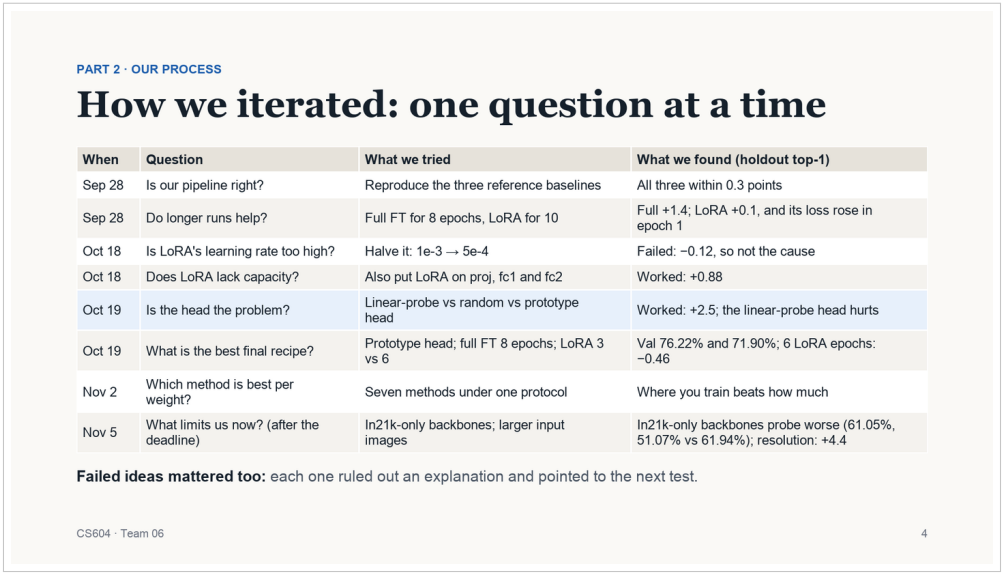
所有七种方法都用同一个“协议”（protocol）：同样的原型 head、3 轮、同样的 holdout、同样的 GPU，所以数字可以直接比。结果在第 5 页。第 4 页先讲我们是怎么一步一步做实验的。

讲这一页时要抓住的一句话

Before trying anything new, we reproduced the three reference numbers in the brief, so our pipeline matches the reference.
（在尝试新东西之前，我们先复现了文档里的三个参考结果，证明我们的流程和参考一致。）

1.4　第 4 页：我们怎么迭代——一次只问一个问题

主讲：B



PART 2 · OUR PROCESS

How we iterated: one question at a time

When	Question	What we tried	What we found (holdout top-1)
Sep 28	Is our pipeline right?	Reproduce the three reference baselines	All three within 0.3 points
Sep 28	Do longer runs help?	Full FT for 8 epochs, LoRA for 10	Full +1.4; LoRA +0.1, and its loss rose in epoch 1
Oct 18	Is LoRA's learning rate too high?	Halve it: 1e-3 → 5e-4	Failed: −0.12, so not the cause
Oct 18	Does LoRA lack capacity?	Also put LoRA on proj, fc1 and fc2	Worked: +0.88
Oct 19	Is the head the problem?	Linear-probe vs random vs prototype head	Worked: +2.5; the linear-probe head hurts
Oct 19	What is the best final recipe?	Prototype head; full FT 8 epochs; LoRA 3 vs 6	Val 76.22% and 71.90%; 6 LoRA epochs: −0.46
Nov 2	Which method is best per weight?	Seven methods under one protocol	Where you train beats how much
Nov 5	What limits us now? (after the deadline)	In21k-only backbones; larger input images	In21k-only backbones probe worse (61.05%, 51.07% vs 61.94%); resolution: +4.4

Failed ideas mattered too: each one ruled out an explanation and pointed to the next test.

CS604 · Team 06　4

第 4 页缩略图

这一页主要讲什么

一张时间表，记录了 8 轮实验。每一行有四列：时间（When）、想回答的问题（Question）、做了什么实验（What we tried）、发现了什么（What we found，用 holdout 准确率）。逐行翻译：

1. **9 月 28 日**：流程对不对？→ 复现三个参考基线 → 三个都在 0.3 分以内。
2. **9 月 28 日**：训练更久有没有用？→ 全量微调训 8 轮，LoRA 训 10 轮 → 全量 +1.4；LoRA 只 +0.1，而且它的 loss 在第 1 轮里反而上升了（异常信号！）。
3. **10 月 18 日**：是不是 LoRA 的学习率太高？→ 减半：$10^{-3} \to 5 \times 10^{-4}$ → **失败**：−0.12，说明不是这个原因。
4. **10 月 18 日**：是不是 LoRA 的容量不够？→ 把 LoRA 也加到 proj、fc1、fc2 上 → **有效**：+0.88。
5. **10 月 19 日**：是不是 head 有问题？→ 比较线性探针 head、随机 head、原型 head → **有效**：+2.5；线性探针 head 有害。（这一行在幻灯片上被高亮，是最大的发现。）
6. **10 月 19 日**：最终配方是什么？→ 原型 head；全量微调 8 轮；LoRA 比较 3 轮和 6 轮 → 验证集 76.22% 和 71.90%；LoRA 训 6 轮反而 −0.46。
7. **11 月 2 日**：按参数量算，哪种方法最好？→ 七种方法用同一个协议 →“在哪里训练”比“训练多少”更重要。
8. **11 月 5 日**（截止之后）：现在还被什么限制？→ 只在 ImageNet-21k 上训练的 backbone；更大的输入图片 → 这两个 backbone 的线性探针更差（61.05%、51.07%，我们的是 61.94%）；分辨率 +4.4。

页面底部一句话：Failed ideas mattered too: each one ruled out an explanation and pointed to the next test.（失败的想法也很重要：每一个都排除了一种解释，并指向下一个实验。）

想解决什么问题，为什么需要

老师的评分标准里有“你们怎么一轮一轮做实验”“怎么发现和验证新想法”“失败的实验和教训”。这一页就是对这三条的直接回答。它展示的是**科学方法**：观察到异常 → 提出假设 → 设计实验验证 → 根据结果提出下一个问题。

关键概念

- **假设**（hypothesis）：对现象的一个可能解释，例如“LoRA 的 loss 上升是因为学习率太高”。
- **控制变量**（one change at a time）：每次只改一个设置，其他都不变。这样结果的变化才能归因到这一个改动上。
- **失败的实验**：结果和预期不一样的实验。它同样有价值，因为它排除了一个错误的解释。
- **消融实验**（ablation）：把某个部分拿掉或换掉，看结果变化多少，从而知道这个部分的作用。

生活类比

就像医生看病：病人发烧（LoRA 的 loss 上升）。医生先怀疑是感冒，开了感冒药（把学习率减半），没用，说明不是感冒；再怀疑是别的原因，做检查（比较三种 head），终于找到病因（线性探针 head）。没用的那一步也很重要，它排除了一个错误的诊断。

与前后页的联系

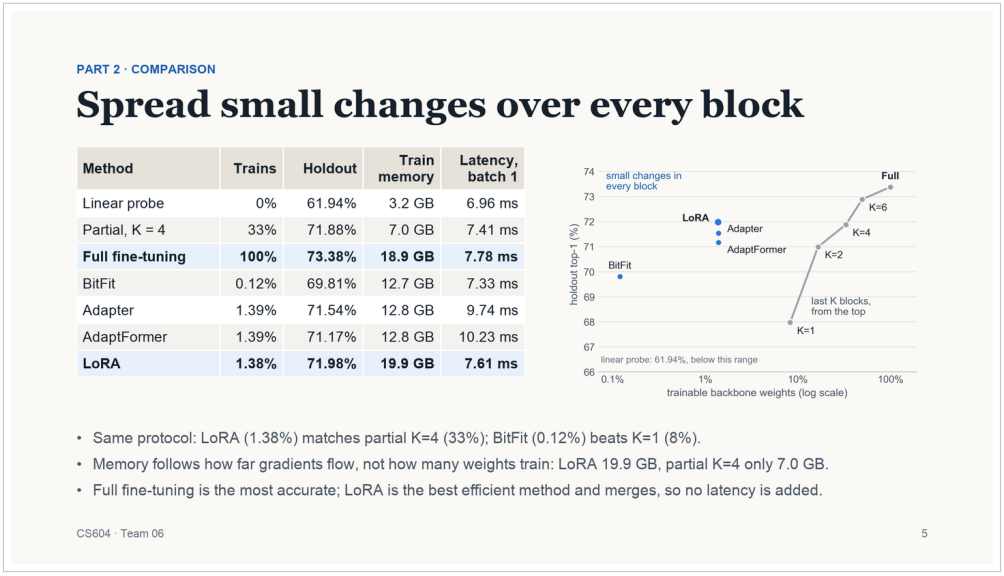
这张表是整个演讲的“目录”：第 5 页展开第 7 行（七种方法），第 6–8 页展开第 2、3、5 行（head 问题），第 9 页展开第 4、6 行（各个设置的影响），第 13 页展开最后一行（分辨率）。

讲这一页时要抓住的一句话

One question per round, and each answer chose the next experiment.
（每一轮只问一个问题，每个答案决定下一个实验做什么。）

1.5　第 5 页：把小改动分散到每一层（七种方法的比较）

主讲：B



第 5 页缩略图

这一页主要讲什么

七种方法在同一个协议下的成绩。左边是表格：方法、训练了多少比例的参数（Trains）、holdout 准确率、训练时的显存（Train memory）、单张图片的推理时间（Latency, batch 1）。右边是一张图：横轴是可训练参数的比例（对数刻度），纵轴是 holdout 准确率。底部三句结论：

1. 同样的协议下，LoRA（1.38%）和部分微调 K=4（33%）一样好；BitFit（0.12%）比 K=1（8%）还好。
2. 显存取决于梯度要往回传多深，而不是训练了多少参数：LoRA 要 19.9 GB，部分微调 K=4 只要 7.0 GB。
3. 全量微调最准；LoRA 是最好的高效方法，而且能合并进权重，不增加推理时间。

想解决什么问题，为什么需要

回答“按每个参数算，哪种方法最划算”，这是老师说的“重点研究适配方法”的核心。它也回答了第 3 页提出的问题：参数应该集中在顶层，还是分散到每一层？

关键概念

- **“在哪里训练”比“训练多少”更重要**：灰线（从顶层开始训练整块）爬得很慢：训练 1 个 block（8% 的参数）只到 68%，训练 6 个 block 到 72.9%，全部训练到 73.4%。蓝点（每一层做小改动）都在灰线的左上方：用很少的参数就达到了很高的准确率。
- **显存和梯度传播**：训练时，为了算梯度，要把前向计算的中间结果存起来。梯度要往回传到第

几层，就要存到第几层。部分微调 K=4 只需要往回传 4 个 block；而 LoRA、BitFit、Adapter 在每一层都有可训练参数，梯度必须一路传回第 1 层。LoRA 和全量微调一样，要保存每个线性层的输入（用来算矩阵 A 的梯度），再加上 96 个额外小矩阵乘法的中间结果，所以它的显存（19.9 GB）甚至比全量微调（18.9 GB）还多。全量微调多出来的优化器状态只有大约 1 GB。

- **合并**（merge）：LoRA 训练完可以把学到的更新加回原来的权重里，模型结构和原来一模一样，所以不增加推理时间。Adapter 和 AdaptFormer 是额外的层，不能合并，所以每张图多 2–3 毫秒。
- **对数刻度**（log scale）：横轴上 0.1%、1%、10%、100% 之间的距离相等，每一格是 10 倍。这样 0.12% 和 100% 才能画在同一张图上。

容易搞错

延迟那一列里，同样结构的模型也会有 0.5 毫秒左右的波动（比如线性探针 6.96 毫秒、全量微调 7.78 毫秒，结构其实一样）。只有 Adapter 和 AdaptFormer 多出来的 2–3 毫秒是真实的变慢。

图怎么看

找到灰线上的 K=4 那个点（约 33%，71.88%），再往左水平看，会看到 LoRA（约 1.4%，71.98%）几乎在同一高度。横向距离是 24 倍的参数差距，纵向几乎没有差距。这就是“在哪里训练比训练多少更重要”的证据。详细读图方法见第 4 章。

与前后页的联系

上承第 3 页的两类方法；下一页转向我们在实验中发现的一个问题：head 的初始化。

讲这一页时要抓住的一句话

Where the trainable weights sit matters more than how many there are: a little capacity in every block beats a lot in a few.
（可训练参数放在哪里，比有多少更重要：每一层都有一点点，胜过少数几层有很多。）

1.6　第 6 页：线性探针 head——起点高，终点低

主讲：B



第 6 页缩略图

这一页主要讲什么

我们在实验中发现的一个问题。上面两张图分别是全量微调和 LoRA（加在所有线性层上）训练 3 轮的过程，横轴是轮数（最左边的“head only”表示还没开始训练、只有 head 的时候），纵轴是 holdout 准确率。橙线是“线性探针 head”，蓝线是“原型 head”。

- **训练前**：线性探针 head 已经有 62.1%，原型 head 只有 44.0%，前者领先 18 分。
- **训练 3 轮后**：全量微调 70.9% 对 73.4%，LoRA 69.6% 对 72.0%，线性探针 head 反而落后约 2.5 分。

底部三句：

1. **推荐的做法 LP-FT**（Kumar 等人，ICLR 2022）：先在冻结的特征上训练一个线性探针 head，再微调整个模型。
2. **起点高，终点低**：训练前领先 18 分，训练后落后 2.5 分。
3. **第一个怀疑对象**：LoRA 的学习率。减半之后没有变化（−0.12），所以我们转去检查 head。

想解决什么问题，为什么需要

这一页展示“我们怎么发现问题”：先看到异常（LoRA 的 loss 在第 1 轮上升），先怀疑学习率（失败），再怀疑 head（成功）。它也揭示了一个反直觉的现象：**起点好不代表终点好**。

关键概念

- **线性探针**（linear probe，LP）：把 backbone 冻结（不训练），只用它提取的特征训练一个线性分类器（head）。训练很快。
- **LP-FT**：Linear Probe then Fine-Tune。论文认为，如果直接用随机 head 微调，一开始的大梯度会“破坏”预训练好的特征；所以先训练一个好的 head，再一起微调。这是很多人默认使用的做法。
- **Head 初始化**：新 head 的参数一开始设成什么。三种选择：随机、线性探针、原型（第 7 页）。
- **“head only”点**：训练前、只有初始化好的 head 时的准确率。

生活类比

两个学生考前拿到同一套练习题。学生甲先把练习题的答案全背下来了（线性探针 head 在训练照片上已经拟合得很好），模拟考一开始分数很高；学生乙只是大概知道每类题的特点（原型 head）。到了真正学习的阶段，甲觉得“我都会了”，学得不认真（梯度信号弱），最后真正考试反而不如乙。

容易搞错

不是说 LP-FT 这篇论文错了。它的目标是保护预训练特征、让模型在**其他数据分布**上更稳。我们的任务恰恰需要特征改变很多（第 2 页：特征必须适应），而且线性探针是在**同一批训练照片**上训练的，每类只有 43–45 张，这时它就变成了负担。

与前后页的联系

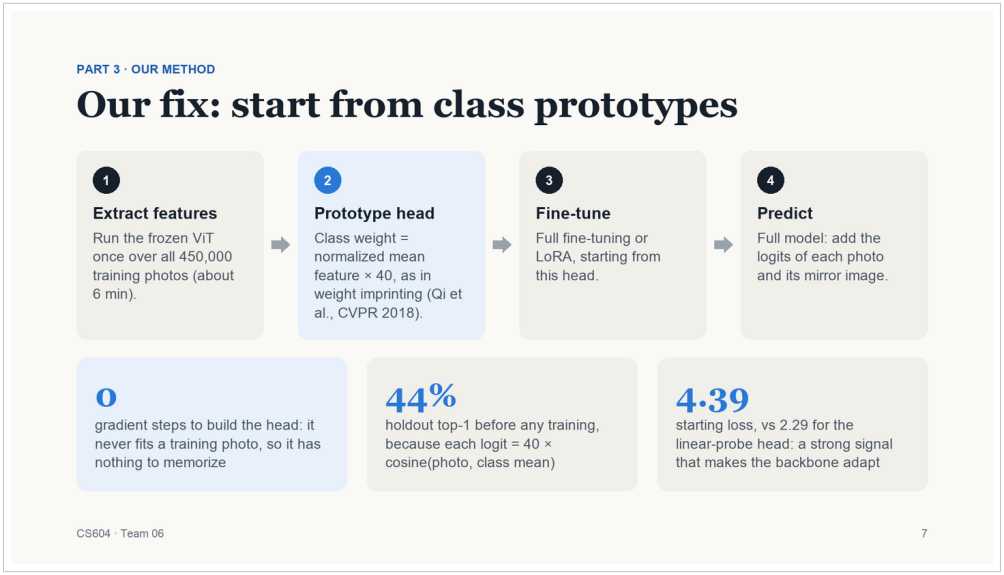
上承第 4 页的“head 有问题吗？”那一行；下一页给出解决办法；第 8 页解释原因。

讲这一页时要抓住的一句话

The linear-probe head starts 18 points ahead, but after three epochs it is about 2.5 points behind.
（线性探针 head 起点领先 18 分，但训练 3 轮后反而落后约 2.5 分。）

1.7　第 7 页：我们的解决办法——从类别原型出发

主讲：C



第 7 页缩略图

这一页主要讲什么

上面是 4 步流程：

1. **提取特征**：用冻结的 ViT 把 450,000 张训练照片都过一遍（约 6 分钟），每张照片得到 768 个数字（特征）。
2. **原型 head**：每个类别的权重 = 这一类所有照片特征的“平均方向”（先把每个特征归一化，再求平均、再归一化），再乘以 40。这个方法叫 **weight imprinting**（权重印刻，Qi 等人，CVPR 2018）。
3. **微调**：从这个 head 出发，做全量微调或 LoRA。
4. **预测**：全量微调模型把每张照片和它的左右镜像各算一次分数，加起来再取最高（flip TTA）。

下面三个数字卡片：

- **0**：建 head 用了 0 步梯度下降。它从来没有去“拟合”任何一张训练照片，所以没有东西可以背。
- **44%**：训练前在 holdout 上的准确率，因为每个分数 = 40× 照片特征和类别平均方向的余弦相似度。
- **4.39**：起始 loss；线性探针 head 是 2.29。loss 高说明“还有很多要学”，给 backbone 一个强的学习信号。

想解决什么问题，为什么需要

线性探针 head 的问题在于它已经在训练照片上拟合过了。我们需要一个“有用但没有背过答案”的起点：既知道每个类别大概长什么样（比随机好），又没有针对任何一张训练照片调过参数。

关键概念

- **特征**（feature）：backbone 把一张图变成的 768 个数字，可以看成这张图的“描述”。
- **原型**（prototype）：一个类别所有照片特征的平均，代表这个类别的“典型样子”。

- **归一化**（L2 normalization）：把一串数字缩放成长度为 1，只保留“方向”，不管“大小”。
- **余弦相似度**（cosine similarity）：两个方向有多一致，范围 −1 到 1。越接近 1 越像。
- **温度 / 分数倍数 40**（temperature, logit scale）：余弦相似度只在 −1 到 1 之间，差别太小，softmax 之后各类概率几乎一样。乘以 40 把差别放大。40 是从 5、10、15、20、30、40、60、80 中选的，选的标准是在 10,000 个训练特征上 loss 最小。
- **Weight imprinting**：这不是我们发明的方法。我们的新发现是：在我们这种“类别多、每类照片少”的设置下，它比推荐的 LP-FT 起点更好。
- **Flip TTA**：测试时增强，用照片和它的镜像各预测一次再合起来，第 2 章有详细解释。

生活类比

认人。线性探针 head 像是把 45 张证件照逐张背下来；原型 head 像是看完 45 张照片后，记住“这个人大概长什么样”（平均印象）。遇到一张新照片时，看它和每个人的“平均印象”有多像，最像的那个就是答案。这个“平均印象”不是对某一张照片死记，所以不会背答案。

三个起始 loss 怎么比较?

这三个数字是 LoRA 训练最开始 100 步的平均训练 loss：

- **随机 head**：**9.28**。完全瞎猜（10,000 类平均分）的 loss 是 $\log 10{,}000 \approx 9.21$，两者几乎一样：随机 head 什么都不知道。
- **原型 head**：**4.39**。比瞎猜低很多，说明它已经知道每类大概长什么样；但离“很有把握”还很远，所以还有很多要学，梯度信号强。
- **线性探针 head**：**2.29**。最低，因为它本来就是在这些训练照片上训练出来的，对它们已经很“熟”。学习信号最弱。

注意：训练时用了标签平滑（第 10 页），所以不能把这些 loss 直接换算成“正确类的概率”，只看相对大小就好。

与前后页的联系

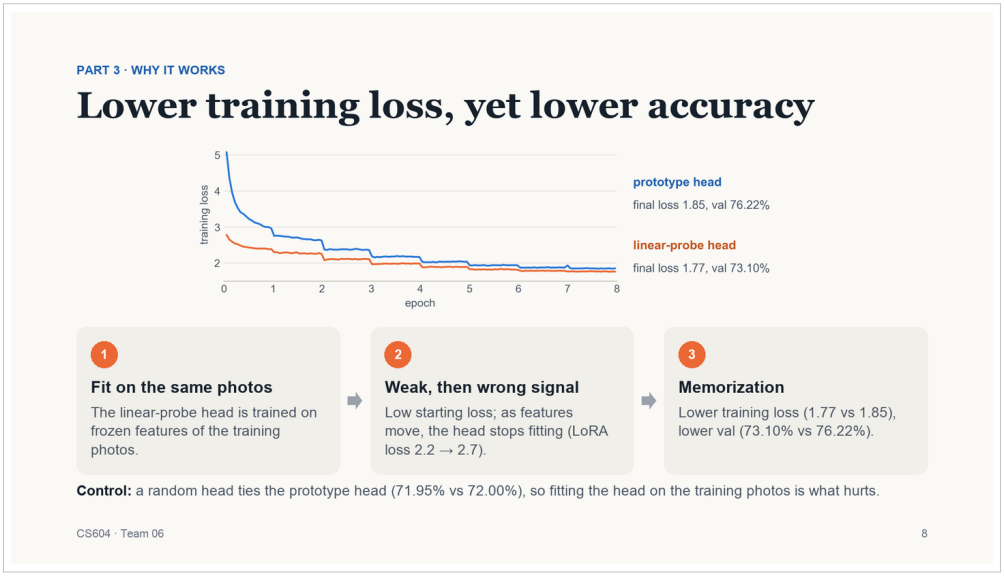
上承第 6 页提出的问题；下一页用 loss 曲线和对照实验证明它为什么有效。

讲这一页时要抓住的一句话

The head never fits any training photo, so it has nothing to memorize, yet it already reaches 44% before any training.
（这个 head 从来没有拟合过任何训练照片，所以没有东西可背，但训练前就已经有 44% 的准确率。）

1.8　第 8 页：训练 loss 更低，准确率反而更低（为什么有效）

主讲：C



第 8 页缩略图

这一页主要讲什么

上面是两次训练的 loss 曲线：都是全量微调、全部数据、8 轮，唯一的区别是 head 的初始化。橙线（线性探针 head）的训练 loss 一直更低，最后是 1.77；蓝线（原型 head）最后是 1.85。但在 CodaBench 验证集上，橙线 73.10%，蓝线 76.22%。**训练 loss 更低，准确率反而更低：这就是背答案。**

中间三步解释：

1. **在同一批照片上拟合**：线性探针 head 是在训练照片的冻结特征上训练出来的。
2. **信号先弱后错**：它的起始 loss 很低，所以 backbone 收到的学习信号很弱；而 backbone 一开始改变，特征就变了，原来拟合好的 head 就不再匹配，所以 LoRA 用线性探针 head 时，训练 loss 在第 1 轮里不降反升（幻灯片上写 2.2 → 2.7，报告里的精确数字是 2.22 → 2.69）。
3. **背答案**：最后训练 loss 更低（1.77 对 1.85），验证集更低（73.10% 对 76.22%）。

底部的对照实验：随机 head 和原型 head 打平（71.95% 对 72.00%）。所以**伤害来自“在训练照片上拟合 head”这件事本身**。

想解决什么问题，为什么需要

光说“原型 head 更好”不够，要解释**为什么**，并且用实验排除其他解释。这就是老师说的“怎么解释结果”。

关键概念

- **训练 loss 和泛化的区别**：训练 loss 只说明模型对训练照片有多“熟”，不能说明它对新照片有多好。
- **学习信号**（梯度的大小）：loss 越大，梯度通常越大，backbone 改变得越多。我们的任务需要 backbone 改变（第 2 页）。
- **对照实验**（control）：为了确认原因，设计一个“只改变一个因素”的实验。随机 head 没有在训练

照片上拟合过，原型 head 也没有；如果两者打平，而线性探针 head 更差，就说明问题出在“拟合过训练照片”。

重点

原型 head 并不神奇。它的价值在于：既不像随机 head 那样从零开始（学得更快：第 1 轮后领先随机 head 5 分，63.3% 对 58.4%），又不像线性探针 head 那样已经背过答案。

图怎么看

两条曲线在每一轮的边界处都会突然下降，在一轮之内比较平。报告认为这是“模型在背这 45 张照片”的迹象：每看完一遍所有照片，再看第二遍时就“认得”它们了。详细见第 4 章。

与前后页的联系

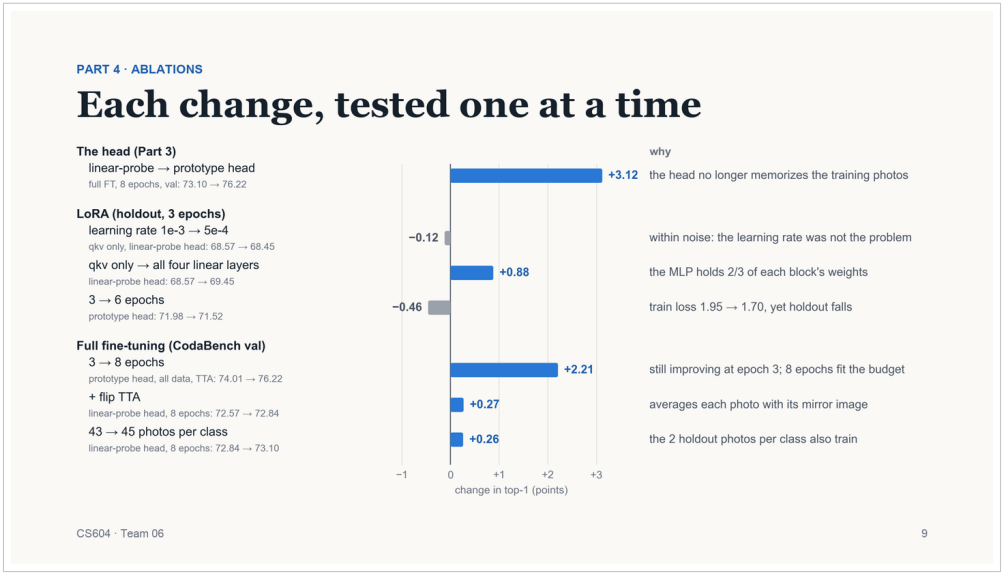
上承第 7 页的方法；下一页把最终配方里的每个设置都按同样的“一次改一个”方法检验一遍。

讲这一页时要抓住的一句话

Lower training loss but lower accuracy: that is memorization. A random head ties the prototype head, so the harm comes from fitting the head on the training photos.
（训练 loss 更低但准确率更低：这就是背答案。随机 head 和原型 head 打平，所以伤害来自在训练照片上拟合 head。）

1.9　第 9 页：每个改动都单独测试（消融实验）

主讲：C



第 9 页缩略图

这一页主要讲什么

一张横向条形图。每一行只改变一个设置，其他都不变；条的长度是准确率变化了多少分；最右边一列写了原因。

改了什么	前 → 后	变化	原因
Head：线性探针 → 原型（全量微调 8 轮，验证集）	73.10 → 76.22	+3.12	head 不再背训练照片
LoRA 学习率 $10^{-3} \to 5 \times 10^{-4}$（holdout）	68.57 → 68.45	−0.12	在噪声范围内，等于没变
LoRA 从只加 qkv → 加到四个线性层	68.57 → 69.45	+0.88	MLP 占每个 block 2/3 的参数
LoRA 3 轮 → 6 轮	71.98 → 71.52	−0.46	训练 loss 从 1.95 降到 1.70，holdout 却下降：过拟合
全量微调 3 轮 → 8 轮（验证集）	74.01 → 76.22	+2.21	第 3 轮时还在进步，8 轮正好在预算内
Flip TTA（验证集）	72.57 → 72.84	+0.27	照片和镜像一起判断
每类 43 张 → 45 张（验证集）	72.84 → 73.10	+0.26	多 2 张照片

想解决什么问题，为什么需要

解释最终配方里**每个设置为什么在那里**，而不是“凭感觉”或“抄别人的”。这正是老师想看的“解释结果”和“实验迭代”。

关键概念

- **消融实验**：一次改一个设置，测量它单独的贡献。

- **噪声**：同样的设置重跑一次，分数也会有一点波动；holdout 有 20,000 张图，0.3 分左右的差异可以看作噪声。所以 −0.12 就是“没变化”；+0.27、+0.26 在噪声边缘，要谨慎看待。
- **全量微调和 LoRA 对“训练更久”的反应相反**：全量微调 3 到 8 轮多了 2.2 分；LoRA 3 到 6 轮反而少了 0.46 分。我们的猜想（还没验证）：LoRA 训练的参数里 87% 是 head（887 万里有 769 万），而 LoRA 的配方给 head 的学习率是全量微调的 10 倍、权重衰减只有 1/500、也没有 drop path，所以 head 最容易背答案。

与前后页的联系

这些结论决定了第 10、11 页两个最终模型的配置：全量微调 8 轮 + TTA + 全部 45 张；LoRA 加在四个线性层、3 轮。

讲这一页时要抓住的一句话

Every row changes one thing and keeps the rest fixed; the right column gives the reason.
（每一行只改一个设置，其他保持不变；最右边一列给出原因。）

1.10　第 10 页：全量微调训练所有参数（最终模型 1）

主讲：D



第 10 页缩略图

这一页主要讲什么

上面是全量微调的结构图：从左到右是图片 → 切块 → 197 个 token → 12 个 Transformer block → 最后的归一化 → [CLS] 特征 → head → 10,000 个分数。蓝色表示要训练，灰色表示冻结；这里全部是蓝色，因为所有参数都训练。下面三张卡片：

- **Model（模型）**：ViT-B/16 的 8,580 万个参数全部训练；head 768 → 10,000，769 万个参数；head 从原型 × 40 开始；每类全部 45 张，224 像素；随机裁剪（面积 25%–100%）+ 左右翻转。
- **Training（训练）**：交叉熵 loss，标签平滑 0.1；AdamW，权重衰减 0.05，梯度裁剪 1.0；学习率 10^{-4}，每往输入方向一层乘 0.75；drop path 0.1；bf16；每批 256 张；8 轮：先 1 轮预热，再按余弦曲线降到 0。

- **Cost and result（成本和结果）**：1.03 GPU 小时；显存峰值 18.9 GB，共 14,056 步；用 flip TTA 时每张图 14.95 毫秒、每秒 1,919 张；验证集 76.22%，测试集 76.09%。

想解决什么问题，为什么需要

把最终模型讲清楚：用了什么 loss、多少参数、什么学习率、花了多少资源。老师和同学常常会问“具体怎么训练的”，这一页就是答案。

关键概念（每个设置为什么这样选）

- **标签平滑**（label smoothing）0.1：不要求模型对正确答案 100% 有把握，减少过度自信和背答案。
- **AdamW**：一种常用的优化器，每个参数有自己的“步长”；W 表示权重衰减和梯度更新分开处理。
- **权重衰减**（weight decay）0.05：每一步都把参数往 0 拉一点点，防止参数变得太大，是一种防止过拟合的方法。
- **梯度裁剪**（gradient clipping）1.0：梯度太大时把它缩小，防止某一步走得太远。
- **逐层学习率衰减**（layer-wise LR decay）0.75：越靠近输入的层，学习率越小。第 1 个 block 只有最大学习率的约 3%（$0.75^{12} \approx 0.03$）。原因：底层学的是边缘、纹理这些通用特征，不需要大改；顶层更任务相关，可以多改。
- **Drop path** 0.1：训练时随机“跳过”一些 block 的计算，让模型不要过度依赖某一部分，是一种正则化。
- **预热 + 余弦**（warmup + cosine）：学习率先从 0 慢慢升到最大，再像余弦曲线一样慢慢降到 0。一开始慢，是为了不让大梯度破坏预训练特征；最后慢，是为了让模型稳定下来。
- **bf16**：用 16 位数字计算，省显存、更快。

这些在第 2 章“训练是怎么进行的”一节里有更详细的解释和例子。

与前后页的联系

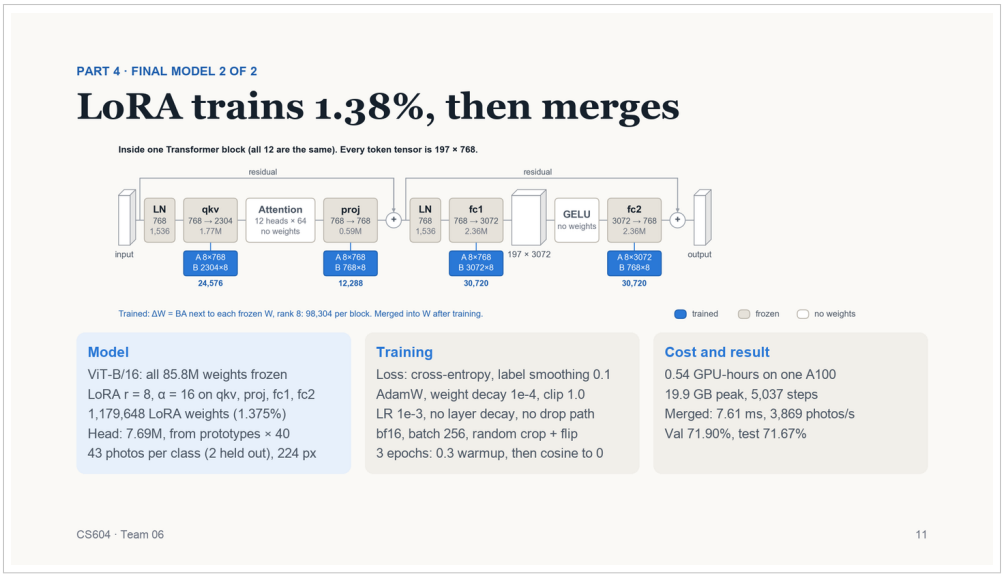
配方来自第 9 页的消融；下一页是另一个最终模型 LoRA；第 12 页汇总两个模型的所有指标。

讲这一页时要抓住的一句话

We train all 85.8 million weights plus the new 7.69 million head, which starts from the class prototypes, with a learning rate that shrinks by 0.75 per block toward the input.
（我们训练全部 8,580 万个参数和新的 769 万参数 head，head 从类别原型开始；学习率每往输入方向一层就乘 0.75。）

1.11　第 11 页：LoRA 只训练 1.38%，训练完再合并（最终模型 2）

主讲：D



第 11 页缩略图

这一页主要讲什么

上面是**一个** Transformer block 的内部结构（12 个 block 都一样）：LayerNorm → qkv → 注意力 → proj → 残差相加 → LayerNorm → fc1 → GELU → fc2 → 残差相加。灰色的原有权重全部冻结；在 qkv、proj、fc1、fc2 这四个线性层旁边各加了一对蓝色的小矩阵 A 和 B，只训练它们。下面三张卡片：

- **Model**：ViT-B/16 的 8,580 万个参数全部冻结；LoRA 秩 r = 8，$\alpha = 16$，加在 qkv、proj、fc1、fc2 上；LoRA 参数共 1,179,648 个（backbone 的 1.375%）；head 769 万个参数，从原型 × 40 开始，也要训练；每类 43 张（留 2 张做 holdout），224 像素。
- **Training**：同样的交叉熵 + 标签平滑 0.1；AdamW，权重衰减 10^{-4}，梯度裁剪 1.0；学习率 10^{-3}，没有逐层衰减，没有 drop path；bf16，每批 256，随机裁剪 + 翻转；3 轮：前 0.3 轮预热，然后余弦下降。
- **Cost and result**：0.54 GPU 小时；显存峰值 19.9 GB，5,037 步；合并后每张 7.61 毫秒、每秒 3,869 张；验证集 71.90%，测试集 71.67%。

想解决什么问题，为什么需要

全量微调每个任务都要存一份完整的模型（374 MB）。如果有很多任务，存储和部署都很贵。LoRA 只训练很少的参数，每个任务只需要存很小的文件，而且训练完可以合并，推理和原模型一样快。课程也要求我们交一个参数高效的模型。

关键概念

- **冻结**（freeze）：参数不参与训练，保持预训练时的值。
- **低秩更新**：不直接改大矩阵 W，而是学一个“改动量”$\Delta W = BA$。A 是 8× 输入维度，B 是输出维度 ×8，两个都很“瘦”。
- **秩 r = 8**：中间那个维度。r 越大，能表达的改动越复杂，参数也越多。
- **α = 16**：缩放系数，实际改动是 $(\alpha/r) \times BA = 2 \times BA$。

- **B 从 0 开始**：所以训练开始时 $BA = 0$，模型和预训练模型完全一样，不会一开始就被破坏。
- **合并**（merge）：训练完计算 $W' = W + 2BA$，替换原来的 W，然后把 A、B 扔掉。模型结构和原来一模一样，所以推理不变慢。

算一算 LoRA 的参数

对一个输入 d_{in} 维、输出 d_{out} 维的线性层，LoRA 的参数是 $r \times (d_{\text{in}} + d_{\text{out}})$：

- qkv（$768 \to 2304$）：$8 \times (768 + 2304) = 24{,}576$
- proj（$768 \to 768$）：$8 \times (768 + 768) = 12{,}288$
- fc1（$768 \to 3072$）：$8 \times (768 + 3072) = 30{,}720$
- fc2（$3072 \to 768$）：$8 \times (3072 + 768) = 30{,}720$

每个 block 合计 98,304，12 个 block 共 $98{,}304 \times 12 = 1{,}179{,}648$，占 backbone 8,580 万的 1.375%。对比：直接训练 qkv 这一个矩阵就要 $768 \times 2304 = 1{,}769{,}472$ 个参数，比 LoRA 在整个模型上的参数还多。

容易搞错

LoRA 的“1.375%”不包括 head。head 有 769 万个参数，LoRA 的可训练参数里其实大部分是 head（887 万里有 769 万）。另外，LoRA 训练时并不省显存（19.9 GB，比全量微调还多），省的是**存储**和**优化器状态**。

与前后页的联系

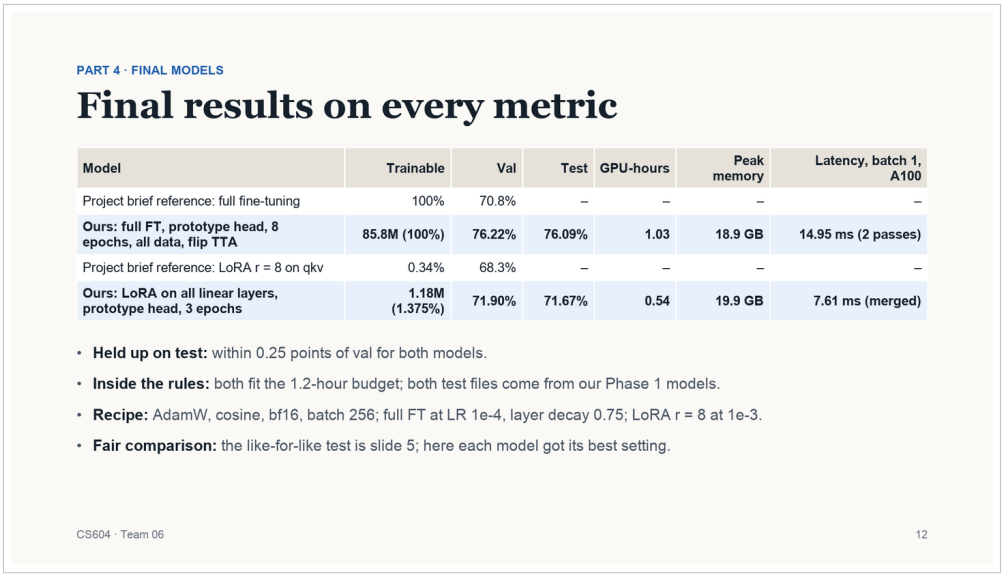
LoRA 只训 3 轮，是因为第 9 页发现 6 轮反而变差；加在四个线性层上，是因为第 9 页发现这样多 0.88 分。下一页把两个模型的所有指标放在一起。

讲这一页时要抓住的一句话

The backbone stays frozen; rank-8 matrices A and B next to qkv, proj, fc1 and fc2 learn the update, and after training the update is merged, so inference is as fast as the base model.
（backbone 冻结；qkv、proj、fc1、fc2 旁边的秩为 8 的矩阵 A、B 学习改动量；训练完合并回去，所以推理和原模型一样快。）

1.12　第 12 页：所有指标上的最终结果

主讲：D


PART 4 · FINAL MODELS

Final results on every metric

Model	Trainable	Val	Test	GPU-hours	Peak memory	Latency, batch 1, A100
Project brief reference: full fine-tuning	100%	70.8%	–	–	–	–
Ours: full FT, prototype head, 8 epochs, all data, flip TTA	**85.8M (100%)**	**76.22%**	**76.09%**	**1.03**	**18.9 GB**	**14.95 ms (2 passes)**
Project brief reference: LoRA r = 8 on qkv	0.34%	68.3%	–	–	–	–
Ours: LoRA on all linear layers, prototype head, 3 epochs	**1.18M (1.375%)**	**71.90%**	**71.67%**	**0.54**	**19.9 GB**	**7.61 ms (merged)**

- **Held up on test:** within 0.25 points of val for both models.
- **Inside the rules:** both fit the 1.2-hour budget; both test files come from our Phase 1 models.
- **Recipe:** AdamW, cosine, bf16, batch 256; full FT at LR 1e-4, layer decay 0.75; LoRA r = 8 at 1e-3.
- **Fair comparison:** the like-for-like test is slide 5; here each model got its best setting.

CS604 · Team 06　12

第 12 页缩略图

这一页主要讲什么

一张总表，列出项目文档要求的所有指标：可训练参数、验证集、测试集、GPU 小时、显存峰值、单张推理延迟（A100）。

模型	可训练	验证集	测试集	GPU 小时	显存	延迟
文档参考：全量微调	100%	70.8%	–	–	–	–
我们：全量微调（原型 head、8 轮、全部数据、flip TTA）	85.8M	76.22%	76.09%	1.03	18.9 GB	14.95 ms
文档参考：LoRA（只在 qkv）	0.34%	68.3%	–	–	–	–
我们：LoRA（四个线性层、原型 head、3 轮）	1.18M	71.90%	71.67%	0.54	19.9 GB	7.61 ms

底部四句：

1. **在测试集上站得住**：两个模型的测试成绩和验证成绩都相差不到 0.25 分。
2. **遵守规则**：两个都在 1.2 小时预算内；两份测试集提交都来自 Phase 1 的同一个模型。
3. **配方**：AdamW、余弦学习率、bf16、每批 256；全量微调学习率 10^{-4}、逐层衰减 0.75；LoRA r = 8、学习率 10^{-3}。
4. **公平比较**：同等条件下的比较在第 5 页；这里每个模型用的是各自最好的设置。

想解决什么问题，为什么需要

给出最终答案，并说明结果可信（测试和验证一致）、合规（预算、规则）、比较公平。

关键概念

- **+5.4 和 +3.6**：$76.22 - 70.8 = 5.42$，$71.90 - 68.3 = 3.6$，都是和文档参考比。
- **测试和验证一致**：$76.22 - 76.09 = 0.13$，$71.90 - 71.67 = 0.23$。说明我们没有“过度针对验证集

调参”，模型在新数据上表现稳定。

- **14.95 ms（2 passes）**：全量微调用了 flip TTA，每张图要算两次（原图和镜像），所以延迟大约是一次的两倍。
- **7.61 ms（merged）**：LoRA 合并后的延迟，和原始 ViT 一样。
- **Phase 2 规则**：课程规定测试阶段必须用 Phase 1 的同一个模型预测。我们照做了。

容易搞错

不要把第 12 页的 76.22 对 71.90 当作“全量微调比 LoRA 好 4.3 分”的公平比较：两者的轮数、数据量、TTA 都不一样。公平比较是第 5 页同样条件下的 73.38 对 71.98（差 1.4 分）。

与前后页的联系

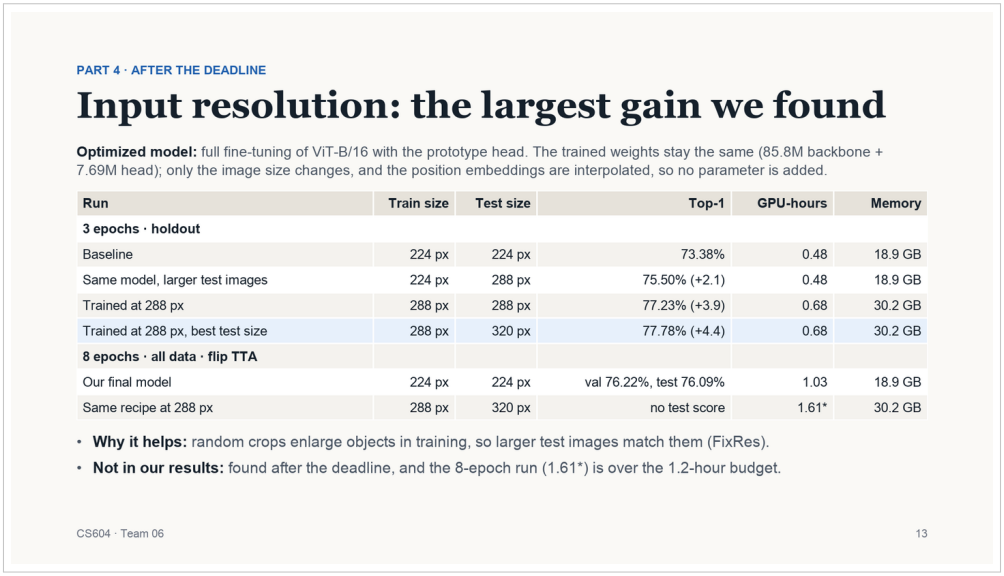
汇总第 10、11 页的两个模型。下一页讲截止日期之后的一个额外发现。

讲这一页时要抓住的一句话

For both models, test is within 0.25 points of val, both fit the budget, and both test files come from our Phase 1 models.
（两个模型的测试成绩都和验证成绩相差不到 0.25 分，都在预算内，两份测试提交都来自 Phase 1 的模型。）

1.13　第 13 页：输入分辨率——我们找到的最大提升（截止之后）

主讲：D


PART 4 · AFTER THE DEADLINE

Input resolution: the largest gain we found

Optimized model: full fine-tuning of ViT-B/16 with the prototype head. The trained weights stay the same (85.8M backbone + 7.69M head); only the image size changes, and the position embeddings are interpolated, so no parameter is added.

Run	Train size	Test size	Top-1	GPU-hours	Memory
3 epochs · holdout					
Baseline	224 px	224 px	73.38%	0.48	18.9 GB
Same model, larger test images	224 px	288 px	75.50% (+2.1)	0.48	18.9 GB
Trained at 288 px	288 px	288 px	77.23% (+3.9)	0.68	30.2 GB
Trained at 288 px, best test size	288 px	320 px	77.78% (+4.4)	0.68	30.2 GB
8 epochs · all data · flip TTA					
Our final model	224 px	224 px	val 76.22%, test 76.09%	1.03	18.9 GB
Same recipe at 288 px	288 px	320 px	no test score	1.61*	30.2 GB

- **Why it helps:** random crops enlarge objects in training, so larger test images match them (FixRes).
- **Not in our results:** found after the deadline, and the 8-epoch run (1.61*) is over the 1.2-hour budget.

CS604 · Team 06　13

第 13 页缩略图

这一页主要讲什么

截止日期之后做的一个实验：只改变输入图片的大小，看准确率变化多少。页面最上面先说明优化的是哪个模型：带原型 head 的 ViT-B/16 全量微调；训练的参数完全一样（8,580 万 backbone + 769 万 head），只是图片变大，位置编码通过插值适应新尺寸，所以没有增加任何参数。

表格（上半部分都是 3 轮、holdout）：

- 基线：训练 224、测试 224：73.38%（0.48 GPU 小时，18.9 GB）。
- 同一个模型，只把测试图片放大到 288：75.50%（+2.1），不用重新训练。
- 用 288 训练、288 测试：77.23%（+3.9；0.68 GPU 小时，30.2 GB）。
- 用 288 训练、320 测试：77.78%（+4.4），最好。

下半部分（8 轮、全部数据、flip TTA）：我们的最终模型（224/224，验证 76.22%、测试 76.09%，1.03 GPU 小时）；同样的配方改用 288 训练、320 测试：**没有测试成绩**，而且要 1.61 GPU 小时，超出预算（表里的 1.61*）。

底部两句：

1. **为什么有效**：训练时的随机裁剪会把物体“放大”，所以测试时用更大的图片，物体的大小才和训练时匹配（FixRes）。
2. **为什么不算进正式结果**：截止日期之后才发现，而且 8 轮版本超出 1.2 小时预算。

想解决什么问题，为什么需要

回答第 4 页最后一行“现在还被什么限制”。也体现了诚实：这个提升很大，但它是截止后才做的，只有 holdout 成绩，我们没有把它说成正式成绩。

关键概念

- **分辨率和 token 数**：图片切成 16 × 16 的块。224 像素是 14 × 14 = 196 块，加 [CLS] 共 197 个 token；288 像素是 18 × 18 = 324 块，共 325 个 token；320 像素是 20 × 20 = 400 块，共 401 个 token。
- **计算量和显存为什么增加**：token 从 197 变成 325，大部分计算多了约 1.7 倍；注意力部分的计算和 token 数的平方成正比，增加得更多。所以显存从 18.9 GB 变成 30.2 GB。
- **位置编码插值**：位置编码原来是 14 × 14 的网格，图片变大后网格变成 18 × 18，用插值（像放大图片一样）把原来的位置编码“拉伸”到新网格上，不需要新参数。
- **FixRes**（Touvron 等人，2019）：训练时随机裁剪一小块再放大到 224，物体看起来比实际大；测试时用的是完整图片缩到 224，物体看起来偏小。测试时把图片放大一点，物体大小就和训练时更接近。

生活类比

你总是用放大镜练习认昆虫（训练时的随机裁剪 = 放大），考试时却只给你一张缩小的全景照片（测试时缩到 224）。如果考试时把照片也放大一点，你认得会更准。

容易搞错

这个结果**没有测试集成绩**，也不是我们的正式结果。被问到时要说清楚：它是截止后的发现，只在 holdout 上测过；3 轮的 288 版本在预算内（0.68 小时），但 8 轮的最终配方用 288 要 1.61 小时，超预算。

与前后页的联系

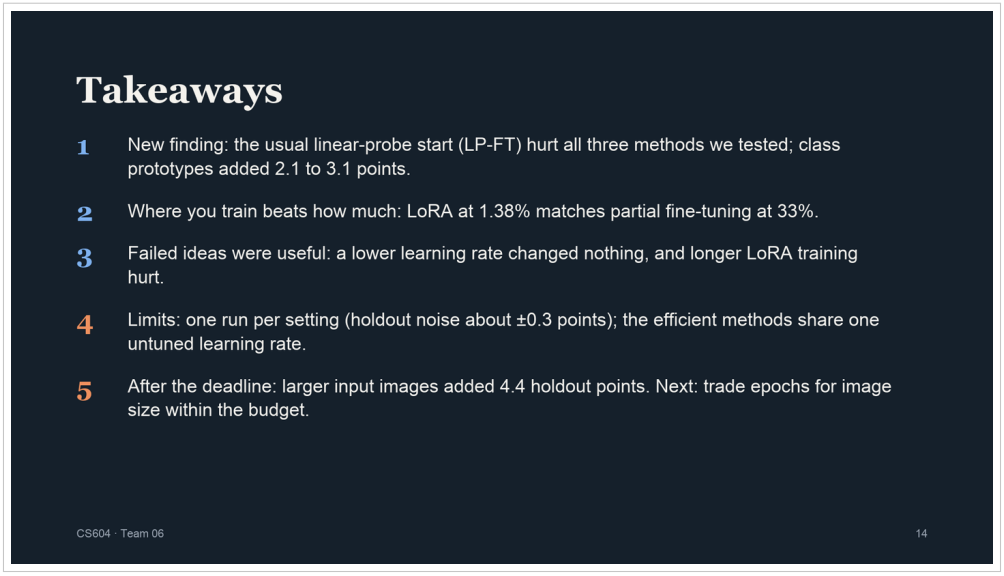
它是第 14 页第 5 条“截止后的教训”的依据：下一步是在预算内用“少一点轮数”换“大一点图片”。

讲这一页时要抓住的一句话

Training at 288 and testing at 320 gives 77.78%, 4.4 points above the baseline, inside the budget, but it was found after the deadline, so it is not part of our results.
（用 288 训练、320 测试得到 77.78%，比基线高 4.4 分，在预算内；但它是截止后才发现的，所以不算正式结果。）

1.14　第 14 页：总结（Takeaways）

主讲：D



第 14 页缩略图

这一页主要讲什么

五条结论，前三条（蓝色数字）是发现，后两条（橙色数字）是局限和下一步：

1. **新发现**：常用的线性探针起点（LP-FT）在我们测试的三种方法上都有害；类别原型多出 2.1 到 3.1 分。（证据：第 6–8 页）
2. **在哪里训练比训练多少更重要**：LoRA 用 1.38% 的参数，追平了训练 33% 参数的部分微调。（证据：第 5 页）
3. **失败的想法也有用**：降低学习率没有变化，LoRA 训练更久反而变差。（证据：第 4、9 页）
4. **局限**：每个设置只跑了一次（holdout 噪声约 ±0.3 分）；几种高效方法共用一个没有单独调过的学习率。
5. **截止后的发现**：更大的输入图片在 holdout 上多 4.4 分。下一步：在预算内用轮数换图片大小。（证据：第 13 页）

想解决什么问题，为什么需要

让听众带走最重要的几点。第 4、5 条回应老师邮件里的“教训”和“诚实记录”：主动说出局限，比被问出来更好。

关键概念

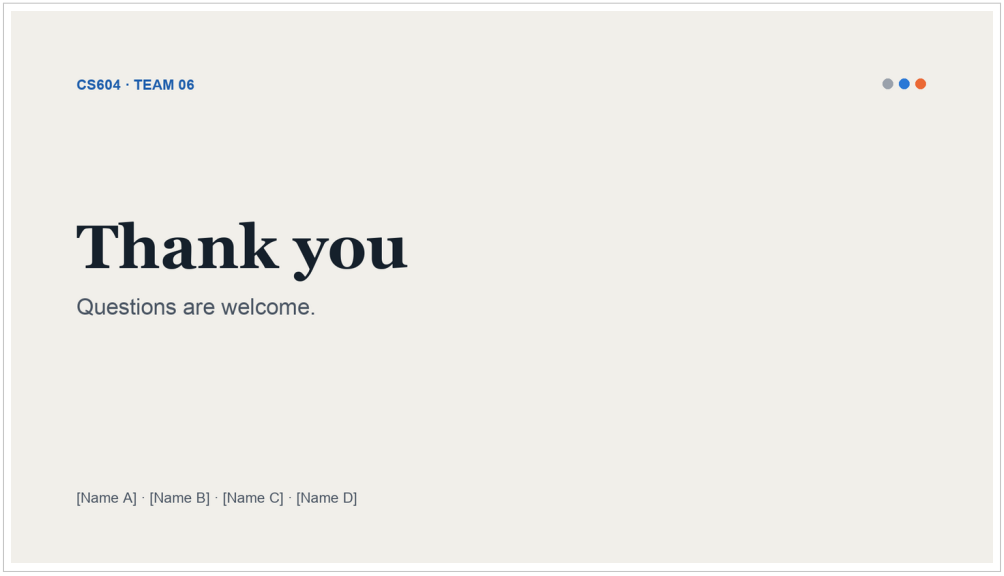
“三种方法”指全量微调、LoRA、AdaptFormer（这三种我们都做了线性探针 head 和原型 head 的对比）。±0.3 分的来源：holdout 有 20,000 张图，准确率在 72% 左右时，统计误差（标准误）约为 0.32 分。

讲这一页时要抓住的一句话

The usual linear-probe start hurt all three methods we tried, while a head built from class prototypes added 2 to 3 points.
（常用的线性探针起点在我们试过的三种方法上都有害，而用类别原型建的 head 多出 2 到 3 分。）

1.15　第 15 页：Thank you（以及 Q&A 分工）

主讲：D



第 15 页缩略图

这一页主要讲什么

结束页，邀请提问。讲稿备注里写了 **Q&A 分工**（不念出来）：

- 原型 head 为什么比线性探针 head 好、证据是什么 → C。
- 为什么 PEFT 不如全量微调、最终比较是否公平、LoRA 为什么训久了变差 → D。
- 方法细节、参数量、延迟、LoRA 和 Adapter 的比较 → B。
- 预算、GPU 小时、backbone、数据和 holdout → A。
- 分辨率是否超预算、局限 → D。

怎么用好这一页

- 回答时先给一句话结论，再给一个数字，最后给原因。
- 如果问题涉及某个具体方法的结构，可以翻到备用页第 16–20 页。
- 不知道的就说“我们没有测过这个，我们的猜想是……”，不要编。

第 5 章有 30 多个可能的问题和英文回答。

讲这一页时要抓住的一句话

Thank you; we are happy to take questions.

1.16　备用页总说明（第 16–20 页）

第 16–20 页是**备用页**（backup slides），正式演讲时不讲，只在被问到某个具体方法时翻出来。它们来自第一版幻灯片，是七种方法里除了全量微调（第 10 页）和 LoRA（第 11 页）之外的另外五种。

每一页的结构都一样：

- **右上角的一排“药丸”**（LP、Partial、Full、BitFit、Adapter、AdaptFormer、LoRA）：七种方法从简单到复杂排成一列，当前方法是实心蓝色，前面讲过的是浅蓝色，后面的是灰色。这是一条“方法阶梯”。
- **结构图**：蓝色是训练的部分，灰色是冻结的部分。前两种方法画的是整个网络，后三种画的是**一个** Transformer block 的内部（12 个 block 都一样）。
- **五行要点**：Idea（想法）、Setting（设置）、Parameters（参数量怎么算）、Result（结果）、Problem → Next（这个方法的问题，以及下一个方法怎么解决它）。

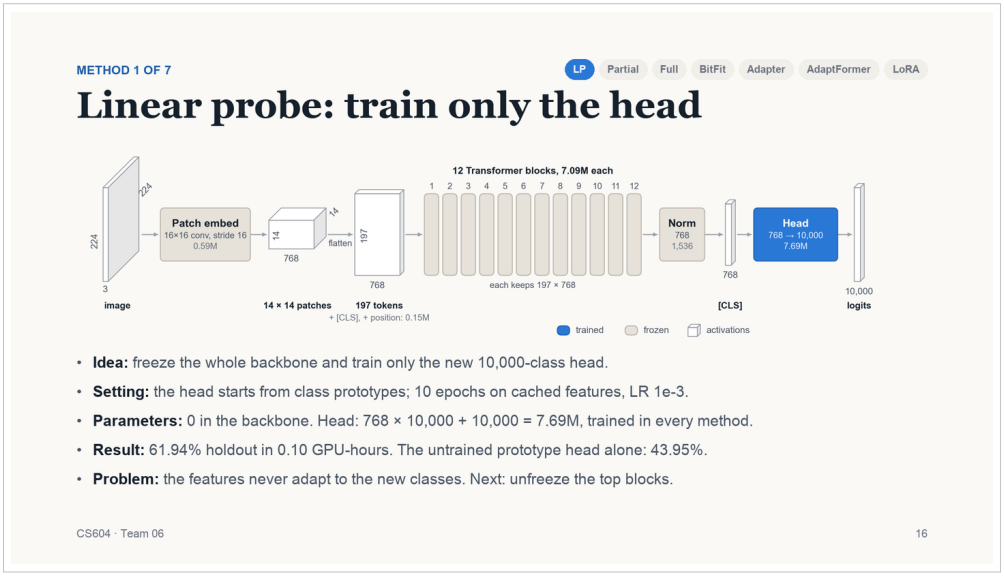
五个备用页连成一个故事

线性探针：特征不变 → 部分微调：只能改顶层，底层不变 → 全量微调：都改，但每个任务要存一整份模型 → BitFit：只改偏置，容量太小 → Adapter：加小模块，但串联在主路上、推理变慢 → AdaptFormer：改成并联的旁路，但还是不能合并 → LoRA：可以合并的更新。

每个方法的“Problem”正好是下一个方法要解决的问题。

1.17　第 16 页（备用）：线性探针——只训练 head

主讲：B（被问到线性探针时）



第 16 页缩略图

这一页讲什么

- **Idea**：冻结整个 backbone，只训练新的 10,000 类 head。
- **Setting**：head 从类别原型开始；在提前算好的特征上训练 10 轮，学习率 10^{-3}。
- **Parameters**：backbone 里 0 个。head 是 $768 \times 10{,}000 + 10{,}000 = 7.69\text{M}$，每种方法都要训练它。
- **Result**：holdout 61.94%，只用 0.10 GPU 小时。不训练的原型 head 是 43.95%。
- **Problem**：特征永远不会针对新类别调整。Next：解冻顶部几个 block。

关键概念

提前算好的特征（cached features）：因为 backbone 不变，每张图的特征只需要算一次、存起来，之后训练 head 只用这些数字，所以特别快、特别省显存（3.2 GB）。

为什么重要

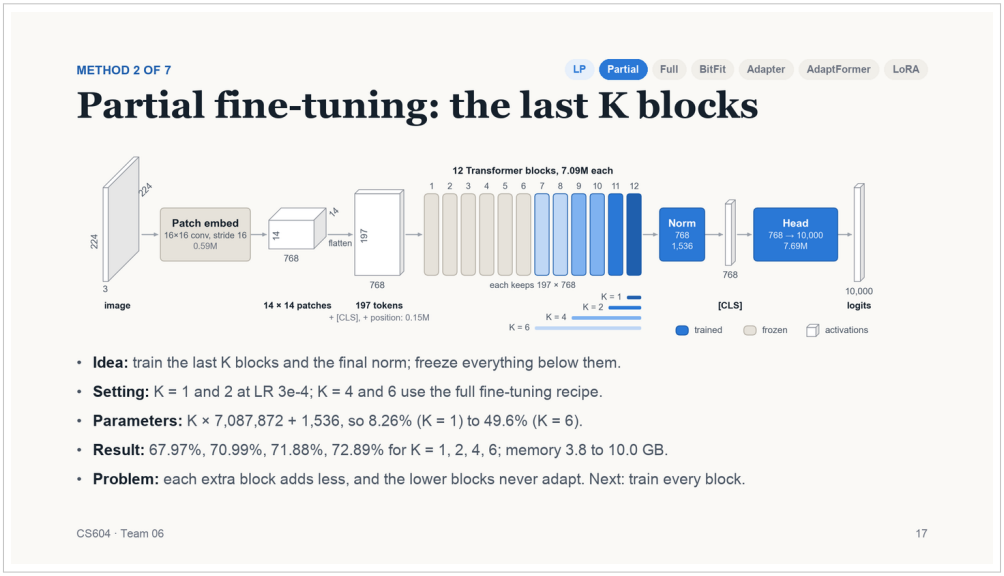
它告诉我们“只用冻结的特征最多能到多少”：61.94%。比全量微调的 73.38% 低 11 分以上，说明这个任务**必须**让特征适应（呼应第 2 页第 1 条）。

容易搞错

第 16 页的“线性探针”是一种**适配方法**（只训练 head）。第 6 页的“线性探针 head”是一种**初始化方式**（用线性探针训练出的 head 作为微调的起点）。两者用的是同一个技术，但用途不同。

1.18　第 17 页（备用）：部分微调——只训练最后 K 个 block

主讲：B（被问到部分微调时）



第 17 页缩略图

这一页讲什么

- **Idea**：训练最后 K 个 block 和最后的归一化层，下面的全部冻结。
- **Setting**：K = 1 和 2 用学习率 3×10^{-4}；K = 4 和 6 用全量微调的配方。
- **Parameters**：$K \times 7{,}087{,}872 + 1{,}536$，从 8.26%（K = 1）到 49.6%（K = 6）。
- **Result**：K = 1、2、4、6 分别是 67.97%、70.99%、71.88%、72.89%；显存 3.8 到 10.0 GB。
- **Problem**：每多训练一个 block，提升越来越小，而且底层永远不变。Next：训练每一个 block。

图怎么看

结构图下面有四条横杠，分别标着 K = 1、2、4、6，表示每种 K 训练的是哪几个 block（从第 12 个往回数）。

关键概念

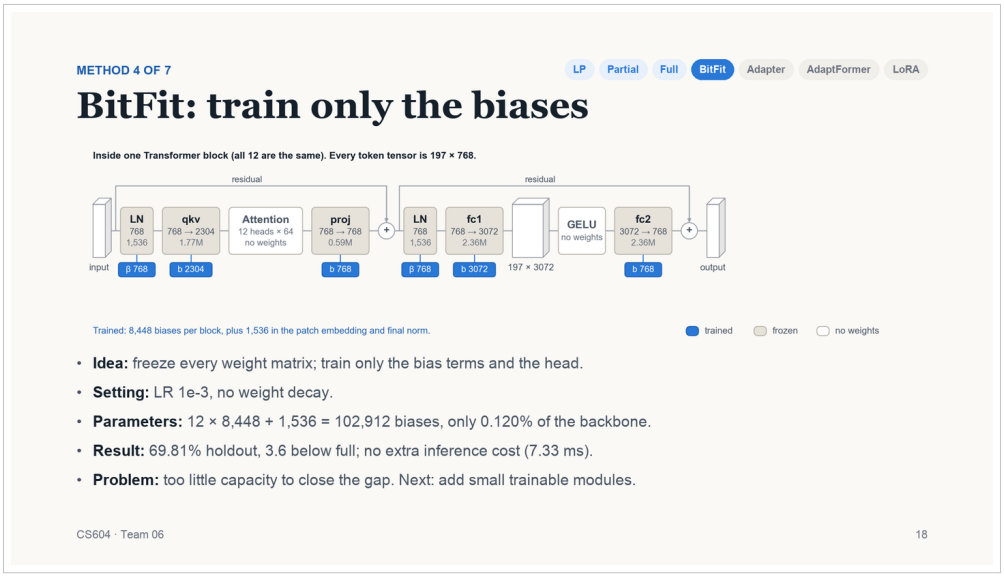
- **收益递减**：第 1 个 block 多 6 分（61.94 → 67.97），第 2 个多 3 分，之后每 2 个 block 只多约 1 分。
- **训练最省**：梯度只需要传到第 $12 - K + 1$ 个 block，所以显存小，K = 6 也只要 10 GB，0.35 GPU 小时。

容易搞错

K = 1、2 和 K = 4、6 用的配方不一样（学习率等），所以 K = 2 到 K = 4 的变化混合了两个因素。这是报告里写明的局限之一。

1.19　第 18 页（备用）：BitFit——只训练偏置

主讲：B（被问到 BitFit 时）



第 18 页缩略图

这一页讲什么

- **Idea**：冻结所有权重矩阵，只训练偏置（bias）项和 head。
- **Setting**：学习率 10^{-3}，没有权重衰减。
- **Parameters**：$12 \times 8{,}448 + 1{,}536 = 102{,}912$ 个偏置，只占 backbone 的 0.120%。
- **Result**：holdout 69.81%，比全量微调低 3.6 分；推理没有任何额外成本（7.33 毫秒）。
- **Problem**：容量太小，追不上。Next：加入小的可训练模块。

关键概念

偏置：神经网络的每个线性层计算 $y = Wx + b$，W 是权重矩阵，b 就是偏置，一个“整体平移”的量。偏置数量很少（每个输出一个），所以 BitFit 的参数极少。从结构图上看，蓝色的小标签就是偏置：两个 LayerNorm 各有 768 个 β，qkv 有 2,304 个，proj 有 768 个，fc1 有 3,072 个，fc2 有 768 个，加起来每个 block 8,448 个；再加上 patch embedding 和最后归一化层的偏置共 1,536 个。

为什么重要

它是“在每一层做小改动”最极端的例子：只用 0.12% 的参数，就比“训练最后一整个 block”（8.26% 的参数，67.97%）还高近 2 分。这是第 5 页“在哪里训练比训练多少更重要”的有力证据。

1.20　第 19 页（备用）：Adapter——在每个 MLP 后面加一个瓶颈

主讲：B（被问到 Adapter 时）



第 19 页缩略图

这一页讲什么

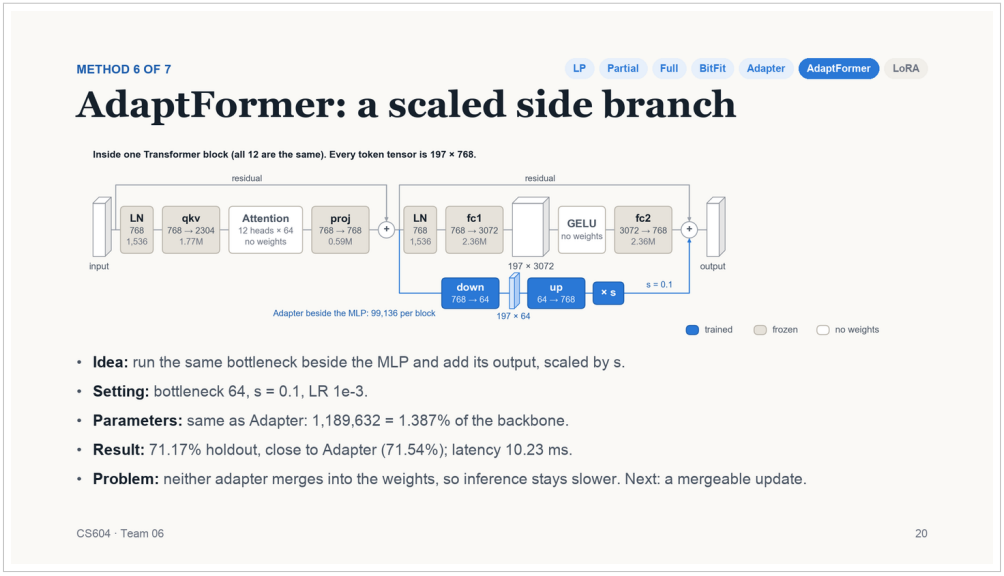
- **Idea**：在每个 MLP 后面串联一个小的“瓶颈”模块：$768 \to 64 \to 768$。
- **Setting**：瓶颈宽度 64，缩放 1.0，学习率 10^{-3}；“升维”那一层初始化为 0。
- **Parameters**：$12 \times (768 \times 64 + 64 + 64 \times 768 + 768) = 1{,}189{,}632$，占 1.387%。
- **Result**：holdout 71.54%；显存 12.8 GB；延迟 9.74 毫秒，原始模型是 7.33 毫秒。
- **Problem**：它串联在主路上，改写了 MLP 的输出。Next：并联的、带缩放的旁路。

关键概念

- **瓶颈**（bottleneck）：先“降维”到 64（把 768 个数字压缩成 64 个），经过 ReLU，再“升维”回 768。中间很窄，像瓶子的颈部，所以参数很少。
- **自己的残差连接**：Adapter 的输出 = 输入 + 瓶颈的结果。因为升维层从 0 开始，一开始瓶颈的结果是 0，模型和原来完全一样。
- **为什么推理变慢**：Adapter 是额外的一层，而且中间有非线性（ReLU），不能像 LoRA 那样合并进原来的权重，每次推理都要多算一遍，所以单张图多 2.4 毫秒左右。

1.21　第 20 页（备用）：AdaptFormer——带缩放的旁路

主讲：B（被问到 AdaptFormer 时）



第 20 页缩略图

这一页讲什么

- **Idea**：把同样的瓶颈模块放在 MLP **旁边**（并联），把它的输出乘以 s 再加回去。
- **Setting**：瓶颈 64，$s = 0.1$，学习率 10^{-3}。
- **Parameters**：和 Adapter 一样：1,189,632，占 1.387%。
- **Result**：holdout 71.17%，和 Adapter（71.54%）接近；延迟 10.23 毫秒。
- **Problem**：两种 adapter 都不能合并进权重，所以推理一直更慢。Next：一个可以合并的更新（LoRA）。

关键概念

- **串联和并联**：Adapter 接在 MLP 输出后面（串联，信息必须经过它）；AdaptFormer 从注意力部分之后（也就是 MLP 的入口）另开一条小路，和 MLP 并排走，最后加到一起（并联，原来的 MLP 输出保持不变）。
- **缩放** $s = 0.1$：旁路的输出只乘 0.1 再加进去，让它的影响温和一些。

容易搞错

Adapter（71.54%）和 AdaptFormer（71.17%）只差 0.37 分，和噪声差不多大；而且两者的缩放也不同（1.0 和 0.1），所以我们不能说“串联比并联好”，只能说在我们的设置里位置差别不大。

备用页一句话总结

七种方法里，只有 LoRA 同时做到：参数少（1.38%）、准确率高（71.98%，高效方法里最高）、推理不变慢（可以合并）。代价是训练时显存最大、速度最慢。

第 2 章　方法与模型详解

这一章是整本手册的“教材”部分。第 1 章按页码讲了每一页 slide 说了什么；这一章换一个角度，把 slides 里出现的**每一个模型和方法**从零讲清楚：它是什么、为什么需要它、内部怎么工作、有什么优缺点、和别的方法有什么区别。读完这一章，即使老师问到 slides 上没有直接写的原理，你也能从“为什么”出发回答。

本章的顺序和整个项目的思路一致：

1. 先认识我们用的模型 ViT-B/16（2.1 节）：图片进去，10,000 个分数出来，中间发生了什么。
2. 再看最关键的新零件——分类头（head），以及它的三种“起点”（2.2 节）。这是我们项目最重要的发现所在。
3. 然后逐个讲七种 adaptation（适配/微调）方法（2.3 节），重点是 LoRA。
4. 接着讲训练和评估的细节（2.4、2.5 节）：损失函数、优化器、学习率、各种评估指标。
5. 最后讲测试时增强和输入分辨率（2.6 节），并用两张总表收尾（2.7 节）。

具体的图表逐个解读和公式的详细计算放在第 4 章；术语的简短定义放在第 3 章，可以随时查。

2.1　我们的模型：ViT-B/16

2.1.1　先看整体：输入是什么，输出是什么

我们的任务是**图像分类**（image classification）：给模型一张照片，让它说出照片属于 10,000 个类别中的哪一类。这些类别是很细的“细粒度”类别（fine-grained），比如不同种类的鸟、蛇、蛾子、植物；而且课程把类别名字藏起来了，我们只知道“第 0 类、第 1 类……第 9,999 类”。

模型的输入和输出

输入：一张彩色照片，先被缩放、裁剪成 224 × 224 像素。每个像素有红、绿、蓝 3 个数，所以输入是 224 × 224 × 3 = 150,528 个数。

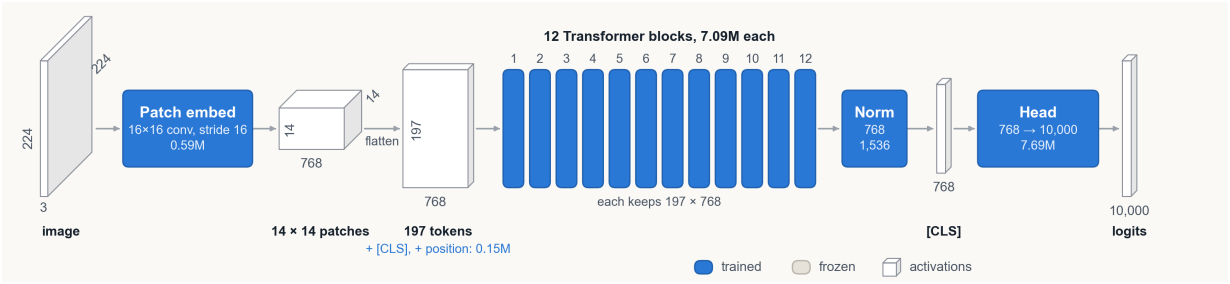
输出：10,000 个分数，叫 logits（未归一化的分数）。每个分数对应一个类别，分数越高，模型越觉得照片属于这一类。

预测：把 10,000 个分数用 softmax 变成概率（加起来等于 1），概率最大的那一类就是模型的答案。

一张照片走完模型之后

假设输入一张麻雀的照片，模型输出 10,000 个分数，经过 softmax 后最大的几个概率是：第 3,812 类 0.71，第 3,815 类 0.12，第 977 类 0.05……模型的 top-1 预测就是第 3,812 类。如果真实标签也是第 3,812 类，这张照片就算“答对”。

ViT-B/16 的名字可以拆开理解：ViT 是 Vision Transformer（视觉 Transformer）；B 是 Base，表示中等大小的版本；16 表示把图片切成 16 × 16 像素的小块。下面按数据流动的顺序，一步一步看它内部发生了什么。



ViT-B/16 的完整结构（这张图来自第 10 页，蓝色 = 全量微调时训练的部分）：图片 → 切块 → 197 个 token → 12 个 Transformer block → 归一化 → [CLS] 特征 → 分类头 → 10,000 个分数

2.1.2　第一步：把图片切成小块（patch），变成向量

Transformer 原本是为文字设计的，它处理的是一串“词”。为了让它能看图片，ViT 把图片切成一块一块的小方格，每个小方格就当作一个“词”。

- 224 像素宽，每块 16 像素，所以每行有 224 ÷ 16 = 14 块，一共 14 × 14 = 196 块。这些小块叫 patch（图块）。
- 每个 patch 有 16 × 16 × 3 = 768 个像素数值。模型用一个线性变换（实现上是一个 16 × 16、步长 16 的卷积）把这 768 个原始数值变成一个 768 维的向量。这一步叫 patch embedding（图块嵌入），它有 16 × 16 × 3 × 768 + 768 = 590,592 个参数（约 0.59M）。
- 变换后的向量叫 token（标记）。现在图片变成了 196 个 token，每个 token 是 768 个数。

生活类比

把一张照片剪成 196 块拼图。每块拼图写一张“描述卡片”，卡片上有 768 个数字，描述这一块里有什么颜色、纹理、边缘。后面的模型不再看原始像素，而是读这 196 张卡片。

2.1.3　第二步：加上 [CLS] 和位置编码

[CLS] token：负责“总结全图”的特殊 token

ViT 在 196 个图块 token 前面再加一个特殊的 token，叫 [CLS]（class，分类用）。它一开始不对应任何图块，只是一个可学习的 768 维向量。经过 12 层处理后，它会从所有图块那里收集信息，最后它的输出就被当作“整张图片的总结”，送去分类。所以 token 总数是 $196+1=197$ 个，每个 768 维，可以写成一个 197×768 的表格。

位置编码（position embedding）：告诉模型每块在哪里

Transformer 本身不知道 token 的顺序和位置。把拼图打乱，它也看不出来区别。所以 ViT 给 197 个位置各准备一个可学习的 768 维向量，加到对应的 token 上，这就是 position embedding（位置编码）。[CLS] 和位置编码一共有 $768+197\times768=152{,}064$ 个参数（约 0.15M）。

CLS

像班长：班上 196 个同学（图块）各自观察了照片的一小块，班长挨个听大家的意见，最后代表全班给出结论。位置编码像座位号：每张描述卡片上都写着“我来自第几行第几列”，这样模型知道哪块在上、哪块在下。

2.1.4　第三步：12 个 Transformer block

这是模型的主体。12 个 block（层）结构完全一样，一个接一个。每个 block 的输入和输出都是 197×768，形状不变，但内容越来越“懂”图片：前面的层更多看边缘、颜色、纹理这类简单的东西，后面的层更多看部件、整体形状这类抽象的东西。

每个 block 里依次是：

1. **LayerNorm（层归一化，LN）**：把每个 token 的 768 个数调整到平均值 0、方差 1，再乘一个可学习的缩放 γ、加一个可学习的偏移 β。作用是让数值保持在合适的范围，训练更稳定。每个 LN 有 $768+768=1{,}536$ 个参数。
2. **多头自注意力（multi-head self-attention）**：让每个 token 去“看”其他所有 token，把有用的信息收集过来。它由三部分组成：qkv 线性层（$768\to2304$，一次算出 Q、K、V 三组向量）、注意力计算本身（没有参数）、proj 线性层（$768\to768$，把 12 个头的结果合在一起）。下面单独解释。
3. **残差连接（residual connection）**：把注意力的输出加回到它的输入上：输出 = 输入 + 注意力(输入)。
4. **第二个 LayerNorm**。
5. **MLP（多层感知机）**：对每个 token 单独做两次线性变换：fc1 把 768 维扩展到 3072 维，经过 GELU 激活函数（一种平滑的“负数压小、正数保留”的函数），再由 fc2 压回 768 维。注意力负责“交流”，MLP 负责每个 token 自己“消化思考”。
6. **第二个残差连接**：输出 = 输入 + MLP(输入)。

自注意力公式

$$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

- Q（Query，查询）：每个 token 发出的“我想找什么信息”。
- K（Key，键）：每个 token 的“我这里有什么信息”的标签。
- V（Value，值）：每个 token 真正要传出去的内容。
- $QK^\top$：每个 Query 和每个 Key 做点积，点积越大说明越“对口”。197 个 token 两两配对，得到 197×197 个分数。
- $\sqrt{d}$：$d=64$ 是每个头的维度。除以 $\sqrt{64}=8$ 是为了让分数不要太大，否则 softmax 会变得过于极端。
- softmax：把每一行分数变成加起来等于 1 的权重。
- 乘以 V：按权重把各个 token 的内容加权平均，得到新的 token。

多头：ViT-B 有 12 个头，每个头 64 维，$12\times64=768$。不同的头可以关注不同的关系，比如一个头看颜色相近的区域，另一个头看形状上相连的区域。

一个两 token 的小计算

设维度 $d=2$。token 1 的 Query 是 $q=(1,0)$，两个 token 的 Key 分别是 $k_1=(1,0)$、$k_2=(0,1)$。

- 点积：$q\cdot k_1=1$，$q\cdot k_2=0$。
- 除以 $\sqrt{2}\approx1.414$：得到 0.707 和 0。
- softmax：$e^{0.707}\approx2.03$，$e^0=1$，所以权重是 $2.03/3.03\approx0.67$ 和 $1/3.03\approx0.33$。
- 新的 token 1 $=0.67\times v_1+0.33\times v_2$：它主要吸收了和自己“对口”的 token 1 的内容，也吸收了一点 token 2 的内容。

生活类比：自注意力像开会

197 个人开会。每个人手里有一个问题（Query），胸前挂着一个名牌写着自己擅长什么（Key），脑子里有具体的知识（Value）。每个人根据自己的问题，看谁的名牌最对口，就多听谁讲（权重高），少听不相关的人讲（权重低）。开完会，每个人都更新了自己的认识。12 个头相当于同时开 12 个不同主题的小组会。

生活类比：残差连接像“修改稿”

残差连接的意思是：每一层不是从零重写，而是在原稿上做修改，输出 = 原稿 + 修改。这样即使某一层学得不好，原来的信息也不会丢；而且训练时信号可以沿着“原稿”这条捷径一路传回前面的层，深层网络更容易训练。

每个 block 有多少参数?

每个线性层的参数 = 输入维度 × 输出维度 + 输出维度（偏置）。所以一个 block 的参数是：

部件	计算	参数个数
qkv（768 → 2304）	$768 \times 2304 + 2304$	1,771,776
proj（768 → 768）	$768 \times 768 + 768$	590,592
fc1（768 → 3072）	$768 \times 3072 + 3072$	2,362,368
fc2（3072 → 768）	$3072 \times 768 + 768$	2,360,064
两个 LayerNorm	$2 \times (768 + 768)$	3,072
一个 block 合计		**7,087,872**

重点

MLP（fc1 + fc2）一共 4,722,432 个参数，占一个 block 的 **三分之二**。这解释了为什么把 LoRA 从只放在 qkv 扩展到 MLP 后，准确率提高了 0.88 个百分点：以前大部分权重根本没有被调整到。

2.1.5　第四步：最后的归一化、[CLS] 特征和分类头

12 个 block 之后，还有一个最终的 LayerNorm（1,536 个参数）。然后只取出 [CLS] 这一个 token 的输出，它是一个 768 维向量，叫做这张图片的**特征**（feature），记作 z。

最后，**分类头**（head）把这个 768 维的特征变成 10,000 个分数：

$$\text{logits} = Wz + b, \qquad W \in \mathbb{R}^{10000\times 768},\ b \in \mathbb{R}^{10000}$$

W 的每一行对应一个类别，可以理解为这个类别的“模板”。特征 z 和某一行越像（点积越大），这一类的分数越高。分类头的参数是 $768 \times 10{,}000 + 10{,}000 = 7{,}690{,}000$ 个。

2.1.6　参数一共有多少

部分	参数个数	说明
patch embedding	590,592	把图块变成 768 维向量
[CLS] + 位置编码	152,064	$768 + 197 \times 768$
12 个 block	85,054,464	$12 \times 7{,}087{,}872$
最终 LayerNorm	1,536	
backbone 合计	**85,798,656**	约 85.8M
新的分类头	**7,690,000**	约 7.69M，是 backbone 的 9.0%

2.1.7　backbone 和 head 是什么关系

- **backbone（骨干网络）**：分类头之前的所有部分，也就是从图片到 768 维特征的整个“特征提取器”。它负责“看懂”图片。
- **head（分类头）**：最后那个 768 → 10,000 的线性层。它负责“下结论”：根据特征判断是哪一类。

我们下载的预训练模型原来有一个 1,000 类的分类头（对应 ImageNet-1k 的 1,000 类），对我们没用，所以换成一个新的 10,000 类分类头。backbone 是“有经验的老手”，分类头是“刚来的新人”。

“固定 backbone”不等于“冻结 backbone”

老师说“请使用推荐的 backbone，不要随便换”，意思是**模型的选择**要固定：所有实验都用同一个 ViT-B/16。这和“冻结”（freeze，训练时不更新它的参数）是两回事。全量微调（full fine-tuning）用的就是这个 backbone，只是把它的 85.8M 个参数全部都训练了；LoRA 等方法则把 backbone 冻结，只训练新加的小模块。两种做法都“没有换 backbone”。

2.1.8　预训练和迁移学习：为什么不从零训练

我们用的是 timm 库里的 vit_base_patch16_224.augreg2_in21k_ft_in1k。名字的意思是：

- in21k：先在 ImageNet-21k 上预训练。这是一个很大的公开数据集，有两万多个类别、一千多万张图片。
- augreg：预训练时用了一套数据增强（augmentation）和正则化（regularization）的方法，叫 AugReg。
- ft_in1k：再在 ImageNet-1k（1,000 类）上微调（fine-tune）过。
- patch16_224：图块 16 像素，输入 224 像素。

迁移学习（transfer learning）的意思是：把在大数据集上学到的“看图能力”迁移到新任务上。我们每类只有 45 张照片，如果从零训练一个 85.8M 参数的模型，它会很快把这 45 张背下来，但学不会真正的规律。预训练模型已经会识别边缘、纹理、形状、物体部件，我们只需要让它适应新的 10,000 个类别。

生活类比

一个学过全科医学的医生转去皮肤科，比一个完全没学过医的人学得快得多：基础知识（解剖、病理）都通用，只需要学皮肤科特有的东西。预训练模型就是“学过全科的医生”，我们的任务是“皮肤科专科培训”，而每种病只有 45 个病例可以学。

为什么我们没有换 backbone

1. 项目说明（brief）第一页推荐的就是这个模型，老师的邮件也说除非有很强的理由，不要换。
2. 我们用它复现了说明里的三个参考结果，差距都在 0.3 个百分点以内，说明我们的代码流程是对的。
3. 截止后我们还专门检查了两个只在 ImageNet-21k 上预训练的版本。比较的方法是：冻结 backbone，只训练分类头（线性探针），看谁的特征更好。结果是：augreg_in21k 61.05%，orig_in21k 51.07%，都低于我们用的 61.94%。所以换 backbone 不会带来好处。

2.2　分类头（head）和它的三种初始化

2.2.1　为什么分类头这么关键

分类头看起来只是“最后一层”，但在我们的任务里它非常特殊：

- **它很大**：7.69M 个参数，相当于 backbone 的 9.0%，是项目说明里参考 LoRA（只在 qkv 上，294,912 个参数）的 26 倍。
- **它是全新的**：backbone 是预训练过的，但分类头是我们新加的，一开始什么都不会。
- **它的数据很少**：10,000 行中的每一行（每个类别的模板），只能从这一类的 43–45 张照片里学。

所以，**分类头从什么样的初始值开始训练**，会影响整个训练过程。这就是“初始化”（initialization）问题。我们比较了三种初始化。

分类头怎样打分

$$\text{logit}_c = w_c \cdot z + b_c$$

- z：图片的 768 维特征。
- w_c：分类头的第 c 行，也是 768 维，可以理解为第 c 类的“模板”。
- $w_c \cdot z$：点积，衡量特征和模板有多像。
- b_c：第 c 类的偏置（一个常数）。

初始化，就是决定训练开始前 w_c 和 b_c 取什么值。

2.2.2　初始化一：随机初始化（random）

是什么（What）

用随机的小数字填满分类头（timm 库对新线性层的默认做法）。

核心原理和问题

随机的分类头对所有类别给出差不多的分数，所以 10,000 类的概率都接近 1/10,000。这时的交叉熵损失约等于 $\ln 10{,}000 \approx 9.21$（损失函数的解释见 2.4 节）。我们在 LoRA 训练的前 100 步测到的平均损失是 9.28，和这个理论值几乎一样。

损失很大意味着梯度很大。训练一开始，又大又乱的梯度会从分类头传回 backbone，可能“破坏”预训练学到的好特征。这正是 LP-FT 那篇论文担心的问题。

优缺点

优点：最简单，不需要额外计算。缺点：起点最差（准确率接近 0），训练前期浪费时间；理论上可能扭曲特征。

2.2.3　初始化二：线性探针头（linear probe，LP）和 LP-FT

是什么（What）

先冻结 backbone，把所有训练图片的特征算出来并存好；然后只训练分类头，让它在这些固定特征上学会分类；最后把这个训练好的分类头作为微调的起点。这叫 linear probe（线性探针）初始化。“先线性探针、再全部微调”这个两步做法叫 **LP-FT**（Linear Probing then Fine-Tuning），来自 Kumar

等人在 ICLR 2022 发表的论文，是一个常用、被推荐的做法。

为什么大家推荐它（Why）

LP-FT 的论文认为：随机分类头在微调初期会产生大而乱的梯度，扭曲（distort）预训练特征，尤其会损害模型在“新领域”数据上的鲁棒性。先把分类头训练好，微调时梯度就小，特征就被保护住了。

核心原理（How）

1. 用冻结的 backbone 把 450,000 张训练图片各算一遍特征（约 6 分钟）。
2. 以原型头（见下面初始化三）为起点，在这些缓存的特征上训练分类头 10 个 epoch（AdamW，学习率 10^{-3}，batch 4,096，余弦衰减，label smoothing 0.1）。只要几秒钟，holdout 准确率就到约 62%。
3. 用这个分类头开始微调（全量微调或 LoRA）。

在我们的任务里发生了什么

LP 头一开始很好（62.1%，比原型头的 44.0% 高 18 个百分点），但训练结束后反而更差：

- 全量微调 3 个 epoch（holdout）：LP 70.88%，原型 73.38%。
- LoRA 3 个 epoch（holdout）：LP 69.45%，原型 72.00%。
- AdaptFormer 3 个 epoch（holdout）：LP 69.05%，原型 71.17%。
- 最终配方（全量微调 8 个 epoch，CodaBench 验证集）：LP 73.10%，原型 76.22%，差 3.12 个百分点。

我们测试的三种方法都一样：LP 头让最终结果低了 2.1–3.1 个百分点。

为什么 LP 头在这里有害：三步解释

第一步：它是在同一批训练照片上拟合出来的。LP 头训练用的特征，来自后面微调要用的同一批照片。所以微调一开始，它已经把这些照片分得不错了，训练损失只有 2.29（原型头是 4.39，随机头是 9.28）。

第二步：信号先弱、后错。损失小意味着梯度小，backbone 得到的学习信号很弱，而且主要来自 LP 头恰好分错的少数照片。等 backbone 的特征开始变化，原来拟合好的分类头就不再匹配新的特征了：LoRA 加 LP 头的训练中，第一个 epoch 里训练损失反而从 2.22 **上升**到 2.69，这时还没有任何一张照片被看第二遍。

第三步：结果是“死记”。最终配方的两个模型只差在分类头的起点。LP 那个模型的训练损失一直更低（最后 1.77，原型是 1.85），验证集准确率却低得多（73.10% 对 76.22%）。训练损失更低、真实准确率更低，这就是**记住了训练照片，而不是学会了规律**（memorization，记忆/过拟合）。

生活类比

LP 头像一个提前背好了练习册答案的学生。第一次做同一本练习册，他分数很高（起点 62%），于是觉得没什么要学的（梯度小）。可是老师接着换了讲法（backbone 的特征在变），他背的答案对不上新讲法了（损失上升）。最后他把练习册背得更熟（训练损失更低），但遇到新题（验证集）反而做不好。

优缺点和适用场景

优点：起点准确率高；在“训练数据多、需要保护特征以应对新领域”的情况下，LP-FT 确实有用。

缺点：在我们这种“每类只有 45 张、类别非常多、分类头又是在同一批照片上拟合的”情况下，它会导致死记，最终更差。

2.2.4　初始化三：原型头（prototype head，weight imprinting）

是什么（What）

不训练分类头，而是直接“算”出来：每一类的模板 w_c，等于这一类所有训练照片特征的平均方向（叫这一类的**原型**，prototype），再乘一个温度 40。这个做法叫 weight imprinting（权重印记），来自 Qi 等人 CVPR 2018 的论文。**方法不是我们发明的**；我们的新发现是：在我们这种情况下，用它代替 LP-FT 的起点，三种方法都提高了 2.1–3.1 个百分点。

核心原理（How，按步骤）

1. 用冻结的 backbone 把 450,000 张训练照片各算一个 768 维特征（约 6 分钟，计入 GPU 时间）。
2. 把每个特征除以它自己的长度，变成长度为 1 的向量（L2 归一化），只保留“方向”。
3. 同一类的方向向量加起来，再归一化一次，得到这一类的原型 μ_c。
4. 分类头的第 c 行设为 $w_c = \mu_c \times \dfrac{40}{\text{特征的平均长度}}$，偏置 $b_c = 0$。
5. 这样每个分数约等于 $40 \times \cos(z, \mu_c)$，也就是“照片特征和这一类平均方向的余弦相似度”再乘 40。

整个过程**没有一步梯度下降**，分类头从来没有“拟合”过任何一张训练照片，所以没有东西可以死记。

原型和余弦相似度

$$\mu_c = \frac{\sum_{i:\, y_i = c} \hat{z}_i}{\left\| \sum_{i:\, y_i = c} \hat{z}_i \right\|}, \qquad \hat{z}_i = \frac{z_i}{\|z_i\|}, \qquad \text{logit}_c \approx \tau \cdot \cos(z, \mu_c)$$

- z_i：第 i 张训练照片的特征；y_i 是它的类别。
- $\hat{z}_i$：归一化后的特征，长度为 1。$\|\cdot\|$ 表示向量的长度。
- $\sum_{i:\, y_i = c}$：把所有属于第 c 类的照片加起来。
- $\cos(z, \mu_c)$：两个向量夹角的余弦，范围 -1 到 1，越接近 1 越相似。
- $\tau = 40$：温度（temperature），也叫 logit scale。我们从 5、10、15、20、30、40、60、80 中挑选，用 10,000 个训练特征算交叉熵，40 最好。

一个二维的小例子

假设特征只有 2 维。“麻雀”类有两张照片，特征是 $(2, 0)$ 和 $(1, 1)$。

1. 归一化：$(2, 0) \to (1, 0)$；$(1, 1) \to (0.707, 0.707)$。
2. 相加：$(1.707, 0.707)$，长度 ≈ 1.848；再归一化得到原型 $\mu_{\text{麻雀}} \approx (0.924, 0.383)$。
3. 来了一张新照片，特征 $(3, 1)$，归一化为 $(0.949, 0.316)$。它和麻雀原型的余弦是 $0.924 \times 0.949 + 0.383 \times 0.316 \approx 0.998$。
4. 假设另一类“燕子”的原型是 $(0, 1)$，余弦是 0.316。

温度的作用：如果不乘温度，两个分数是 0.998 和 0.316，softmax 后概率只有 0.66 和 0.34，太“犹豫”。乘 5 以后分数是 4.99 和 1.58，麻雀的概率变成 0.97。真实任务有 10,000 类，很多类的余弦都差不多，所以需要一个较大的温度（40）让正确的类突出出来。

生活类比

给每个人做一张“平均脸”：把他的 45 张照片叠在一起取平均。认人时，把新照片和每个人的平均脸比，最像谁就认为是谁。这不需要背任何一张具体的照片，所以不会“死记”；但平均脸已经包含了这个人的主要特征，所以起点不错（44%）。

为什么它有效

- **它编码了类别的结构**，所以学得快：LoRA 训练一个 epoch 后，原型头 63.3%，随机头只有 58.4%。
- **它没有拟合任何训练照片**，所以不会让模型一开始就“觉得自己都会了”。起始损失 4.39，比 LP 头的 2.29 高，backbone 能得到足够强、而且有意义的学习信号。

2.2.5　关键对照实验：随机头和原型头打平

如果原型头只是“比随机好”，那 LP 头的问题也可能只是“不够好”。为了分清楚，我们做了对照（control）实验：LoRA 训练 3 个 epoch，随机头 71.95%，原型头 72.00%，几乎一样（这两次都在 RTX PRO 6000 上跑；原型头在 A100 上重跑一次是 71.98%）。

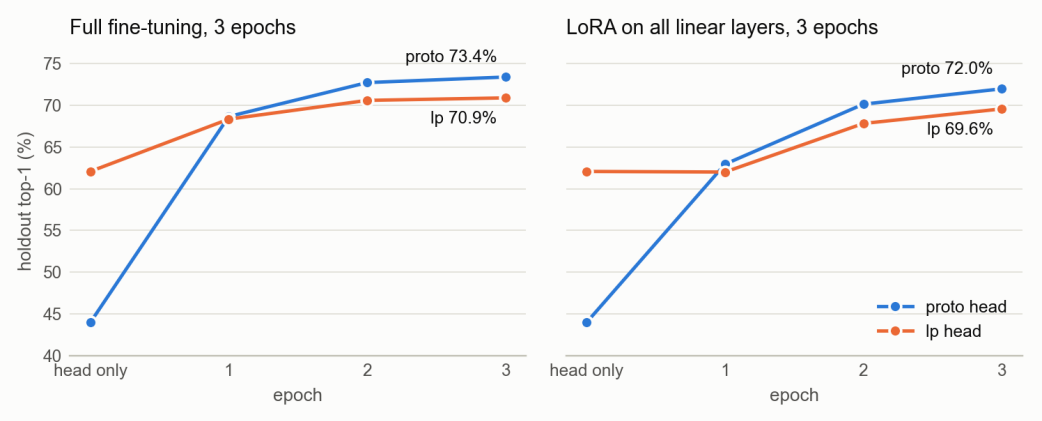
对照实验说明了什么

随机头这个最差的起点，最后和原型头一样好；只有 LP 头明显更差。所以问题不在于“起点不够好”，而在于**分类头事先在同一批训练照片上被拟合过**。原型头并不神奇，它的价值是：起点合理、学得快、而且没有被拟合过。这也部分回应了 LP-FT 论文的担心：在我们的任务里，随机头造成的“特征扭曲”并没有带来损害，因为我们的 10,000 个细粒度类别本来就需要特征发生变化。

2.2.6　三种初始化对比

	随机头	LP 头（LP-FT）	原型头（imprinting）
怎么得到	随机数	在缓存特征上训练 10 个 epoch	每类特征的平均方向 × 40
训练前准确率	接近 0（随机猜）	62.1%	44.0%
起始训练损失 *	9.28	2.29	4.39
LoRA 3 epoch（holdout）	71.95%	69.45%	72.00%
全量微调 3 epoch（holdout）	–	70.88%	73.38%
最终配方（验证集）	–	73.10%	76.22%
主要优点	简单	起点高	学得快，不死记
主要缺点	起点差、前期梯度乱	在同一批照片上拟合，导致死记	需要先算一遍特征（约 6 分钟）

* 起始训练损失是 LoRA 训练前 100 步的平均训练损失，包含 label smoothing 和数据增强的影响，只适合三者互相比较，不要换算成“正确类的概率”。9.28 约等于瞎猜（$\ln 10{,}000 = 9.21$）；4.39 表示大致知道每类长什么样，但还很不确定；2.29 表示已经在这批训练照片上拟合过。



第 6 页的图：holdout 准确率随 epoch 的变化。LP 头（橙色）起点高、终点低；原型头（蓝色）起点低、终点高。这条 LoRA 曲线的最后一点是 69.6% 对 72.0%（逐 epoch 记录的值）。详细解读见第 4 章。

2.3　七种 adaptation（适配）方法

Adaptation（适配）指的是：怎样让预训练模型适应我们的新任务。区别在于**训练哪些参数**。我们比较了七种方法，可以分成两大类：

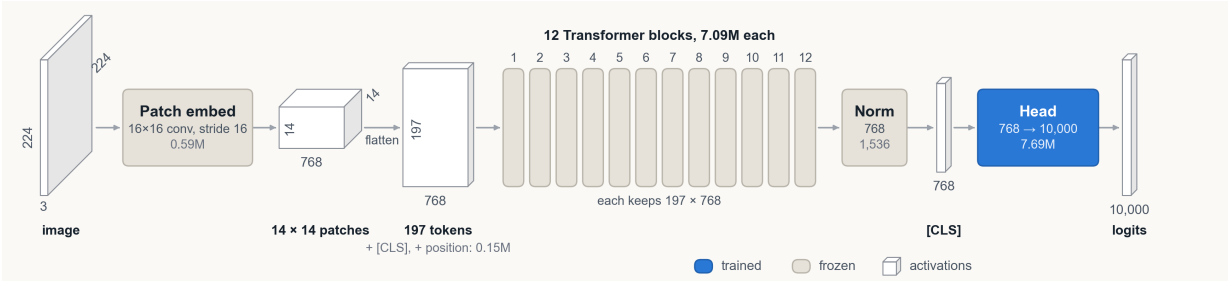
类别	方法
第一类：从顶部开始训练整块	线性探针（只训练分类头）、部分微调（训练最后 K 个 block）、全量微调（训练所有参数）
第二类：在每个 block 里做小改动	BitFit（只训练偏置）、Adapter（串联小模块）、AdaptFormer（并联小模块）、LoRA（低秩更新）

第二类方法常被统称为 **PEFT**（Parameter-Efficient Fine-Tuning，参数高效微调）：只训练很少的参数，就能达到接近全量微调的效果。

公平比较的规则：同一个 protocol

为了让七种方法可以直接比较，它们使用完全相同的设置（protocol，实验规程）：原型头、训练 3 个 epoch、同一个 20,000 张的 holdout、同一种 GPU（A100）。所有方法都会训练分类头（7.69M 个参数），所以下面说的“可训练参数”按项目说明的习惯**不算分类头**，只算 backbone 里被训练的部分。

2.3.1　方法一：线性探针（Linear probe）



线性探针：整个 backbone 冻结（灰色），只训练最后的分类头（蓝色）。

是什么（What）

冻结整个 backbone，一个参数都不改，只训练分类头。backbone 只作为一个固定的“特征提取器”。

解决什么问题（Why）

它是最便宜的基线：花很少的计算，就能知道“预训练特征本身有多好”。我们也用它来比较不同 backbone 的特征质量（见 2.1 节）。

核心原理（How，按步骤）

1. 用冻结的 backbone 把所有训练照片的特征算一遍，存起来。
2. 分类头从原型头开始，在缓存的特征上训练 10 个 epoch（学习率 10^{-3}）。
3. 因为 backbone 不变，不需要往 backbone 里传梯度，所以非常快、非常省显存。

实际例子（Example：生活例子 + 简单数据）

生活类比

请了一位经验丰富的摄影师（backbone）帮你看照片，他会告诉你照片里有什么。你只培训一个新助理（分类头），让他根据摄影师的描述去贴标签。摄影师的眼光一点都不变，即使他从来没认真区分过这 10,000 种生物。

数据：holdout 61.94%，只用 0.10 GPU 小时，训练显存 3.2 GB，单张推理延迟 6.96 ms。作为对比，完全不训练的原型头（zero-shot）是 43.95%。

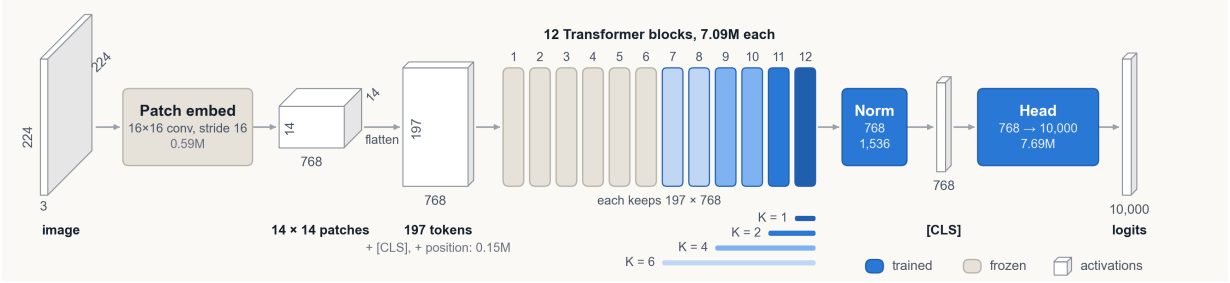
优缺点（Pros & Cons）与适用场景

优点：最快、最省；backbone 不变，所以一个 backbone 可以配很多个小分类头。缺点：特征永远不适应新类别，准确率有上限（约 62%，比全量微调低 11 个百分点以上）。适用场景：快速基线、检查特征好坏、计算资源极少的时候。

与其他方法的区别

和零样本原型头比：线性探针训练了分类头，多了约 18 个百分点。和其他六种方法比：它是唯一不改变 backbone 特征的方法，所以它的结果可以看成“冻结特征的天花板”。

2.3.2　方法二：部分微调（Partial fine-tuning，最后 K 个 block）



部分微调：只训练最后 K 个 block、最终的 LayerNorm 和分类头；下方的横条表示 K = 1、2、4、6 分别训练哪些 block。

是什么（What）

解冻最后 K 个 Transformer block（以及最终的 LayerNorm），训练它们和分类头；下面的 $12-K$ 个 block 保持冻结。我们试了 K = 1、2、4、6。

解决什么问题（Why）

线性探针完全不改特征，全量微调全部都改，部分微调是两者之间的折中。直觉是：底层学的是边缘、纹理这些通用的东西，不太需要改；顶层更接近具体任务，最需要适应新类别。

核心原理（How，按步骤）

1. 冻结前 $12-K$ 个 block 和图块嵌入。
2. 训练最后 K 个 block、最终 LayerNorm 和分类头。
3. 梯度只需要从分类头传到第 $12-K+1$ 个 block，不用再往下传，所以显存和计算都省。

可训练参数 $= K \times 7{,}087{,}872 + 1{,}536$（1,536 是最终 LayerNorm）。

参数和结果

K	可训练参数	占 backbone	holdout
1	7,089,408	8.26%	67.97%
2	14,177,280	16.5%	70.99%
4	28,353,024	33.0%	71.88%
6	42,528,768	49.6%	72.89%

再加上线性探针（K = 0，61.94%）和全量微调（K = 12，73.38%），可以看到：第一个 block 带来约 6 个百分点，第二个约 3 个，之后每多两个 block 只多约 1 个。这叫**边际收益递减**。训练显存从 3.8 GB（K = 1）到 10.0 GB（K = 6），每多训练一个 block 约多 1.5 GB。注意：K = 1、2 用学习率 3×10^{-4}，K = 4、6 用全量微调的配方，所以 K = 2 到 K = 4 的变化混合了两个因素。

生活类比

一栋 12 层的办公楼要改造成新用途。部分微调是只装修最上面几层，下面的楼层保持原样。装修的楼层越多效果越好，但越往下装修，带来的改善越少。

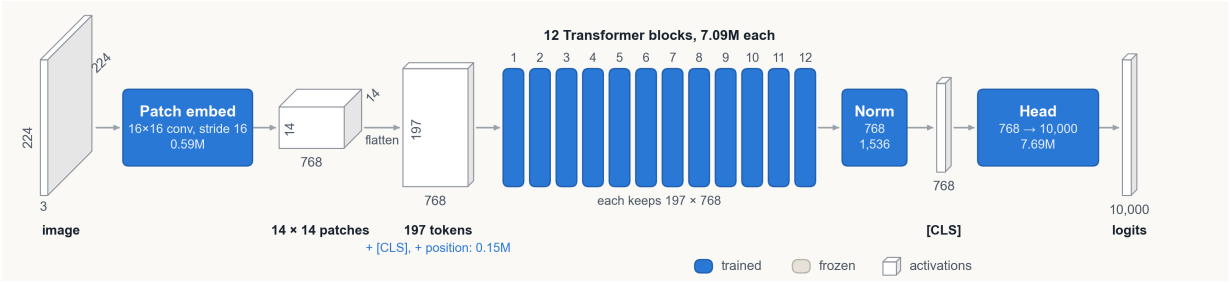
优缺点（Pros & Cons）与适用场景

优点：训练最便宜（显存最多 10 GB、0.35 GPU 小时），推理没有额外开销；训练一半的 block 只比全量微调低 0.5 个百分点。缺点：底层永远不变；可训练参数很多（8%–50%），每个任务要存一大份权重；参数效率低。适用场景：显存紧张、但存储不是问题的时候。

与其他方法的区别

它是“在少数几层做大改动”，而 BitFit、Adapter、AdaptFormer、LoRA 是“在每一层做小改动”。对比非常明显：BitFit 只训练 0.12% 就超过了 K = 1（8.26%）；LoRA 训练 1.38% 就追平了 K = 4（33%）。这就是我们的结论“**训练的位置比训练的数量更重要**”。

2.3.3　方法三：全量微调（Full fine-tuning）



全量微调：所有部分都是蓝色，全部训练。

是什么（What）

训练模型的每一个参数：backbone 的 85.8M 个加上分类头的 7.69M 个。这是我们最终排名第一的模型所用的方法。

解决什么问题（Why）

自由度最大，模型可以在每一层按需要调整，所以通常最准确，常被当作其他方法的“上限”参考。我们的 10,000 个细粒度类别需要特征在每一层都发生变化，所以它最适合。

核心原理（How，按步骤）

1. 所有参数都可训练。
2. 用**小学习率** 10^{-4}，避免一下子把预训练学到的东西冲掉。
3. 用**分层学习率衰减**（layer-wise LR decay）0.75：越靠近输入的层，学习率越小（第一个 block 只有约 3%），底层的通用特征改得少，顶层改得多。
4. 用 **drop path** 0.1、**weight decay** 0.05 等正则化，防止死记。
5. 这些细节在 2.4 节详细解释。

实际例子（Example：生活例子 + 简单数据）

生活类比

整个团队一起接受新业务培训：每个人都可以调整自己的工作方式。资深员工（底层）只微调一点点，靠近客户的员工（顶层）调整多一些。

数据：同一 protocol 下 holdout 73.38%，是七种方法里最高的；训练显存 18.9 GB；训练速度 1,139 张/秒；0.48 GPU 小时；每个任务要存一份完整模型，约 374 MB。最终模型（训练 8 个 epoch、用全部 45 张/类、翻转 TTA）：CodaBench 验证集 76.22%，测试集 76.09%，比项目说明的参考（70.8%）高 5.4 个百分点。

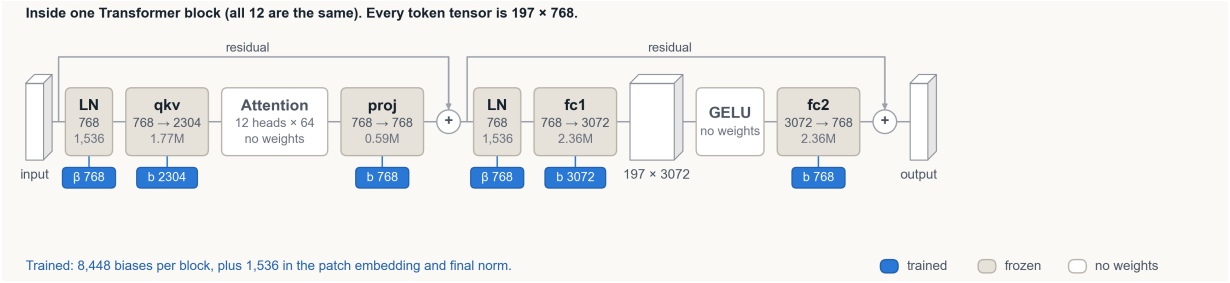
优缺点（Pros & Cons）与适用场景

优点：最准确；训练更久还会继续变好（从 3 个 epoch 到 8 个 epoch 提高 2.21 个百分点）。缺点：训练显存大；每个任务都要存一整份 85.8M 参数的模型；如果学习率太大，可能破坏预训练特征。适用场景：数据总量够多（我们有 45 万张）、只关心一个任务的最高准确率。

与其他方法的区别

它是所有方法的上限：在同一 protocol 下，没有任何 PEFT 方法超过它。最好的 PEFT 方法 LoRA 达到它的 98%（71.98% 对 73.38%），但只训练 1.38% 的参数。

2.3.4　方法四：BitFit（只训练偏置）



BitFit：放大看一个 Transformer block。所有权重矩阵冻结，只训练蓝色小标签表示的偏置向量。

是什么（What）

冻结所有权重矩阵，只训练**偏置**（bias）向量，再加上分类头。来自 Ben Zaken 等人 2022 年的论文。

解决什么问题（Why）

想知道“最少要改多少参数”。偏置只占极小一部分参数，但它出现在每一层，改它相当于在每一层都做一个小小的平移。

核心原理（How，按步骤）

一个线性层是 $y = Wx + b$。BitFit 只改 b，W 不动。b 的作用是把输出整体平移，相当于调整每个神经元的“阈值”。

1. 每个 block 里有偏置的地方：两个 LayerNorm 的 β（各 768）、qkv（2304）、proj（768）、fc1（3072）、fc2（768），合计 $768 + 768 + 2304 + 768 + 3072 + 768 = 8{,}448$ 个。
2. 12 个 block 共 $12 \times 8{,}448 = 101{,}376$，再加上图块嵌入的偏置（768）和最终 LayerNorm 的 β（768），一共 **102,912 个**，只占 backbone 的 0.12%。
3. 训练时学习率 10^{-3}，不用 weight decay。

实际例子（Example：生活例子 + 简单数据）

生活类比

一家连锁餐厅换了新口味市场。BitFit 不改任何菜谱（权重矩阵），只允许每道菜调整一下盐的多少（偏置）。改动极小，但每道菜都改了，整体口味就能明显变化。

数据：holdout 69.81%；训练显存 12.7 GB；单张延迟 7.33 ms，没有任何推理开销。LoRA 比线性探针高了约 10 个百分点，BitFit 只用 LoRA 约 1/11 的参数，就拿到了其中的 7.9 个百分点。

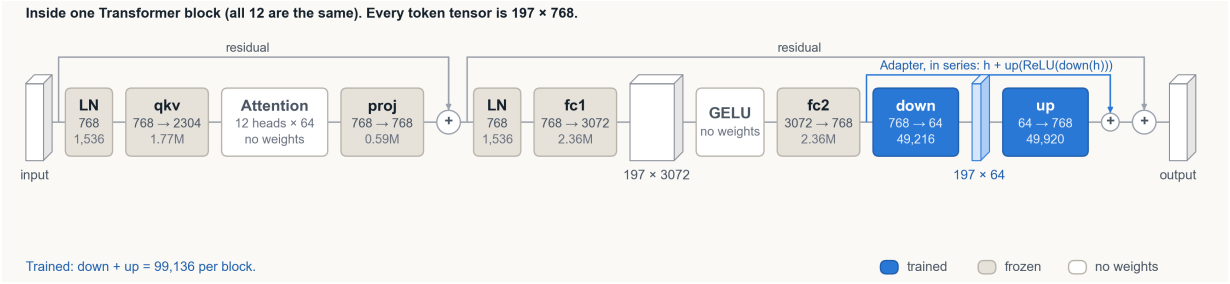
优缺点（Pros & Cons）与适用场景

优点：参数极少，每个任务只需要存 10 万个数；推理完全没有额外开销。缺点：能力有限，比 LoRA 低 2.2 个百分点；训练显存并不小（12.7 GB），因为梯度仍要传遍所有 12 个 block。适用场景：需要为大量任务分别保存模型、存储极其紧张的时候。

与其他方法的区别

和部分微调 K = 1 比：BitFit 只训练 0.12% 的参数，却比训练整整一个 block（8.26%）的 K = 1 更准（69.81% 对 67.97%）。这是“在每一层做小改动，胜过在一层做大改动”最直接的证据。

2.3.5　方法五：Adapter（串联的瓶颈模块）



Adapter：在每个 block 的 MLP 后面串联一个小的瓶颈模块（蓝色）：768 → 64 → 768，并有自己的跳跃连接。

是什么（What）

在每个 block 的 MLP 后面，**串联**插入一个很小的模块，叫 adapter（适配器）。它先把 768 维压缩到 64 维，经过 ReLU 激活，再放大回 768 维，然后加回原来的输出。这个结构像一个沙漏，所以叫**瓶颈**（bottleneck）。来自 Houlsby 等人 2019 年的论文。backbone 原有的参数全部冻结，只训练 adapter 和分类头。

解决什么问题（Why）

不动预训练权重，又给每一层增加一点“新能力”，让模型能适应新任务。因为每个任务只需要保存小小的 adapter，一个 backbone 可以服务很多任务。

核心原理（How，按步骤）

设 h 是 MLP 的输出（768 维），adapter 计算：

$$h' = h + W_{\text{up}}\,\text{ReLU}(W_{\text{down}}\,h)$$

1. W_{down}：$768 \to 64$ 的线性层（压缩），参数 $768 \times 64 + 64$。
2. ReLU：负数变 0，正数不变，带来非线性。
3. W_{up}：$64 \to 768$ 的线性层（放大），参数 $64 \times 768 + 768$。它一开始全是 0，所以训练开始时 adapter 的输出是 0，模型和预训练模型完全一样。
4. 加回 h（adapter 自己的跳跃连接），再进入 block 的残差相加。

每个 block：$2 \times 768 \times 64 + 64 + 768 = 99{,}136$ 个参数；12 个 block 共 **1,189,632 个**，占 1.387%。

实际例子（Example：生活例子 + 简单数据）

生活类比

公司每个部门（block）原来的工作流程不变，但在每个部门的出口加了一个“小翻译”：他先把部门的报告浓缩成要点（$768 \to 64$），再按新业务的要求改写回完整报告（$64 \to 768$）。刚上岗时他什么都不改（$W_{\text{up}} = 0$），慢慢学会该改哪里。

数据：holdout 71.54%；训练显存 12.8 GB；训练速度 1,509 张/秒，比 LoRA 快约 1.7 倍；但单张推理延迟 9.74 ms，比原始模型的 7.33 ms 慢约 2.4 ms。

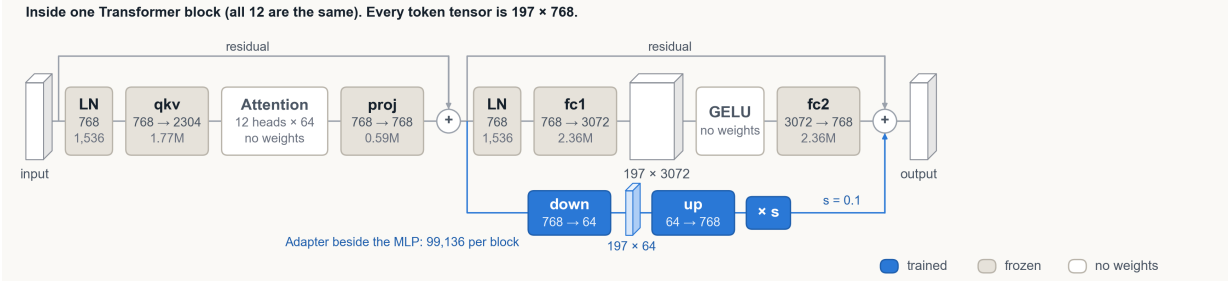
优缺点（Pros & Cons）与适用场景

优点：准确率接近 LoRA（只低 0.44 个百分点，在噪声范围内）；训练快、显存比 LoRA 少 36%。缺点：推理变慢。adapter 是主干路径上多出来的一层，而且中间有 ReLU，不能合并进原来的权重，所以每次推理都要多算它。适用场景：训练资源紧、对推理速度要求不高。

与其他方法的区别

和 AdaptFormer：结构几乎一样，区别是“串联在 MLP 后面”还是“并联在 MLP 旁边”。和 LoRA：参数量差不多（1.19M 对 1.18M），但 LoRA 训练完可以合并，推理不变慢。

2.3.6　方法六：AdaptFormer（并联的瓶颈分支）



AdaptFormer：同样的瓶颈模块，但放在 MLP 旁边作为并联分支，输出乘以 $s = 0.1$ 后加入。

是什么（What）

和 Adapter 用同样的瓶颈模块（768 → 64 → 768），但不是串在 MLP 后面，而是**并联**在 MLP 旁边：它和 MLP 拿同样的输入，各算各的，最后把它的输出乘一个小系数 $s = 0.1$ 加进来。来自 Chen 等人 2022 年的论文。

解决什么问题（Why）

串联的 adapter 会直接改写 MLP 的输出。并联分支让原来的 MLP 输出保持不变，只在旁边加一个小小的“修正量”，更温和。

核心原理（How，按步骤）

设 x' 是注意力部分之后的结果（也就是 MLP 部分的输入）：

$$x_{\text{下一层}} = x' + \text{MLP}(\text{LN}(x')) + s \cdot W_{\text{up}}\,\text{ReLU}(W_{\text{down}}\,x')$$

1. 主路：x' 经过 LayerNorm 和 MLP，这部分冻结不变。
2. 旁路：x' 经过 W_{down}（768 → 64）、ReLU、W_{up}（64 → 768），再乘 $s = 0.1$。旁路还用了 dropout 0.1，W_{up} 初始为 0。
3. 三部分相加，得到下一层的输入。

参数和 Adapter 完全一样：1,189,632 个（1.387%）。

实际例子（Example：生活例子 + 简单数据）

生活类比

部门的报告照常发出（主路不变），旁边坐着一位顾问，他也看了同样的材料，写一份简短的补充意见；补充意见只按 10% 的分量加进最终报告（$s = 0.1$）。

数据：holdout 71.17%（用 LP 头时是 69.05%，换成原型头提高 2.1 个百分点）；训练显存 12.8 GB；单张推理延迟 10.23 ms，同样比原始模型慢。

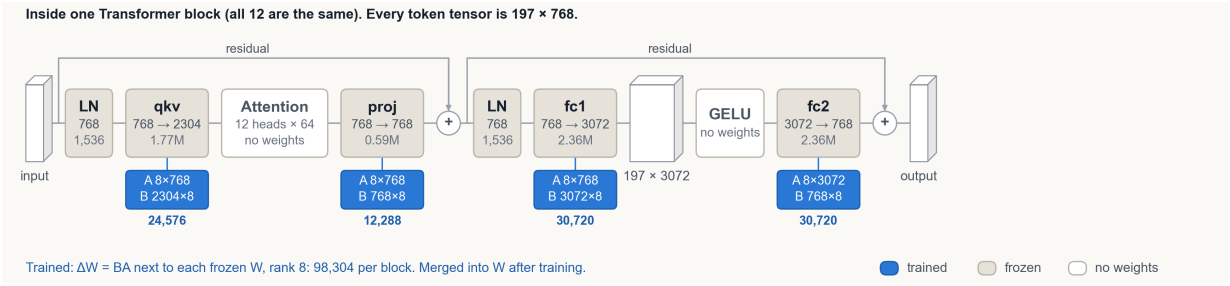
优缺点（Pros & Cons）与适用场景

优点：不改动原来 MLP 的输出，更稳；训练快、显存少。缺点：和 Adapter 一样不能合并，推理变慢约 30%–40%。

与其他方法的区别

和 Adapter 比：71.17% 对 71.54%，差 0.37 个百分点，和 ±0.3 左右的噪声差不多大；而且两者的缩放也不同（Adapter 是 1.0，AdaptFormer 是 0.1），所以这不是只改了一个因素的比较。我们的结论是：在我们的任务里，串联还是并联差别不大。

2.3.7　方法七：LoRA（低秩更新，可以合并）



LoRA：放大看一个 block。每个冻结的线性层（qkv、proj、fc1、fc2）旁边都有一对小矩阵 A 和 B（蓝色），学习更新量 $\Delta W = BA$；训练后合并进 W。

LoRA 是我们最终的参数高效模型，也是最常被问到的方法，所以这里讲得最详细。它来自 Hu 等人 ICLR 2022 的论文，全名是 Low-Rank Adaptation（低秩适配）。

是什么（What）

冻结原来的权重矩阵 W_0，在它旁边加两个很"瘦"的矩阵 A 和 B，用它们的乘积 BA 表示对 W_0 的修改量。训练时只学 A 和 B（以及分类头）；训练结束后把 BA 加进 W_0，变回原来的结构。

解决什么问题（Why）

- 全量微调要为每个任务改动并保存 85.8M 个参数，太重。
- LoRA 的假设是：适应新任务所需要的"修改量"其实很简单，可以用很少的参数表示（数学上叫**低秩**）。
- 和 adapter 不同，LoRA 的修改是线性的，训练后可以合并进原权重，**推理时完全不变慢**。

先理解"低秩"是什么意思

一个 $d_{\text{out}} \times d_{\text{in}}$ 的矩阵有 $d_{\text{out}} \times d_{\text{in}}$ 个数。如果它可以写成两个瘦矩阵的乘积 B（$d_{\text{out}} \times r$）和 A（$r \times d_{\text{in}}$），就只需要 $r \times (d_{\text{in}} + d_{\text{out}})$ 个数。r 叫**秩**（rank），r 越小越省。

秩为 1 的小例子

$$B = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 0 & -1 \end{pmatrix}, \quad BA = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 3 & 0 & -3 \end{pmatrix}$$

这个 3×3 矩阵有 9 个数，但它其实只由 $3 + 3 = 6$ 个数决定：每一行都是同一个模式 $(1, 0, -1)$ 的不同倍数。这就是"低秩"：变化虽然铺满了整个矩阵，但只有很少几个"方向"。

放到真实的 qkv 层：原矩阵 $768 \times 2304 = 1{,}769{,}472$ 个数；用 $r = 8$ 的 LoRA 只要 $8 \times (768 + 2304) = 24{,}576$ 个数，约为原来的 1.4%。

核心原理（How，按步骤）

1. **冻结**：backbone 所有原有参数不再更新。
2. **插入**：在 12 个 block 的四个线性层（qkv、proj、fc1、fc2）旁边各加一对矩阵：A 是 $r \times d_{\text{in}}$，随机初始化；B 是 $d_{\text{out}} \times r$，**初始化为 0**。我们用 $r = 8$。
3. **前向计算**：$y = W_0 x + \frac{\alpha}{r} BAx$。因为一开始 $B = 0$，额外项为 0，所以训练开始时模型和预训练模型**完全一样**，不会一上来就被破坏。
4. **缩放**：$\alpha = 16$，$r = 8$，所以额外项乘以 $\alpha / r = 2$。这个缩放让我们改变 r 时，不用重新调学习率。
5. **训练**：只更新 A、B 和分类头。我们用学习率 10^{-3}、weight decay 10^{-4}，训练 3 个 epoch。
6. **合并**：训练完后计算 $W = W_0 + \frac{\alpha}{r} BA$，再把 A、B 删掉。模型变回原来的结构。

LoRA 公式和合并

$$y = W_0 x + \frac{\alpha}{r} BAx = \left(W_0 + \frac{\alpha}{r} BA \right) x = Wx$$

- x：这一层的输入（维度 d_{in}）；y：输出（维度 d_{out}）。
- W_0：冻结的预训练权重；$\Delta W = BA$：学到的修改量。
- Ax：先把输入压到只有 $r = 8$ 维；$B(Ax)$：再放回 d_{out} 维。
- 等号中间那一步说明：两次矩阵乘法可以先合并成一个矩阵 W。合并后只需要一次矩阵乘法，和原模型一样快。

参数怎么算出 1,179,648 个

每个线性层增加 $r \times (d_{\text{in}} + d_{\text{out}})$ 个参数：

层	计算（$r = 8$）	参数
qkv（768 → 2304）	$8 \times (768 + 2304) = 8 \times 3{,}072$	24,576
proj（768 → 768）	$8 \times (768 + 768) = 8 \times 1{,}536$	12,288
fc1（768 → 3072）	$8 \times (768 + 3072) = 8 \times 3{,}840$	30,720
fc2（3072 → 768）	$8 \times (3072 + 768) = 8 \times 3{,}840$	30,720
一个 block		98,304
12 个 block	$12 \times 98{,}304$	**1,179,648（1.375%）**

项目说明里的参考 LoRA 只放在 qkv 上：$12 \times 24{,}576 = 294{,}912$ 个参数，占 0.344%。

实际例子（Example：生活例子 + 简单数据）

生活类比

你要把一本很厚的教科书（W_0）改编成新版本。全量微调是逐页重写整本书。LoRA 是不改原书，只在每章旁边贴几张便利贴（A、B），写下“这一章要这样改”。便利贴很少，但因为每一章都贴了，整本书的意思就变了。最后出版时，把便利贴的内容直接印进正文（合并），读者读的还是一本正常的书，不需要一边看正文一边翻便利贴，所以阅读速度不变。

我们在 LoRA 上做了三次改进尝试：

- 学习率减半（$10^{-3} \to 5 \times 10^{-4}$）：68.57% → 68.45%，变化 −0.12，在噪声内，说明学习率不是问题。
- 从只放 qkv 扩展到四个线性层：68.57% → 69.45%，+0.88。因为 MLP 占每个 block 三分之二的权重，以前它们完全没被调整。（这两次都用 LP 头。）
- 训练 6 个 epoch 而不是 3 个（原型头）：71.98% → 71.52%，−0.46。训练损失从 1.95 降到 1.70，holdout 却下降，说明过拟合了，所以最终用 3 个 epoch。

最终 LoRA 模型：holdout 71.98%，CodaBench 验证集 71.90%，测试集 71.67%，比参考 LoRA（68.3%）高 3.6 个百分点；0.54 GPU 小时；合并后单张延迟 7.61 ms。

为什么 LoRA 训练时反而比全量微调更占显存

直觉上，训练的参数少，显存应该少。但实际 LoRA 用 19.9 GB，比全量微调的 18.9 GB 还多。原因是：

- 训练时最占显存的不是参数本身，而是为了反向传播而保存的**中间结果**（activations，激活值）。
- 要算 A 的梯度，必须保存每个线性层的输入 x，这和全量微调一样多；LoRA 还额外保存 96 个小矩阵乘法（12 个 block × 4 层 × 2 个矩阵）的中间结果。

- 全量微调多出来的部分主要是优化器状态：85.8M 个参数的梯度和 Adam 的两个统计量，每个参数约 12 字节，一共只有约 1 GB。

所以：**训练显存取决于梯度要往回传多远、要保存多少中间结果，而不是训练了多少参数**。部分微调 K = 4 只需要把梯度传回 4 个 block，所以只用 7.0 GB。LoRA 训练也最慢（888 张/秒）。

优缺点（Pros & Cons）与适用场景

优点：参数少（1.38%），每个任务的检查点只有 35.5 MB（其中 30.8 MB 是分类头）；可以合并，推理零额外开销；是我们最准的 PEFT 方法。缺点：在我们的设置里，训练显存和速度都不比全量微调好；比全量微调低约 1.4 个百分点；训练更久会过拟合。适用场景：一个 backbone 要服务很多任务、需要每个任务存储小、推理速度不能变慢。

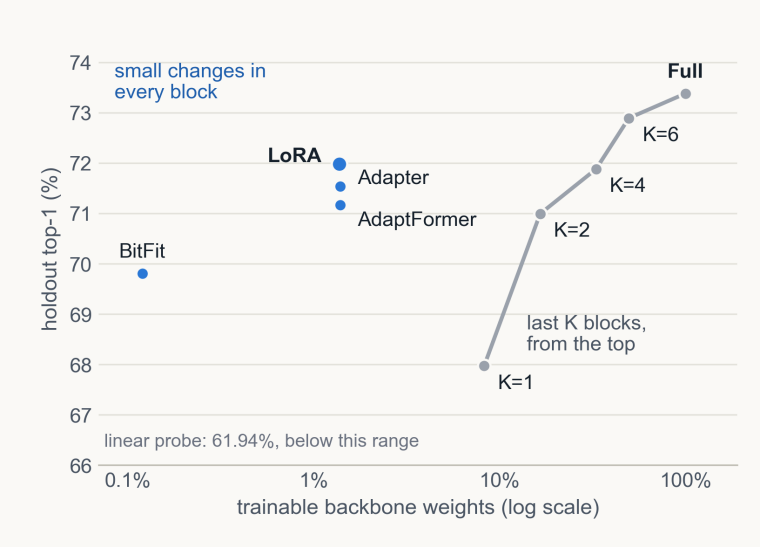
LoRA 训练久了为什么会变差：一个还没验证的猜想

LoRA 训练时可训练的参数一共 8.87M，其中 7.69M（87%）是分类头。在我们的 LoRA 配方里，分类头的学习率是 10^{-3}（全量微调里只有 10^{-4}），weight decay 只有 10^{-4}（全量微调是 0.05），也没有 drop path（全量微调是 0.1）。所以我们猜：训练久了，主要是分类头在死记那 45 张照片，和 LP 头的问题是同一类。验证方法：LoRA 训练 6 个 epoch，只把分类头的学习率降到 10^{-4}。我们已经准备好了 notebook（methods/10_lora_head_lr.ipynb），但**还没有运行**，所以这只是猜想。

与其他方法的区别

- 和 Adapter、AdaptFormer：参数差不多，准确率高 0.44 和 0.81 个百分点（和 Adapter 的差距在噪声内）；关键区别是 LoRA 可以合并，推理不变慢。
- 和 BitFit：BitFit 只平移输出，LoRA 能改变权重矩阵本身的“方向”，能力更强，高 2.2 个百分点。
- 和全量微调：达到它 98% 的准确率，只训练 1.38% 的参数；但在我们的任务里，训练成本并没有更低。

2.3.8　七种方法放在一起看：位置比数量更重要



第 5 页的图：横轴是训练的 backbone 参数比例（对数刻度），纵轴是 holdout 准确率。灰线是“从顶部训练整块”，蓝点是“每个 block 做小改动”。

这一节最重要的三个结论

1. 训练的位置比训练的数量更重要。灰线（部分微调）往右爬得很慢：训练 8% 才 68%，训练 50% 才 72.9%。蓝点（每个 block 做小改动）都在灰线的左上方：LoRA 用 1.38% 追平了 K = 4 的 33%，BitFit 用 0.12% 超过了 K = 1 的 8%。在每一层都改一点点，比在少数几层改很多更有效。

2. 训练显存跟着“梯度要传多远”走。只训练顶部几层的方法最省显存（K = 4 只要 7.0 GB）；每层都有可训练参数的方法，梯度要一直传到第一层，显存就大（BitFit、Adapter、AdaptFormer 约 12.8 GB，LoRA 19.9 GB）。

3. 能不能合并决定推理速度。LoRA 和 BitFit 训练完不改变结构，推理不变慢；两种 adapter 多了不能合并的层，单张推理慢 2–3 ms（约 30%–40%）。

2.4　训练是怎么进行的

前面讲了“训练哪些参数”，这一节讲“怎样训练”。我们用两个最终模型做贯穿全节的例子：全量微调模型（1019_1319_1_full）和 LoRA 模型（1019_1319_3_lora）。

2.4.1　一次训练步（step）里发生了什么

1. **取一批照片**：一次拿 256 张（batch size = 256），连同它们的标签。
2. **数据增强**：每张照片随机裁剪、随机左右翻转（见本节后面的“数据增强”）。
3. **前向传播**（forward pass）：照片经过模型，得到每张照片的 10,000 个分数（logits）。
4. **算损失**：把分数和真实标签比较，得到一个数，表示“错得有多严重”。这就是损失（loss）。
5. **反向传播**（backward pass）：从损失出发，算出每个可训练参数的梯度（gradient），也就是“这个参数往哪个方向改，损失会变小”。
6. **梯度裁剪**：如果所有梯度合起来太大，就等比例缩小（见本节后面的“梯度裁剪”）。
7. **更新参数**：优化器 AdamW 按照当前学习率，把每个参数沿着让损失变小的方向挪一小步。

这样一次叫一个 **step**（步）。把所有训练照片都用过一遍叫一个 **epoch**（轮）。

step 和 epoch 的数量

全量微调：450,000 张 ÷ 256 ≈ 1,757 步/epoch，8 个 epoch 共 **14,056 步**。
LoRA：430,000 张（每类 43 张）÷ 256 ≈ 1,679 步/epoch，3 个 epoch 共 **5,037 步**。

2.4.2　logits、softmax 和交叉熵损失

softmax：把分数变成概率

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^{K} e^{z_j}}$$

- z_c：第 c 类的分数（logit）；$K = 10{,}000$ 是类别数。
- e^z：指数函数，把任意分数变成正数，而且分数高一点，结果就大很多。
- 分母把所有类加起来，保证所有 p_c 加起来等于 1。

交叉熵损失（cross-entropy）

$$L = -\ln p_y$$

y 是正确的类别，p_y 是模型给正确类别的概率。p_y 越接近 1，损失越接近 0；p_y 越小，损失越大。它衡量的是“模型对正确答案有多大把握”。

一个只有 3 类的简化例子（不含 label smoothing）

模型给三类的分数是 $(2.0,\ 1.0,\ 0.1)$。

- $e^{2.0} \approx 7.39$，$e^{1.0} \approx 2.72$，$e^{0.1} \approx 1.11$，总和 ≈ 11.21。
- 概率：$7.39/11.21 \approx 0.66$，$2.72/11.21 \approx 0.24$，$1.11/11.21 \approx 0.10$。
- 如果正确答案是第一类：$L = -\ln 0.66 \approx 0.42$（损失小，答得好）。
- 如果正确答案是第三类：$L = -\ln 0.10 \approx 2.31$（损失大，答错了）。

如果模型完全在瞎猜，10,000 类的概率都是 $1/10{,}000$，损失就是 $\ln 10{,}000 \approx 9.21$。这就是随机分类头起始损失（9.28）的来历。

训练损失只能互相比较

2.2 节里的起始损失（LP 头 2.29、原型头 4.39、随机头 9.28）是 LoRA 训练前 100 步的**平均训练损失**，其中包含了 label smoothing 和数据增强的影响。所以不要把它们换算成“正确类的概率”，只能互相比较：9.28 约等于瞎猜（$\ln 10{,}000 = 9.21$）；4.39 表示大致知道每类长什么样，但还很不确定，梯度很强；2.29 表示已经在这批训练照片上拟合过，梯度很弱。

2.4.3　label smoothing 0.1：别让模型太自信

普通的交叉熵要求正确类的概率是 1，其他类都是 0。Label smoothing（标签平滑）把目标改得“温和”一点：正确类的目标是 $1 - \varepsilon + \varepsilon/K$，其他每一类都是 ε/K。我们用 $\varepsilon = 0.1$，$K = 10{,}000$，所以正确类的目标是 0.90001，其他每类是 0.00001。

带 label smoothing 的交叉熵

$$L = (1-\varepsilon)\,(-\ln p_y) + \varepsilon \cdot \frac{1}{K}\sum_{k=1}^{K}(-\ln p_k)$$

第一项：大部分权重（0.9）仍然要求正确类的概率高。第二项：一小部分权重（0.1）要求模型对所有类都留一点点概率，不能把其他类压到 0。

接着上面 3 类的例子

$\varepsilon = 0.1$，$K = 3$，正确类是第一类。目标变成 $(0.933,\ 0.033,\ 0.033)$。损失 $= 0.933 \times 0.42 + 0.033 \times 1.42 + 0.033 \times 2.31 \approx 0.52$，比不平滑时的 0.42 大一些。因为它不再只奖励“对正确答案很自信”，还惩罚“把别的类压得太死”。

为什么要用它：我们每类只有 45 张照片，模型很容易对训练照片过度自信，也就是死记。label smoothing 让模型留一点余地，是一种温和的正则化（regularization，防止过拟合的手段）。它也解释了为什么训练损失不会降到 0：即使完全答对，带平滑的损失也有一个下限。

2.4.4　反向传播和梯度：怎么知道往哪里改

梯度告诉我们：每个参数稍微增加一点，损失会增加还是减少，变化有多快。参数沿着梯度的反方向改一小步，损失就会下降。这叫**梯度下降**（gradient descent）。

生活类比

你在大雾中的山坡上，想走到山谷最低处（损失最小）。你看不见远处，只能用脚感觉脚下哪个方向往下倾斜（梯度），然后朝那个方向走一小步（学习率决定步子大小），一步一步走下去。

反向传播（backpropagation）就是从损失开始，用链式法则一层一层往回算梯度。只有**可训练**的参数才需要梯度，但为了算出底层参数的梯度，梯度信号必须穿过上面所有的层。这就是为什么“梯度要传多远”决定了训练显存。

2.4.5　优化器 AdamW 和 weight decay

AdamW 做了什么

最简单的梯度下降是“参数 = 参数 − 学习率 × 梯度”。Adam 更聪明：

- 它记住梯度的“平均方向”（动量，记为 m），让更新更平稳，不会被一两步的噪声带偏。
- 它记住梯度平方的平均大小（记为 v），给每个参数自动调整步长：一直有大梯度的参数，步子小一点；梯度一直很小的参数，步子大一点。

weight decay（权重衰减）

每一步都把参数往 0 的方向拉一点点：参数 ← 参数 − 学习率 $\times\lambda\times$ 参数。这让参数不要变得太大，模型更“简单”，不容易死记。AdamW 的 W 指的就是把这一项和 Adam 的更新分开来做（decoupled weight decay）。我们的设置：全量微调 $\lambda = 0.05$（偏置、归一化参数、[CLS] 和位置编码不衰减）；LoRA $\lambda = 10^{-4}$。

AdamW 的简化写法

$$\theta \leftarrow \theta - \eta\left(\frac{m}{\sqrt{v}+\epsilon} + \lambda\theta\right)$$

θ：一个参数；η：学习率；m：梯度的移动平均；v：梯度平方的移动平均；ϵ：防止除以 0 的极小数；λ：weight decay 系数。括号里第一项是“按梯度走”，第二项是“往 0 拉一点”。

2.4.6　学习率：先预热（warmup），再余弦下降（cosine）

学习率（learning rate）是每一步走多大。我们的学习率随训练变化：

1. **预热（warmup）**：一开始从接近 0 线性增加到最大值。刚开始模型和新分类头还没“磨合”，步子太大容易把预训练特征破坏掉。全量微调预热 1 个 epoch；LoRA 预热 0.3 个 epoch。
2. **余弦衰减（cosine decay）**：之后按余弦曲线慢慢降到 0。前期步子大、学得快，后期步子小、精细调整。

预热之后的余弦衰减

$$\eta(s) = \eta_{\max} \cdot \frac{1}{2}\left(1 + \cos\left(\pi \cdot \frac{s - s_{\text{warmup}}}{S - s_{\text{warmup}}}\right)\right)$$

s：当前步数；S：总步数；s_{warmup}：预热步数；$\eta_{\max}$：最大学习率（全量微调 10^{-4}，LoRA 10^{-3}）。

预热刚结束时 $\cos 0 = 1$，学习率是最大值；到最后一步 $\cos \pi = -1$，学习率是 0。

LoRA 的学习率时间表

LoRA 共 5,037 步，每个 epoch 1,679 步，预热 $0.3 \times 1{,}679 \approx 503$ 步。第 0 到 503 步：学习率从接近 0 涨到 10^{-3}。之后的 4,534 步按余弦下降：走到一半（约第 2,770 步）时是 5×10^{-4}，最后一步是 0。

2.4.7　分层学习率衰减 0.75（只在全量微调里用）

全量微调里，不同层用不同的学习率：

$$\text{第}l\text{ 层的学习率} = 10^{-4} \times 0.75^{\,13-l}$$

分类头和最终 LayerNorm 算第 13 层，学习率是完整的 10^{-4}；最后一个 block（第 12 层）乘 0.75；第一个 block（第 1 层）乘 $0.75^{12} \approx 0.032$，只有约 3%；图块嵌入（第 0 层）乘 $0.75^{13} \approx 0.024$。

为什么：底层学的是边缘、颜色、纹理这类通用特征，对新任务也有用，改得太多反而有害；顶层更接近具体类别，需要多改。这和部分微调的直觉一样，但更柔和：每一层都改，只是改的幅度从上到下逐渐变小。

2.4.8　drop path 0.1 和梯度裁剪 1.0

drop path（随机深度，stochastic depth）

训练时，每个 block 的注意力分支和 MLP 分支有一定概率被整个“跳过”，只走残差那条捷径。按 timm 的默认做法，跳过概率从第一个 block 的 0 逐渐增加到最后一个 block 的 0.1。测试时所有 block 都正常使用。

生活类比

球队训练时，每次随机让几个队员休息，剩下的人也必须能配合好。这样比赛时（测试时）全员上场，每个人都不会过度依赖某一个队友。drop path 让模型不过度依赖某一层，是一种防止死记的正则化。

梯度裁剪（gradient clipping）

把所有梯度看成一个很长的向量，如果它的长度超过 1.0，就等比例缩小到 1.0。这能防止某一步因为一个异常的 batch 而走出一大步，让训练更稳定。

2.4.9　bf16 混合精度和 batch size

bf16

普通的小数用 32 位（fp32）存储。bf16（bfloat16）只用 16 位：表示范围和 fp32 一样大，但精度低一些。用 bf16 做大部分计算，显存减半、速度更快，对准确率几乎没影响。A100 支持 bf16，所以我们所有训练和延迟测试都用 bf16。“混合精度”的意思是：大部分计算用 bf16，少数对精度敏感的地方仍用 fp32。

batch size 256

每一步用 256 张照片算一次平均梯度。batch 越大，梯度越稳定，GPU 利用率越高，但显存也越

大。我们在 80 GB 的 A100 上用 256。

2.4.10　数据增强：从 45 张照片里“造”出更多变化

RandomResizedCrop（随机缩放裁剪），scale = (0.25, 1)

每次读一张训练照片时，随机选一块区域，面积是原图的 25% 到 100%，位置和长宽比也随机，然后把这块缩放成 224 × 224。同一张照片每个 epoch 被裁成不同的样子。

水平翻转（horizontal flip）

以 50% 的概率把照片左右翻转。一只朝左的鸟和朝右的鸟是同一类，翻转不改变类别。

为什么：每类只有 45 张照片，如果每次都给模型看一模一样的照片，它很快就会背下来。增强让每次看到的都略有不同，迫使模型学“真正的特征”。测试时不做随机增强：照片先缩放到 248 像素，再从中间裁出 224 × 224。

随机裁剪的一个副作用

随机裁剪经常只取原图的一部分再放大到 224，所以训练时物体看起来**比实际更大**。测试时用的是整张图的中心裁剪，物体看起来相对更小。这个“训练和测试时物体大小不一致”的现象，正是 2.6 节中“用更大的图片测试会更准”的原因。

2.4.11　两个最终模型的配置对比

设置	全量微调（1019_1319_1_full）	LoRA（1019_1319_3_lora）
训练什么	backbone 85.8M + 分类头 7.69M	LoRA 1,179,648 + 分类头 7.69M；backbone 冻结
分类头起点	原型头 × 40	原型头 × 40
训练数据	每类 45 张（全部）	每类 43 张（留 2 张做 holdout）
损失函数	交叉熵，label smoothing 0.1	同左
优化器	AdamW，weight decay 0.05，梯度裁剪 1.0	AdamW，weight decay 10^{-4}，梯度裁剪 1.0
学习率	10^{-4}，每往下一层乘 0.75	10^{-3}，不分层
正则化	drop path 0.1	无 drop path
训练长度	8 个 epoch：1 个预热 + 余弦	3 个 epoch：0.3 个预热 + 余弦
其他	bf16，batch 256，随机裁剪 + 翻转	同左
步数和成本	14,056 步；1.03 GPU 小时；18.9 GB	5,037 步；0.54 GPU 小时；19.9 GB
推理	翻转 TTA：14.95 ms/张，1,919 张/秒	合并后：7.61 ms/张，3,869 张/秒
结果	验证集 76.22%，测试集 76.09%	验证集 71.90%，测试集 71.67%

2.5　怎么评估：怎么判断模型好不好

2.5.1　准确率：top-1 和 top-5

- **top-1 准确率**：模型分数最高的那一类正好是正确答案的比例。这是项目最主要的指标，排行榜也用它。
- **top-5 准确率**：正确答案落在分数最高的 5 类之中的比例。它总是比 top-1 高。例如我们的 LoRA 模型在 holdout 上 top-1 是 71.98%，top-5 是 89.07%。

验证集每类正好 5 张，类别完全平衡，所以 top-1 准确率也等于“每一类准确率的平均值”。

2.5.2　三个数据集：holdout、验证集、测试集

数据集	规模和标签	用途
holdout（我们自己留出的）	每类 2 张，共 20,000 张，标签已知	每天的实验、比较方法、调参；随时可以用
CodaBench 验证集（val）	每类 5 张，共 50,000 张，标签隐藏	第一阶段：上传预测，服务器给分
CodaBench 测试集（test）	每类 10 张，共 100,000 张，标签隐藏	第二阶段：最终成绩；一共只能提交 2 次，必须用第一阶段的同一个模型

为什么要自己留 holdout：验证集和测试集的标签都是隐藏的，只能上传到服务器打分。如果每个实验都上传，不但慢，还等于在“用考试题调参”。所以我们从训练集里每类拿出 2 张不参与训练，作为自己的“模拟考试”。它和服务器的结果非常接近：全量微调 72.31% 对 72.57%，LoRA 71.98% 对 71.90%，差距在 0.3 个百分点以内。所以本手册里很多比较用的都是 holdout 的数字。

说数字时一定要说清楚来自哪个数据集

同一个模型在不同数据集上的数字不同。例如 LoRA：holdout 71.98%，验证集 71.90%，测试集 71.67%。全量微调的“73.38%”是 3 个 epoch 的 holdout 结果，“76.22%”是最终模型的验证集结果，两者不能直接比较。

2.5.3　成本指标：参数、延迟、吞吐量、GPU 小时、显存

项目要求每个方法都报告四个指标：top-1 准确率、可训练参数、推理延迟、训练 GPU 小时。我们还额外报告了峰值显存。

- **可训练参数**：训练中被更新的参数个数。按项目习惯不含分类头（例如 LoRA 1,179,648 个，占 1.375%）。
- **延迟**（latency，batch size = 1）：处理一张照片要多少毫秒。只算模型的前向计算，用 bf16，重复 100 次取平均。例如全量微调 7.78 ms，LoRA 合并后 7.61 ms，Adapter 9.74 ms。
- **吞吐量**（throughput，batch size = 256）：一秒能处理多少张照片。延迟关心“一张有多快”，吞吐量关心“大批量时一秒处理多少”。
- **GPU 小时**（GPU-hours）：用了几块 GPU × 用了多少小时。我们只用一块 A100，所以就是训练用的小时数。计入的时间包括：为原型头提取特征（约 6 分钟）、所有训练步、每个 epoch 后的 holdout 评估。
- **峰值显存**：训练过程中 PyTorch 最多占用了多少 GPU 内存。

预算是怎么换算的

项目规定每次训练最多“2 小时 × 48 GB 显存”，即 $2 \times 48 = 96$ GB· 小时，允许 ±10%。我们用的是 80 GB 的 A100，所以 $96 \div 80 = 1.2$ 小时（加上 10% 是 1.32 小时）。最终全量微调 1.03 小时，LoRA 0.54 小时，都在预算内。

2.5.4　怎样判断一个结果是好是坏

1. **和参考值比**：项目说明给了参考结果。我们的最终模型在验证集上比参考的全量微调高 5.4 个百分点（76.22% 对 70.8%），比参考的 LoRA 高 3.6 个百分点（71.90% 对 68.3%）。
2. **看差距是否大于噪声**：holdout 有 20,000 张，准确率本身就有随机波动。每个设置我们只跑了一次（种子 0），所以小于约 0.3 个百分点的差距都应该当作噪声，不能说谁更好。
3. **看是否在预算内**：再准，超出预算也不能算进结果。
4. **看测试集和验证集是否一致**：两个最终模型的测试集结果都在验证集的 0.25 个百分点以内，说明模型没有对验证集“过拟合”，结果可信。
5. **看有没有过拟合的信号**：如果训练损失在下降，但 holdout 准确率也在下降，说明模型在死记训练照片。LoRA 从 3 个 epoch 加到 6 个 epoch，以及 LP 头的实验，都出现了这个信号。

0.3 个百分点的噪声是怎么来的

准确率可以看成 20,000 次“对或错”的抽样。它的标准误差（standard error）是：

$$\mathrm{SE} = \sqrt{\frac{p\,(1-p)}{n}} = \sqrt{\frac{0.72 \times 0.28}{20{,}000}} \approx 0.0032 = 0.32 \text{ 个百分点}$$

$p \approx 0.72$ 是准确率，$n = 20{,}000$ 是 holdout 的照片数。也就是说，即使模型完全不变，换一批 holdout 照片，准确率也可能上下波动 0.3 个百分点左右。所以 0.12、0.26 这么小的变化不能当真，而 2.5、3.1 这样的差距远远超过噪声，是可信的。

2.6　测试时增强（TTA）和输入分辨率

2.6.1　翻转 TTA（flip test-time augmentation）

是什么

Test-time augmentation（测试时增强）：预测时，把一张照片和它的左右镜像各送进模型一次，把两次的 10,000 个分数相加，再选分数最高的一类。

为什么有用

模型对同一张照片的镜像，判断会稍有不同。两次结果相加，相当于让两个“略有不同的视角”投票，偶然的错误会被抵消一部分。

效果和代价

在验证集上提高 0.27 个百分点（72.57% → 72.84%），改变了 4.6% 的预测；但每张照片要算两次，推理时间翻倍（最终全量微调模型单张 14.95 ms）。只有最终的全量微调模型用了 TTA，最终的 LoRA 模型没有用。

生活类比

考试时把一道题从正面看一遍，再换个角度看一遍，两次的判断综合起来，更不容易看错。代价是每道题要花两倍时间。

2.6.2　输入分辨率：截止日期之后发现的“最大收益”

是什么

把输入图片从 224 像素放大到 288 或 320 像素，模型结构和参数完全不变。

为什么不用新参数就能处理大图

- 图块大小还是 16 像素，图片变大，图块就变多：288 像素是 $18 \times 18 = 324$ 块，加 [CLS] 共 325 个 token；320 像素是 $20 \times 20 = 400$ 块，共 401 个 token（224 像素是 197 个）。
- 所有线性层（qkv、proj、fc1、fc2）对每个 token 单独计算，所以 token 多了也能用同样的参数。
- 唯一和图块数有关的是位置编码：它原来是 14×14 的网格。我们用插值（interpolation）把它拉伸成 18×18 或 20×20，就像把一张小地图放大，中间缺的值按周围的值估计出来。这一步不增加任何参数。
- 代价是计算量：token 从 197 个变成 325 个，计算量约为原来的 1.7 倍；训练显存从 18.9 GB 增加到 30.2 GB。

为什么大图更准：FixRes

2.4 节提到，训练时的随机裁剪会让物体看起来比实际大；测试时用中心裁剪，物体看起来相对更小。Touvron 等人在 2019 年的 FixRes 论文里指出了这个“训练和测试物体大小不一致”的问题：测试时用稍大一点的图片，物体的大小就能和训练时对得上。

我们的结果（全部是 3 个 epoch 的 holdout 实验）

- 224 训练、224 测试：73.38%（基线）。
- 同一个模型，只是测试时用 288：75.50%（+2.1），完全不用重新训练。
- 288 训练、288 测试：77.23%（+3.9），0.68 GPU 小时，仍在预算内，但显存要 30.2 GB。
- 288 训练、320 测试：77.78%（+4.4），是最好的组合，也符合 FixRes 的预测：测试尺寸比训练尺寸稍大一点最好。

这部分结果不是我们的正式成绩

分辨率实验是在截止日期**之后**做的，只有 holdout 结果。把最终配方（8 个 epoch、全部数据）放到 288 像素训练，需要 1.61 GPU 小时，超出了 1.2 小时的预算，而且这个模型**没有测试集成绩**。我们的正式成绩只来自第一阶段的两个模型。它在 slides 里的定位是“截止后学到的教训”：我们很早就研究了 adaptation 方法，却很晚才去看输入本身。

2.7　一张总对比表

2.7.1　七种 adaptation 方法

所有数字都来自同一个 protocol：原型头、3 个 epoch、holdout、A100。可训练参数不含分类头。

方法	训练什么	参数比例	holdout	显存	延迟	能合并
线性探针	只训练分类头	0%	61.94%	3.2 GB	6.96 ms	–
部分微调 K=4	最后 4 个 block	33%	71.88%	7.0 GB	7.41 ms	–
全量微调	所有参数	100%	73.38%	18.9 GB	7.78 ms	–
BitFit	只训练偏置	0.12%	69.81%	12.7 GB	7.33 ms	是
Adapter	串联瓶颈模块	1.39%	71.54%	12.8 GB	9.74 ms	否
AdaptFormer	并联瓶颈分支	1.39%	71.17%	12.8 GB	10.23 ms	否
LoRA	低秩矩阵 A、B	1.38%	71.98%	19.9 GB	7.61 ms	是

（“能合并”只对新增参数的方法有意义：线性探针、部分微调和全量微调本来就不改变模型结构，所以推理也不会变慢；BitFit 改的是原有的偏置，也不增加结构。）

方法	主要优点	主要缺点
线性探针	最快最省（0.10 GPU 小时、3.2 GB）	特征不变，准确率上限低
部分微调	训练最便宜，推理无开销	底层不变；参数多（8%–50%）；收益递减
全量微调	最准，训练更久还会提高	每个任务存一整份模型；显存大
BitFit	参数极少（102,912），推理无开销	能力有限，比 LoRA 低 2.2
Adapter	准确率接近 LoRA，训练快、显存少	不能合并，推理慢约 2.4 ms
AdaptFormer	不改原 MLP 输出，训练快	不能合并，推理慢约 2.9 ms
LoRA	最准的 PEFT，可合并，推理零开销	训练显存和速度不比全量微调好；训练久会过拟合

2.7.2　三种分类头初始化

	随机头	LP 头	原型头
训练前准确率	接近 0	62.1%	44.0%
LoRA 3 epoch（holdout）	71.95%	69.45%	72.00%
是否拟合过训练照片	否	是	否
结论	起点差，但最终和原型头打平	起点最高，最终最差（死记）	起点合理、学得快，是我们的选择

本章最重要的四个结论

1. 分类头的起点最关键。常用的 LP-FT 起点在我们测试的三种方法上都让结果变差 2.1–3.1 个百分点，因为 LP 头是在同一批训练照片上拟合出来的，会导致死记。原型头（weight imprinting）没有拟合任何照片，起点合理、学得快。

2. 训练的位置比训练的数量更重要。在每个 block 里做小改动（LoRA 1.38%、BitFit 0.12%），胜过在顶部几层做大改动（部分微调 8%–33%）。

3. 训练显存跟着梯度走，推理速度看能否合并。参数少不等于显存少（LoRA 19.9 GB）；LoRA 可以合并，所以推理不变慢，adapter 不行。

4. 全量微调最准，LoRA 是最好的高效方法。全量微调训练越久越好（8 个 epoch），LoRA 3 个 epoch 最好，训练更久会过拟合（我们猜是分类头的原因，还没有验证）。

第 3 章　重要技术概念（术语表）

这一章把 slides、讲稿和报告里出现的专业术语集中在一起，按主题分成 5 组。每个术语有三栏：

- **术语**：英文原名（答辩时老师会用英文说）和中文名；
- **它到底是什么意思**：用一两句大白话解释，不背定义；
- **生活例子 / 在我们项目里**：一个生活类比，或者它在我们项目里出现在哪、对应什么数字。

第一次读，可以从头到尾快速浏览一遍；准备答辩时，把它当字典查。每个方法的完整讲解在第 2 章，公式的推导和计算例子在第 4 章。

怎么用这张表

不需要一次全记住。先把每组表格里加粗的英文名读一遍，确认自己能用一句中文说出它的意思；说不出来的，再看第二栏和第三栏。答辩前最少要能脱口而出的词：backbone、head、fine-tuning、LoRA、prototype、linear probe、holdout、overfitting、cross-entropy、GPU-hours。

3.1　数据与评估

术语	它到底是什么意思	生活例子 / 在我们项目里
dataset 数据集	一大批图片和它们的正确答案放在一起，用来训练和检验模型。	我们的数据有 10,000 个类，训练集共 450,000 张图。
class / label 类别 / 标签	类别是要区分的“种类”；标签是某一张图的正确答案，也就是它属于哪一类。	10,000 个类，类名被隐藏，只看到编号。训练集有标签，val 和 test 的标签看不到。
fine-grained classification 细粒度分类	类和类之间差别很小的分类任务，比如区分不同种的鸟，而不是区分“鸟”和“汽车”。	比如分清两种长得很像的鸟或蛾。同一类的照片差别又很大：一只鸟和一群鸟、一朵花和整棵灌木。
training set 训练集	模型用来学习的数据，模型能看到图片和答案。	450,000 张，每类正好 45 张。
holdout set 留出集	从训练集里拿出一小部分，不参加训练，只用来打分。因为它模型没见过，所以分数可信。	每类留 2 张，共 20,000 张，固定随机种子。所有对比实验都看它；它和 CodaBench 的分数相差不到约 0.3 分。
validation set (val) 验证集	用来检查模型、挑选设置的数据。	CodaBench 上的 val：50,000 张（每类 5 张），标签隐藏，上传预测后才看到分数。最终全量模型 val 76.22%。
test set 测试集	最后一次性检验、决定排名的数据，不能拿来调参数。	100,000 张（每类 10 张）。我们的 test：全量 76.09%，LoRA 71.67%。
CodaBench	课程使用的在线评测平台：上传预测文件，它算出准确率并排名。	val 和 test 的分数都来自它。
Phase 1 / Phase 2 第一 / 第二阶段	比赛的两个阶段：Phase 1 用 val 打分；Phase 2 用 test 打分，只能提交 2 次，而且必须用 Phase 1 的同一个模型。	我们的两个 test 结果都来自 Phase 1 的模型 1019_1319_1_full 和 1019_1319_3_lora。
leaderboard 排行榜	各组分数的排名。	老师说排名只是参考，分析、迭代、失败的经验和新想法更重要。

术语	它到底是什么意思	生活例子 / 在我们项目里
top-1 accuracy 准确率	模型给出的第一名答案正好是正确答案的比例。	76.22% 意思是：100 张图里大约猜对 76 张。
top-5 accuracy	正确答案只要在模型给出的前 5 名里，就算对。它一定不低于 top-1。	像考试允许写 5 个候选答案。报告里作为补充指标。
overfitting 过拟合	在训练数据上越来越好，在没见过的数据上却变差。	LoRA 从 3 个 epoch 训到 6 个：训练 loss 从 1.95 降到 1.70，holdout 反而从 71.98% 掉到 71.52%。
memorization 死记硬背	模型记住了训练图片本身，而不是学会能用在新图片上的规律。过拟合常常就是因为死记硬背。	像背下了练习题的答案，换一道题就不会。每类只有 45 张，很容易背下来。
generalization 泛化	在没见过的新数据上也表现好的能力。这才是训练的真正目标。	我们只信 holdout、val、test 上的分数，不信训练 loss。
noise 噪声（随机误差）	因为随机性产生的小波动，不代表模型真的变好或变差。	holdout 上 0.3 分以内的差别算噪声。例如学习率减半的 -0.12 就是噪声。
standard error 标准误	用有限的样本估计一个数时，典型的误差大小。样本越多，误差越小。	在 20,000 张图上测 72% 的准确率：$\sqrt{0.72 \times 0.28/20{,}000} \approx 0.32$ 个百分点，所以说“约 0.3 分是噪声”。
seed 随机种子	控制随机过程的一个数字。种子一样，随机的结果就能重复出来。	我们所有实验都用 seed 0，每个设置只跑一次，这是我们承认的局限。

术语	它到底是什么意思	生活例子 / 在我们项目里

3.2　模型与结构

术语	它到底是什么意思	生活例子 / 在我们项目里
neural network 神经网络	由很多层简单计算串起来的大函数，靠大量可调的数字把输入变成输出。	输入一张图，输出 10,000 个分数，每类一个。
parameter / weight 参数 / 权重	网络里可以调整的数字。训练就是不断调这些数字，让答案更准。	ViT-B/16 有 85,798,656 个参数（约 85.8M，M = 百万）。
bias 偏置	线性层 $y = Wx + b$ 里加上的那个常数 b，像给结果整体“抬高或压低一点”。	BitFit 只训练 bias，一共 102,912 个。
linear layer 线性层（全连接层）	把输入向量乘一个矩阵再加偏置：$y = Wx + b$。参数量 = 输入维度 × 输出维度 + 输出维度。	每个 block 有 4 个：qkv、proj、fc1、fc2。fc1 是 768 → 3,072，约 2.36M 参数。
activation function 激活函数	夹在线性层之间的“非线性开关”。没有它，多层线性层叠起来还是一个线性层。	ReLU：负数变 0、正数不变；GELU 是它的平滑版。ViT 的 MLP 用 GELU，Adapter 用 ReLU。
backbone 主干网络	负责“看懂图片、提取特征”的大部分网络。	我们的 backbone 是课程推荐的 ViT-B/16，全程不换。
head 分类头	接在 backbone 后面的最后一层，把特征变成每一类的分数。	新 head：768 → 10,000，共 7,690,000 个参数，约为 backbone 的 9%。
feature 特征	backbone 对一张图的“数字描述”，是一串数字（向量）。相似的图，特征也相近。	每张图一个 768 维的 [CLS] 特征。
Transformer	一种以“注意力”为核心的网络结构，最早用于翻译文字，后来也用于图片。	ViT 就是把 Transformer 用在图片上。
ViT / ViT-B/16 视觉 Transformer	把图片切成小方块，当作一串“单词”交给 Transformer。B = Base（中等大小），16 = 每块 16 × 16 像素。	我们用 timm 的 vit_base_patch16_224.augreg2_in21k_ft_in1k：先在 ImageNet-21k、再在 ImageNet-1k 上训练过。
patch 图块	图片被切成的小方块。	224 × 224 的图切成 14 × 14 = 196 块。
patch embedding 图块嵌入	把每个图块变成一个 768 维向量的那一层（16 × 16、步长 16 的卷积）。	590,592 个参数（图上标 0.59M）。
token / embedding 词元 / 嵌入向量	token 是 Transformer 处理的基本单位；embedding 是表示它的一串数字。图片里一个 token 就是一个图块的向量。	196 个图块 token + 1 个 [CLS] = 197 个 token，每个 768 维，所以图上写“197 × 768”。
[CLS] token 分类 token	额外加在最前面的一个 token，专门用来汇总整张图的信息。最后只拿它去分类。	像开会时专门负责做总结的人。最后一层的 [CLS] 向量（768 维）送进 head。
position embedding 位置嵌入	加在每个 token 上的“位置信息”，让模型知道每个图块在图中的位置。	197 × 768。换成 288 像素时，要把 14 × 14 的网格插值成 18 × 18，不增加参数。
Transformer block Transformer 模块	Transformer 的一个重复单元：注意力 + MLP，各带一个 LayerNorm 和一条残差连接。	ViT-B/16 有 12 个一模一样的 block，每个 7,087,872 个参数（约 7.09M）。
self-attention 自注意力	每个 token 去“看”所有 token，按相关程度给它们打权重，再把信息加权汇总。	像开会时每个人先听全场发言，再根据和自己有多相关来整理笔记。

术语	它到底是什么意思	生活例子 / 在我们项目里
Q / K / V 查询 / 键 / 值	注意力里的三种向量：Q 表示“我想找什么”，K 表示“我能提供什么”，V 是“我的实际内容”。Q 和 K 越匹配，就越多地取用那个 V。	qkv 层一次算出三者：768 → 2,304 = 3 × 768，约 1.77M 参数。
multi-head attention 多头注意力	把注意力分成几组并行计算，每组关注不同的关系，最后拼起来。	12 个头，每头 64 维（12 × 64 = 768）。注意力本身没有参数。
MLP 多层感知机	block 里的两层全连接网络：先放大、过激活函数、再缩回去，逐个 token 加工信息。	768 → 3,072 → 768，占一个 block 约 2/3 的参数。
LayerNorm (LN) 层归一化	把每个 token 的 768 个数调到相近的尺度，让训练更稳定。它有缩放和平移两组参数。	每个 LN 有 1,536 个参数：768 个缩放 + 768 个平移（β）。
residual connection 残差连接	把一层的输入直接加到它的输出上：$x + f(x)$。信息和梯度可以“抄近路”，深层网络更好训练。	每个 block 有两处“+”。
logits 原始分数	head 输出的、还没变成概率的分数。分数最高的类就是预测结果。	每张图 10,000 个 logits。
softmax	把一组分数变成加起来等于 1 的概率。分数越高概率越大，而且差距会被放大。	分数 (2, 1, 0) 变成概率约 (0.67, 0.24, 0.09)。
prototype 原型	一个类的“平均长相”：把这类所有图片的特征（先归一化）平均，得到一个方向。	10,000 个类各有一个 768 维原型，用来初始化 head。
L2 normalization L2 归一化	把向量除以它自己的长度，变成长度为 1 的单位向量，只保留“方向”。	(3, 4) 的长度是 5，归一化后是 (0.6, 0.8)。
cosine similarity 余弦相似度	两个向量方向有多接近：1 表示同方向，0 表示无关，−1 表示相反。	原型 head 的分数约等于 40 × cos（图片特征，类原型）。
temperature / logit scale 温度 / 缩放系数	乘在相似度上的系数，控制 softmax 概率有多“尖锐”。太小时概率都差不多，太大时又过度自信。	我们从 5、10、15、20、30、40、60、80 里按交叉熵挑出 40。

3.3　训练

术语	它到底是什么意思	生活例子 / 在我们项目里
training 训练	反复做“预测 → 算出错多少 → 调整参数”，让模型越来越准。	像做题、对答案、改正，再做下一题。
forward pass 前向传播	输入一路算到输出，得到预测。	一张图经过 patch embedding、12 个 block、head，得到 10,000 个分数。
backward pass 反向传播	从 loss 出发反方向计算，得到每个参数应该怎么改（梯度）。为此要保存前向传播时的中间结果。	梯度要往回传多远，决定显存用多少：部分微调 $K = 4$ 只传过 4 个 block（7.0 GB），LoRA 要传到第 1 个 block。
gradient 梯度	告诉每个参数“往哪个方向改、改多少，loss 会下降”的信息。	像下山时脚下的坡度，指出哪边是下坡。
loss function 损失函数	衡量预测有多错的一个数，越小越好。训练就是让它变小。	注意：训练 loss 低不等于模型好，见第 8 页。
cross-entropy 交叉熵	分类最常用的 loss：$-\log$(正确类的概率)。正确类的概率越低，惩罚越重。	概率 0.9 → loss 0.11；概率 0.1 → loss 2.30；1 万类里完全瞎猜 → $\ln 10{,}000 \approx 9.21$。
label smoothing 标签平滑	不要求模型 100% 确定：目标改成正确类 0.9，剩下的 0.1 平均分给所有类。防止过度自信。	我们两个最终模型都用 0.1。

术语	它到底是什么意思	生活例子 / 在我们项目里
optimizer 优化器	根据梯度更新参数的具体规则。	我们用 AdamW。
AdamW	最常用的优化器之一：自动给每个参数调合适的步长，并把权重衰减单独处理（W 指 decoupled weight decay）。	全量微调和 LoRA 都用它。
learning rate (LR) 学习率	每一步改参数的幅度。太大会改坏，太小学得太慢。	全量微调 1e-4（= 0.0001）；LoRA 1e-3（= 0.001）。
weight decay 权重衰减	每一步把参数轻轻往 0 拉一点，防止参数变得太大、过拟合。	全量微调 0.05，LoRA 1e-4。
warmup 预热	训练刚开始时，学习率从很小慢慢升到目标值，避免一开始就把预训练好的参数改乱。	全量微调预热 1 个 epoch，LoRA 预热 0.3 个 epoch。
cosine schedule 余弦退火	预热之后，学习率沿着余弦曲线慢慢降到 0：先降得慢，中间快，最后又慢。	像跑步冲线前慢慢减速。
layer-wise LR decay 逐层学习率衰减	越靠近输入的层，学习率越小。底层学的是通用的边缘、纹理，不该大改。	每往下一层乘 0.75；第 1 个 block 只有 head 学习率的约 3%（$0.75^{12} \approx 0.032$）。
drop path 随机深度	训练时随机“跳过”某些 block 的计算，只走残差这条近路。模型不能依赖某一层，是一种正则化。	全量微调用 0.1；LoRA 没用。
regularization 正则化	各种防止过拟合的手段的总称。	weight decay、drop path、label smoothing、数据增强都算。
gradient clipping 梯度裁剪	梯度太大时按比例缩小，防止一步改得太猛。	梯度的范数上限设为 1.0。
batch / batch size 批 / 批大小	一次一起计算的一组图片，以及这组图片的数量。	我们的 batch size 是 256。
step 步	一次参数更新，也就是处理完一个 batch 后改一次参数。	最终全量模型一共 14,056 步，LoRA 5,037 步。
epoch 轮	把整个训练集完整地过一遍。	450,000 张 ÷ 256 ≈ 1,757 步为一轮。全量训 8 轮，LoRA 训 3 轮。
data augmentation 数据增强	训练时对图片做随机变化，让模型见到更多“变体”，不容易死记。	像同一道题换个说法再练一次。
RandomResized-Crop 随机缩放裁剪	随机裁下原图的一块（我们是面积的 25% 到 100%），再缩放到 224 × 224。	裁的块越小，物体被放得越大，这正是后面 FixRes 的原因。
horizontal flip 水平翻转	把图片左右镜像。	训练时随机翻转；测试时的 flip TTA 也用它。
bf16 / mixed precision 混合精度	用更短的 16 位数字格式计算，更快、更省显存，精度损失很小。	所有训练都用 bf16，所以要用 A100、L4、H100 这类支持它的 GPU。
freeze 冻结	训练时让某些参数保持不变。	LoRA 冻结整个 backbone，只训练 A、B 矩阵和 head。
hyperparameter 超参数	训练前由人设定、不是训练学出来的设置。	学习率、epoch 数、LoRA 的秩 r、温度 40 都是超参数。

3.4　微调方法

术语	它到底是什么意思	生活例子 / 在我们项目里
pretraining 预训练	先在超大的通用数据集上训练模型，让它学会通用的“看图”能力。	我们的 backbone 先在 ImageNet-21k（名字里的 21k 指约 2.1 万个类别）、再在 ImageNet-1k 上训练过。
ImageNet-21k / ImageNet-1k	两个著名的大型图片数据集：21k 指约 2.1 万个类别，1k 指 1,000 个类别。	我们的 backbone 先在 21k 上预训练、再在 1k 上微调，这正是课程推荐的版本。
in21k-only checkpoint 只用 21k 的版本	只在 ImageNet-21k 上训练、没有再在 1k 上微调的 ViT-B/16。课程资料也提到了它。	截止后我们检查过两个：线性探针 61.05% 和 51.07%，都不如我们用的 61.94%，所以不换。
AugReg	一种训练 ViT 的配方，加强了数据增强和正则化（Steiner et al.）。	backbone 名字里的 augreg 就是指它。
transfer learning 迁移学习	把在一个任务上学到的本领用到新任务上。	像会骑自行车的人学骑摩托车更快。
fine-tuning 微调	拿预训练好的模型，在新数据上继续训练，让它适应新任务。	本项目的核心：把 ViT-B/16 适配到 10,000 个新类别。
adaptation method 适配方法	“怎么微调”的具体方案：训练哪些参数、加不加新模块。	我们比较了 7 种：线性探针、部分微调、全量微调、BitFit、Adapter、AdaptFormer、LoRA。
full fine-tuning 全量微调	backbone 的所有参数都训练。	100%，85.8M。最准：holdout 73.38%；最终模型 val 76.22%。
partial fine-tuning 部分微调	只训练最后 K 个 block（加上最后的 LayerNorm 和 head），下面的全部冻结。	$K = 1, 2, 4, 6$：holdout 67.97%、70.99%、71.88%、72.89%。
linear probe (LP) 线性探针	冻结整个 backbone，只训练 head。它“探测”冻结的特征本身有多好。	holdout 61.94%，只用 0.10 GPU 小时。
zero-shot 零训练	这里指一步都不训练，直接用原型 head 预测。	holdout 43.95%（课程参考 43.7%）。
LP-FT	先训练一个线性探针 head（LP），再在它的基础上微调整个模型（FT）的两步法（Kumar et al., ICLR 2022）。	我们最初也这么做；后来发现在我们的设置里，它在测过的三种方法上都更差，差 2.1 到 3.1 分。
PEFT 参数高效微调	parameter-efficient fine-tuning：只训练很少的参数来适配新任务，省存储，也方便一个模型服务多个任务。	我们比较了 4 种：BitFit、Adapter、AdaptFormer、LoRA。
BitFit	只训练所有 bias（以及 LayerNorm 的平移参数），其他权重全部冻结。	只有 0.12% 的参数，就到了 69.81%。
Adapter 适配器	在每个 block 的 MLP 后面串联一个小模块：768 → 64 → 768，自带一条残差。	1.39% 的参数，holdout 71.54%，但推理要多算两层，延迟 9.74 ms。
AdaptFormer	把同样的小模块并联在 MLP 旁边，输出乘一个小系数 $s = 0.1$ 再加回去。	holdout 71.17%，和 Adapter 差 0.37，接近噪声；延迟 10.23 ms。
bottleneck 瓶颈	先把维度压小再放大（768 → 64 → 768），用很少的参数做变换。	像把一本书先写成提纲，再根据提纲补充。
LoRA 低秩适配	Low-Rank Adaptation：冻结原来的权重 W，在旁边学一个“低秩”的修改量 $\Delta W = BA$，训练完再合并进 W。	4 个线性层都加 LoRA，共 1,179,648 个参数（1.375%）。最终模型 val 71.90%。
rank r / low-rank 秩 / 低秩	一个大矩阵可以近似写成两个“细长”矩阵的乘积；r 是中间那个维度，决定能表达多少种修改。	$r = 8$：一个 768 × 768 的修改量只用 8 × (768 + 768) = 12,288 个数。
LoRA alpha (α)	LoRA 输出的缩放系数，实际乘的是 α/r。	$\alpha = 16$，$r = 8$，所以乘 2。

术语	它到底是什么意思	生活例子 / 在我们项目里
merging 合并	训练完把 LoRA 的修改量加回原权重：$W' = W + (\alpha/r)BA$。之后模型结构和原来一模一样，推理不变慢。	合并后 LoRA 的延迟是 7.61 ms。Adapter 中间有 ReLU，没法这样合并。
weight imprinting 权重印刻	不经过训练，直接把每个类的平均特征“印”进分类头的权重（Qi et al., CVPR 2018）。	我们的原型 head 就是这个方法。方法不是我们发明的，新的是我们的发现。
head initialization head 初始化	训练开始前，head 的权重怎么设置。	我们比了三种：随机、线性探针、原型。线性探针最差。
feature distortion 特征扭曲	LP-FT 论文的担心：随机 head 一开始给出的梯度很乱，会破坏预训练学到的特征。	我们的随机 head 对照并没有变差（71.95% 对 72.00%），因为我们的任务本来就需要特征改变。

3.5　实验与工程

术语	它到底是什么意思	生活例子 / 在我们项目里
GPU / A100 图形处理器	GPU 擅长同时做大量相同的计算，是训练神经网络的主力。A100 是 NVIDIA 的数据中心 GPU。	我们在 Colab 上用 80 GB 显存的 A100。
timm	一个常用的 PyTorch 图像模型库，提供大量预训练模型和工具。	我们的 ViT-B/16 权重来自 timm；换分辨率时，它负责插值位置嵌入。
Colab	Google 的在线 notebook 环境，可以按需租用 GPU。	我们在 Colab 的 A100 上训练，结果存在 Google Drive 里。
VRAM / peak memory 显存 / 峰值显存	GPU 上的内存，放模型参数、梯度和中间结果。峰值是训练过程中用到的最高值。	全量 18.9 GB，LoRA 19.9 GB，288 像素训练 30.2 GB。
GPU-hours GPU 时数	GPU 数量 × 使用小时数。衡量一次训练花了多少计算资源。	全量最终模型 1.03，LoRA 0.54。
compute budget 计算预算	课程规定每次训练最多 2 小时 × 48 GB。按显存换算，在 80 GB 的 A100 上就是 1.2 小时（允许 ±10%）。	所有算进预算的时间：提取特征、训练、每个 epoch 后的 holdout 评估。
latency 延迟	处理一张图需要多长时间。常用 batch size 1（一次只处理一张）来测，最接近实时使用。	LoRA 合并后 7.61 ms；Adapter 9.74 ms；全量模型做 flip TTA 要 14.95 ms。
throughput 吞吐量	每秒能处理多少张图。训练时叫训练速度，推理时用大 batch 测。	推理：最终 LoRA 3,869 张/秒。训练：全量 1,139 张/秒，LoRA 888 张/秒。
TTA 测试时增强	test-time augmentation：测试时对同一张图做几个变体，把结果合起来。	flip TTA：原图和镜像图的 logits 相加；+0.27 分，但要算两遍，延迟翻倍。
ablation 消融实验	每次只改一个设置，其他都不变，看这个设置到底有多大作用。	第 9 页的条形图就是消融。
baseline 基线	用来比较的参考结果。	课程给的 43.7%、70.8%、68.3%；我们先复现，误差都在 0.3 分以内。
protocol 实验协议	所有对比实验共用的一套固定设置，保证结果可以直接比较。	原型 head、3 个 epoch、同一个 holdout、同一种 GPU。
resolution / input size 分辨率 / 输入尺寸	输入图片的边长（像素）。越大细节越多，计算也越多。	224 → 288：token 从 197 个变成 325 个，计算约 1.7 倍。

术语	它到底是什么意思	生活例子 / 在我们项目里
FixRes	训练和测试分辨率不一致的现象（Touvron et al., 2019）：随机裁剪让训练时的物体显得偏大，所以测试时用稍大的图更匹配。	同一个 224 训练的模型，在 288 上测反而高 2.1 分。
interpolation 插值	根据已知的点估算中间的值。	把 14 × 14 的位置嵌入“拉伸”成 18 × 18，像放大一张小图片。
checkpoint 检查点	保存下来的模型参数文件。	全量模型每个任务要存 374 MB；LoRA 只要 35.5 MB，其中 30.8 MB 是 head。
run 一次运行	一次完整的训练实验，有自己的名字和日志。	1019_1319_1_full = 10 月 19 日 13:19 这批的第 1 个 run，方法是 full。
post-deadline analysis 截止后的分析	比赛截止之后做的实验，只用于理解问题，不改变已经提交的结果。	分辨率实验和 in21k-only backbone 检查，都只在 holdout 上，没有 test 分数。

第 4 章　图表、公式与架构图

这一章专门“读图”。slides 上的每一张图、每一个表、每一张结构图，我们都拆开来讲：图里每一部分是什么，数据或信息怎么流动，每一部分为什么存在，各部分之间是什么关系；涉及数字的地方，带你亲手算一遍。最后一节把所有公式集中起来，每个符号都解释清楚，并配一个小计算。

方法本身是什么、为什么、优缺点，在第 2 章讲；这一章只负责“看懂图和公式”。

读任何一张图，先问自己四个问题

1. 横轴、纵轴（或者表格的每一列）分别是什么？单位是什么？
2. 每个点、每条线、每种颜色代表什么？
3. 这张图在比较谁和谁？比较时其他条件是不是都一样？
4. 一句话结论是什么？它和前后几页有什么关系？

答辩时老师指着一张图提问，先在心里把这四个问题过一遍，再开口。

4.1　第 2 页：数据示例图和四个数字



第 2 页左边的 16 张训练图片：每一行是同一个类的 4 张照片

这张图怎么读

16 张图排成 4 行 4 列，规则只有一条：**每一行是同一个类**。

- 第 1 行是同一种鸟：望远镜里看到的一只、天上的一大群、喂食器上的、芦苇上的。
- 第 2 行是同一种蛇：在石头上、路面上、碎石上、落叶上。
- 第 3 行是同一种蛾：在墙上、叶子上、特写、布上。
- 第 4 行是同一种植物：花、果实、整株灌木、一根枝条。

这张图想说明什么

它用 16 张图说明了任务难在哪里：

1. **同一类的照片差别很大。**一只鸟和一群鸟，一朵花和一整棵灌木，在像素上几乎没有共同点，模型却要把它们分到同一类。这叫“类内差异大”。
2. **不同类之间差别很小。**10,000 个类里，很多类之间只差一点点，比如两种长得很像的鸟。这叫“细粒度”（fine-grained）。
3. **每类只有 45 张。**照片这么少、差别又这么大，模型很容易把每一张背下来，而不是学会这个类的规律。

生活类比

就像只看 45 张照片就要认出一个人：有正脸、有侧影、有远处的背影，还有和一群人的合影。真正认识这个人，靠的是抓住共同特征；如果只是背照片，换一张新照片就认不出来了。

右边的四个数字

每个数字后面都跟着一句“So:”，意思是“所以我们要怎么做”。这一页把“分析问题”直接变成了“设计选择”：

数字	它说的事实	所以我们这样设计（So）
10,000	细粒度类别，而且类名被隐藏	预训练特征不是为这些区别准备的，特征本身必须改变，只换 head 不够
45	每类训练照片，共 450,000 张	很容易死记；只看 20,000 张留出照片的成绩，不看训练 loss
1.2 h	每次训练的预算：A100-80GB 上 1.2 小时（2 h × 48 GB，±10%）	每个想法都放进同一个预算里比较，所有 GPU 时间都计入
ViT-B/16	先在 ImageNet-21k、再在 ImageNet-1k 上训练过，85.8M 参数	用课程推荐的 backbone，每个实验都不换

计算：预算 1.2 小时是怎么来的

课程的规定是“2 小时 × 48 GB 显存”。把它看成一种资源总量：

$$2\ \text{h} \times 48\ \text{GB} = 96\ \text{GB} \cdot \text{h}.$$

我们的 A100 有 80 GB 显存，同样多的资源能用的时间是

$$96\ \text{GB} \cdot \text{h} \div 80\ \text{GB} = 1.2\ \text{h}.$$

规定允许 ±10%，所以最多是 $1.2 \times 1.1 = 1.32$ 小时。两个最终模型分别用了 1.03 和 0.54 GPU 小时，都在 1.2 小时以内。

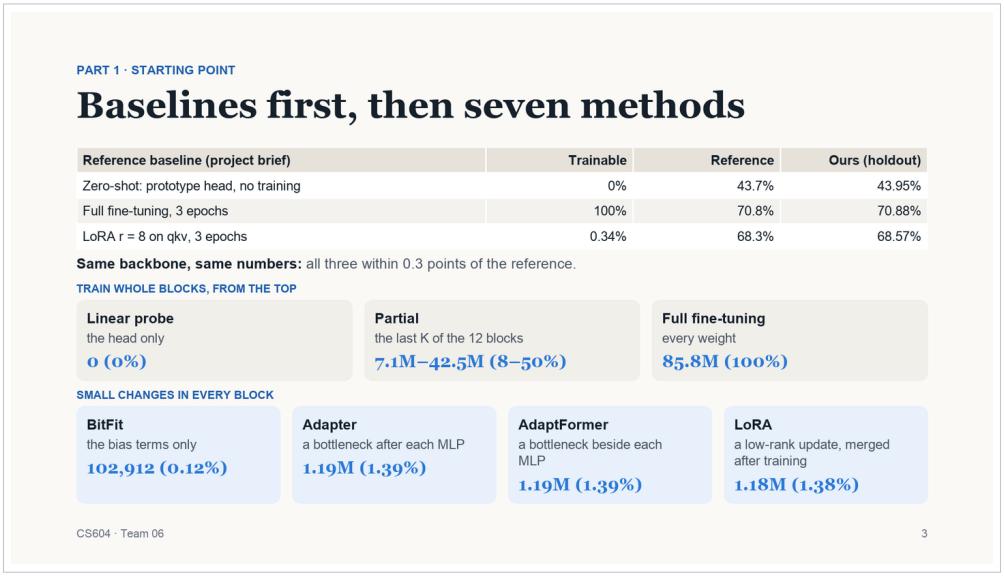
计算：数据量和 head 的大小

- 训练集：10,000 类 × 45 张 = 450,000 张。
- 留出集：10,000 × 2 = 20,000 张；对比实验用剩下的 10,000 × 43 = 430,000 张训练。
- 验证集：10,000 × 5 = 50,000 张；测试集：10,000 × 10 = 100,000 张。两者的标签都看不到。
- 新 head：$768 \times 10{,}000 + 10{,}000 = 7{,}690{,}000$ 个参数，是 backbone 的 $7{,}690{,}000 \div 85{,}798{,}656 \approx 9.0\%$。每个类在 head 里有 769 个数（768 个权重 + 1 个偏置），却只有 45 张照片来学。

容易搞错

“1.2 小时”算的是 GPU 时间，包括三部分：开头用冻结模型提取特征（约 6 分钟）、训练本身、每个 epoch 结束后在 holdout 上评估。解压数据、写最终的预测文件不算在内。

4.2　第 3 页：基线表和两类方法卡片



第 3 页缩略图

上面的表：先复现参考结果

表有四列：参考实验的名字（来自课程 PDF）；Trainable（训练了 backbone 的多少）；Reference（PDF 给的分数）；Ours (holdout)（我们用自己的代码，在 holdout 上重跑的分数）。三行的对比：

- 零训练（原型 head，一步都不训练）：43.7% 对 43.95%；
- 全量微调 3 个 epoch：70.8% 对 70.88%；
- 只在 qkv 上加 LoRA（$r = 8$）3 个 epoch：68.3% 对 68.57%。

三组都相差不到 0.3 分。

为什么要先做这一步

在尝试任何新想法之前，先确认“我们的流水线是对的”：数据读对了，评估算对了，训练设置和参考一致。起点可信，后面每一个改进的分数才可信。

生活类比

称东西之前先校准秤：放一个已知是 1 公斤的砝码，秤显示 1.00 公斤，之后称别的东西才可信。三个参考实验就是我们的“砝码”。

同样是“全量微调 3 个 epoch”，为什么这里是 70.88%，第 5 页是 73.38%?

复现参考结果时（9 月 28 日），我们用的还是当时的常规做法：先训练一个线性探针 head，再微调（即 LP-FT）。第 5 页的七方法比较，换成了后来发现更好的原型 head。两次唯一的区别就是 head 的起点，相差约 2.5 分。这正是第 6 到 8 页要讲的故事。参考 LoRA 那一行同理（68.57% 也是线性探针 head）。

下面的卡片：两族方法

七种方法分成两族：

- **第一族（灰色卡片）：从上往下，整块训练。**线性探针（只训练 head，0%）、部分微调（最后 K

个 block，8% 到 50%）、全量微调（全部，100%）。

- **第二族（蓝色卡片）：每个 block 都做一点小改动。**BitFit（只训练偏置，0.12%）、Adapter（MLP 后面串一个瓶颈，1.39%）、AdaptFormer（MLP 旁边并一个瓶颈，1.39%）、LoRA（低秩修改量，训练后合并，1.38%）。

卡片上的数字是“训练的参数量（占 backbone 的百分比）”，都可以自己算出来。

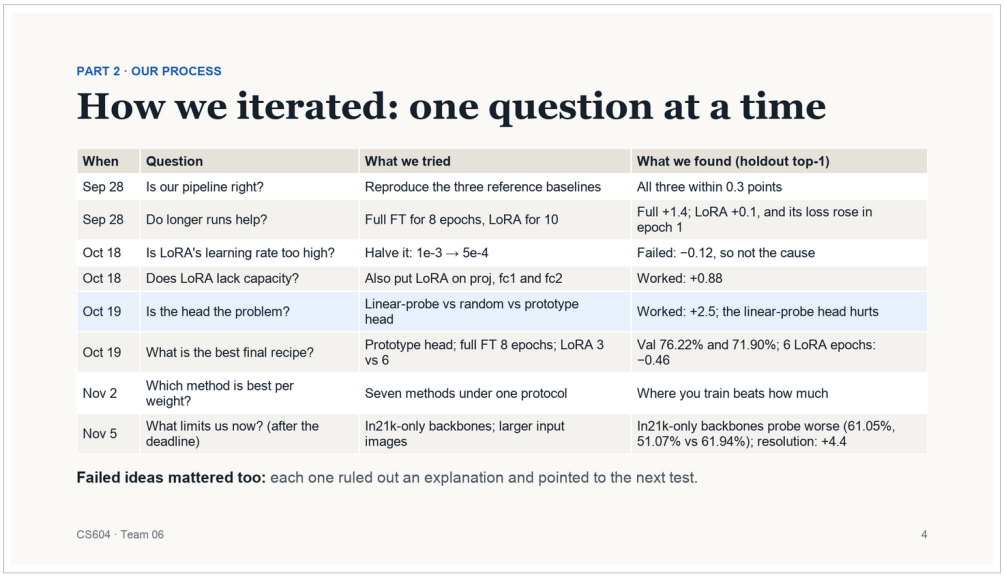
计算：参数占比（backbone 共 85,798,656 个参数）

- 部分微调 $K = 1$：一个 block 7,087,872 + 最后的 LayerNorm 1,536 = 7,089,408，占 8.26%；$K = 6$：$6 \times 7{,}087{,}872 + 1{,}536 = 42{,}528{,}768$，占 49.6%。
- BitFit：$102{,}912 \div 85{,}798{,}656 \approx 0.120\%$。
- Adapter、AdaptFormer：$1{,}189{,}632 \div 85{,}798{,}656 \approx 1.387\%$（卡片上写 1.39%）。
- LoRA（四个线性层）：$1{,}179{,}648 \div 85{,}798{,}656 \approx 1.375\%$（卡片上写 1.38%）。
- 参考 LoRA（只在 qkv 上）：$12 \times 24{,}576 = 294{,}912$，占 0.344%，所以表里写 0.34%。

重点

两族的区别是“改动放在哪里”：第一族把改动集中在网络顶部几层；第二族把一点点改动分散到全部 12 层。第 5 页会用数据证明：参数量差不多时，分散到每一层的效果好得多。

4.3　第 4 页：迭代时间表



PART 2 · OUR PROCESS

How we iterated: one question at a time

When	Question	What we tried	What we found (holdout top-1)
Sep 28	Is our pipeline right?	Reproduce the three reference baselines	All three within 0.3 points
Sep 28	Do longer runs help?	Full FT for 8 epochs, LoRA for 10	Full +1.4; LoRA +0.1, and its loss rose in epoch 1
Oct 18	Is LoRA's learning rate too high?	Halve it: 1e-3 → 5e-4	Failed: −0.12, so not the cause
Oct 18	Does LoRA lack capacity?	Also put LoRA on proj, fc1 and fc2	Worked: +0.88
Oct 19	Is the head the problem?	Linear-probe vs random vs prototype head	Worked: +2.5; the linear-probe head hurts
Oct 19	What is the best final recipe?	Prototype head; full FT 8 epochs; LoRA 3 vs 6	Val 76.22% and 71.90%; 6 LoRA epochs: −0.46
Nov 2	Which method is best per weight?	Seven methods under one protocol	Where you train beats how much
Nov 5	What limits us now? (after the deadline)	In21k-only backbones; larger input images	In21k-only backbones probe worse (61.05%, 51.07% vs 61.94%); resolution: +4.4

Failed ideas mattered too: each one ruled out an explanation and pointed to the next test.

CS604 · Team 06　4

第 4 页缩略图

四列怎么读

- **When**：日期，表示研究的先后顺序；
- **Question**：这一轮想回答的一个问题；
- **What we tried**：为了回答它做的实验，每次只改一个设置；
- **What we found**：结果（holdout top-1，标了 Val 的是 CodaBench 验证集），并写明 Worked（成功）或 Failed（失败）。

最重要的是行与行之间的因果

这张表要看的不是单独某一行，而是“上一轮的答案决定下一轮问什么”：

1. **9 月 28 日**，流水线对吗？复现三个参考结果 → 都在 0.3 分以内，可以继续。
2. **9 月 28 日**，训练久一点有用吗？全量 8 个 epoch、LoRA 10 个 epoch → 全量 +1.4，LoRA 只 +0.1，而且 LoRA 的训练 loss 在第 1 个 epoch 里反而上升。这是一个“异常信号”。
3. **10 月 18 日**，是 LoRA 的学习率太高吗？减半（1e-3 → 5e-4）→ −0.12。失败：不是学习率的问题。
4. **10 月 18 日**，是 LoRA 的容量不够吗？把 LoRA 也加到 proj、fc1、fc2 → +0.88。成功。
5. **10 月 19 日**，是 head 的问题吗？比较线性探针、随机、原型三种 head → +2.5，而且发现线性探针 head 有害。这是全项目最大的发现，所以这一行用蓝色高亮。
6. **10 月 19 日**，最终配方是什么？原型 head；全量 8 个 epoch；LoRA 比较 3 和 6 个 epoch → val 76.22% 和 71.90%；LoRA 训 6 个 epoch 反而 −0.46。
7. **11 月 2 日**，按参数量算，哪种方法最划算？七种方法用同一套协议 → “训练的位置比数量更重要”。
8. **11 月 5 日（截止后）**，现在还被什么限制？只在 ImageNet-21k 上训练过的 backbone，线性探针更差（61.05%、51.07%，我们的是 61.94%）；分辨率 +4.4。

生活类比

像医生找病因：病人发烧（LoRA 的 loss 上升）。先怀疑是感冒（学习率太高），吃了药不见效（−0.12），排除；再怀疑另一个原因（head），一查就确诊（+2.5）。没查出问题的检查同样有用：它让你不再往错误的方向浪费时间。

重点

老师特别看重“怎么一轮一轮做实验”和“失败的实验”。这张表把两者放在了同一页：8 轮，每轮一个问题，成功和失败都写出来。表下面那句话就是总结：失败的想法也有用，每一个都排除了一种解释，指向下一个测试。

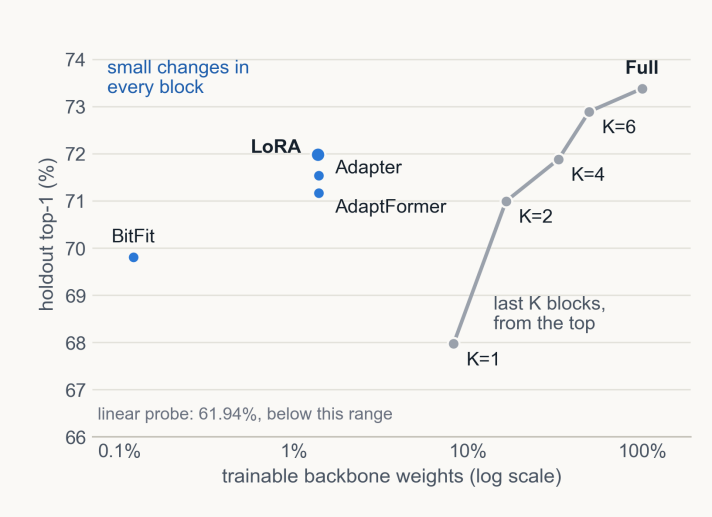
4.4　第 5 页：七种方法对比表和“准确率–参数量”图

左边的表

所有方法用同一套协议：原型 head、3 个 epoch、同一个 holdout、同一种 GPU，所以数字可以直接比较。淡蓝色的两行（全量微调和 LoRA），是我们最终采用的两种方法，幻灯片上也是这两行高亮。

方法	训练比例	Holdout	训练显存	延迟（batch 1）
线性探针	0%	61.94%	3.2 GB	6.96 ms
部分微调 $K = 4$	33%	71.88%	7.0 GB	7.41 ms
全量微调	100%	73.38%	18.9 GB	7.78 ms
BitFit	0.12%	69.81%	12.7 GB	7.33 ms
Adapter	1.39%	71.54%	12.8 GB	9.74 ms
AdaptFormer	1.39%	71.17%	12.8 GB	10.23 ms
LoRA	1.38%	71.98%	19.9 GB	7.61 ms

右边的图怎么读



第 5 页右图：横轴是训练的参数占比（对数刻度），纵轴是 holdout 准确率

- **横轴**：训练了 backbone 的多少参数，用对数刻度。
- **纵轴**：holdout top-1 准确率，从 66% 到 74%。
- **灰色折线**：“从上往下训练最后 K 个 block”，$K = 1, 2, 4, 6$，一直到全量（Full）。
- **蓝色的点**：“每个 block 都做小改动”的四种方法。
- **左下角的小字**：线性探针 61.94%，比纵轴最低处还低，所以没有画进去。

对数刻度是什么意思

普通刻度上，相邻两格差一个固定的数（比如每格 +10）。对数刻度上，相邻两格差一个固定的倍数：0.1%、1%、10%、100% 之间的距离一样，每一格都是 ×10。

为什么要用它？BitFit 训练 0.12%，全量训练 100%，相差约 833 倍。在普通刻度上，0.12% 和 1.38% 都会挤在最左边一小段，根本看不清；用对数刻度，它们就分开了。

灰线：每多训练一个 block，收获越来越少

从线性探针（61.94%）出发，沿着灰线往右：

- 训练最后 1 个 block（8.3%）：67.97%，比线性探针 +6.0；
- 2 个 block（16.5%）：70.99%，再 +3.0；
- 4 个 block（33%）：71.88%，再 +0.9；
- 6 个 block（49.6%）：72.89%，再 +1.0；
- 全部 12 个 block（100%）：73.38%，再 +0.5。

越往后，每多花一份参数换来的准确率越少，这叫“收益递减”。训练一半的 block，已经只比全量低 0.5 分。

蓝点：在灰线左上方很远的地方

蓝点在灰线的左上方，意思是“用少得多的参数，达到差不多甚至更高的准确率”。和灰线上准确率相近的点比一比：

计算：省了多少倍参数

- LoRA（1.375%，71.98%）和训练最后 4 个 block（33.0%，71.88%）准确率相当：$33.0 \div 1.375 = 24$ 倍的参数差距。
- Adapter（1.387%，71.54%）超过了最后 2 个 block（16.5%，70.99%）：约 12 倍。
- BitFit（0.120%，69.81%）超过了最后 1 个 block（8.26%，67.97%）：约 69 倍。

这就是报告里说的“用少 12 到 69 倍的参数”。

结论：**训练的“位置”比“数量”更重要。**在每一层都做一点改动，比只在顶部几层做大改动有效得多。一个合理的解释是：冻结的底层特征没办法为新类别调整，而分散的小改动能到达每一层。

显存那一列：显存跟着“梯度往回传多远”走

直觉上，训练的参数越少，显存应该越少。表里却不是这样：

- LoRA 只训练 1.38%，却要 19.9 GB，比全量微调（100%，18.9 GB）还多；
- 训练最后 4 个 block（33%）只要 7.0 GB；
- BitFit（0.12%）和两种 Adapter（1.39%）要 12.7 到 12.8 GB，比训练一半网络（$K = 6$，10.0 GB）还多。

原因在反向传播：计算梯度时，要用到前向传播时存下来的中间结果（activations，激活值）。梯度传到哪一层，那一层的中间结果就必须存着。

- 部分微调：梯度只需要传过顶部 K 个 block，下面的 block 不用存，所以省显存。
- BitFit、Adapter、LoRA：每一层都有要训练的参数，梯度必须一路传到第 1 个 block，所有层的中间结果都要存。
- 不过 BitFit 和两种 Adapter 的大权重矩阵是冻结的，不需要为它们算权重梯度，能少存一些，所以比全量少约三分之一，训练也更快（1,407 到 1,509 张/秒，全量是 1,139 张/秒）。
- LoRA 是例外：为了算 A 矩阵的梯度，它必须像全量微调一样存下每个线性层的输入，还要额外存它自己 96 个小矩阵乘法（12 个 block × 4 层 × 2 个矩阵）的中间结果，所以比全量还多一点，训练也最慢（888 张/秒）。

计算：每多训练一个 block 多用多少显存

部分微调在 $K = 2, 4, 6$ 时分别用 4.1、7.0、10.0 GB。每多一个 block 约多 $(10.0-4.1)\div(6-2) \approx 1.475$ GB（约 1.5 GB）。按这个速度推到 12 个 block：$4.1 + 10 \times 1.475 \approx 18.8$ GB，和全量微调实测的 18.9 GB 几乎一样。

所以全量微调的显存主要花在“存中间结果”上。它比 LoRA 多出来的优化器状态（85.8M 个参数各自的梯度和 AdamW 的两组统计量，每个数 4 字节）只有

$$85{,}798{,}656 \times 4 \text{ 字节} \times 3 \approx 1.03 \text{ GB}.$$

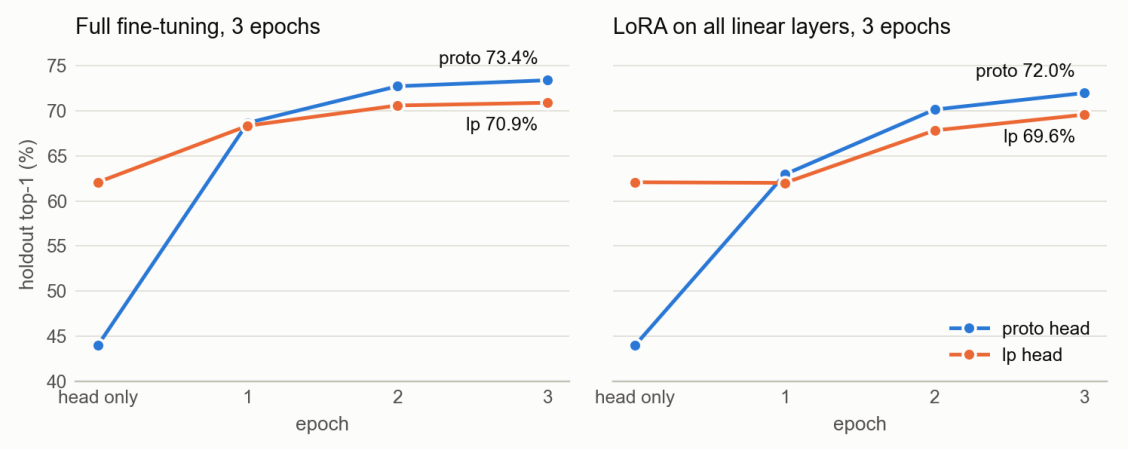
延迟那一列：只有两种 Adapter 真的变慢

- 线性探针、部分微调、全量微调、BitFit、合并后的 LoRA，推理时的网络结构和原来的 ViT-B/16 完全一样，延迟都在 7 ms 左右（6.96 到 7.78 ms）。它们之间零点几毫秒的差别是测量误差：结构完全相同的 $K = 1$ 到 6，也有约 0.5 ms 的差别。
- Adapter（9.74 ms）和 AdaptFormer（10.23 ms）多出 2 到 3 ms：每个 block 里多了两层计算，而且中间有 ReLU，没法合并进原来的权重。

这一页的三个结论

1. 位置比数量重要：LoRA 用 1.38% 的参数，追平了训练 33% 参数的部分微调。
2. 显存跟着“梯度传多远”走，不跟着“训练多少参数”走。
3. 全量微调最准；高效方法里 LoRA 最好，而且合并后不增加延迟。

4.5　第 6 页：head 初始化曲线



第 6 页的图：左边全量微调，右边 LoRA（四个线性层）；橙色是线性探针 head，蓝色是原型 head

横轴和纵轴

- **横轴**：最左边的“head only”，是只有 head、还没开始训练时的准确率；然后是第 1、2、3 个 epoch 结束时的准确率。
- **纵轴**：holdout top-1 准确率。
- 左右两个小图只差在方法（全量微调或 LoRA）；每个小图里的两条线，只差在 head 的起点。

从左往右看发生了什么

1. **起点（head only）**：线性探针 head 62.1%，原型 head 44.0%，线性探针领先 62.1 − 44.0 = 18.1 分。因为线性探针 head 在冻结特征上专门训练过，原型 head 一步都没训练。
2. **第 1 个 epoch 之后**：左图两条线几乎交在一起（约 68%）。右图更有意思：橙线基本没动（还在约 62%），蓝线已经升到约 63%。橙线停住不动，正好对应“这个设置下训练 loss 在第 1 个 epoch 里反而上升”。
3. **第 3 个 epoch 结束**：蓝线反超，而且拉开差距。全量微调 73.4% 对 70.9%，差 2.5 分；LoRA 72.0% 对 69.6%，差 2.4 分。

重点

这张图的一句话：**好的开始不等于好的结束**。线性探针 head 开局领先 18 分，训练结束时却落后约 2.5 分。所以我们判断任何设置，只看训练结束后、在留出数据上的结果，不看起点，也不看训练 loss。

两个小细节

- 曲线上的点，是训练过程中每个 epoch 结束时记录的；表格里的数，是训练结束后做的最终评估。所以个别数字会差零点几，例如 LoRA + 线性探针 head 在曲线上是 69.6%，在第 9 页的图和报告的表里是 69.45%。差别在噪声范围内，不影响结论。
- 这张图是“3 个 epoch、holdout”的实验，第 8 页是“8 个 epoch、全部数据、val”的实验，不是同一组。两组实验得出同一个结论，说服力更强。

4.6　第 7 页：原型 head 的流程和三个数字

上面的四步流程

1. **Extract features（提取特征）**：用冻结的 ViT，把 450,000 张训练图各过一遍，每张得到一个 768 维特征。约 6 分钟，计入 GPU 时间。
2. **Prototype head（原型 head）**：每一类的权重 = 这一类特征的归一化平均方向 × 40。这是关键一步，所以用蓝色高亮。这个做法叫 weight imprinting（Qi et al., CVPR 2018）。
3. **Fine-tune（微调）**：从这个 head 出发，做全量微调或者 LoRA。
4. **Predict（预测）**：全量模型把一张图和它的镜像图的 logits 相加（flip TTA）。

箭头表示数据从左往右流：特征 → 原型 → 初始化好的 head → 训练 → 预测。

第 2 步怎么算，一步一步来

原型 head 的公式

设第 i 张训练图的特征是 z_i（768 维），它的类别是 y_i。

1. 每个特征先归一化成长度 1，只保留方向：$\hat{z}_i = z_i / \|z_i\|$。
2. 同一类的归一化特征加起来，再归一化，得到这一类的原型（平均方向）：

$$\mu_c = \frac{\sum_{i:\, y_i = c} \hat{z}_i}{\left\| \sum_{i:\, y_i = c} \hat{z}_i \right\|}.$$

3. 把原型写进 head：第 c 行的权重 $W_c = \dfrac{\tau}{\overline{\|z\|}}\mu_c$，偏置 $b_c = 0$。

4. 一张新图 z 在第 c 类上的分数：

$$\text{logit}_c = W_c \cdot z = \tau \cdot \frac{\|z\|}{\overline{\|z\|}} \cdot \cos(z, \mu_c) \approx \tau \cos(z, \mu_c).$$

符号：$\|z\|$ 是向量的长度；μ_c 是第 c 类的原型；$\tau = 40$ 是温度；$\overline{\|z\|}$ 是所有特征长度的平均值；cos 是余弦相似度。

它想表达：一张图属于哪一类，就看它的方向和哪一类的“平均方向”最接近。head 收到的是没有归一化的特征 z，所以先除以平均长度 $\overline{\|z\|}$，让分数约等于“40 × 余弦相似度”。

简化示意：3 个类、2 维特征（不考虑 label smoothing）

真实情况是 768 维、10,000 类；这里缩小成 2 维、3 类，方便手算。每类有两张训练图：

- 鸟：(3, 4) 和 (4, 3)。长度都是 5，归一化得 (0.6, 0.8) 和 (0.8, 0.6)；相加得 (1.4, 1.4)，再归一化，原型 $\mu_{\text{鸟}} = (0.707, 0.707)$。
- 蛇：(−3, 4) 和 (0, 5) → (−0.6, 0.8) 和 (0, 1) → 相加 (−0.6, 1.8) → 原型 (−0.316, 0.949)。
- 蛾：(5, 0) 和 (4, −3) → (1, 0) 和 (0.8, −0.6) → 相加 (1.8, −0.6) → 原型 (0.949, −0.316)。

新来一张图，特征是 (6, 8)，归一化后是 (0.6, 0.8)。它和三个原型的余弦相似度：

- 鸟：$0.6 \times 0.707 + 0.8 \times 0.707 = 0.990$；
- 蛇：$0.6 \times (-0.316) + 0.8 \times 0.949 = 0.569$；
- 蛾：$0.6 \times 0.949 + 0.8 \times (-0.316) = 0.316$。

乘以温度 40：logits = (39.6, 22.8, 12.6)。最大的是鸟，所以预测“鸟”。整个过程一步训练都没有。

温度 40 是干什么的

余弦相似度只在 −1 到 1 之间。直接拿它当分数，softmax 之后的概率会非常“平”，模型显得很没把握。

同一个例子，换不同温度（简化示意，不考虑 label smoothing）

- 温度 1：logits (0.990, 0.569, 0.316) → 概率约 (0.46, 0.30, 0.24)。答对了，但很没把握。
- 温度 5：logits (4.95, 2.85, 1.58) → 概率约 (0.86, 0.11, 0.03)。
- 温度 40：logits (39.6, 22.8, 12.6) → 鸟的概率几乎是 1。

真实情况有 10,000 个类，很多类的原型方向很接近，正确类的余弦只比别的类高一点点。温度太小，概率分得太平，训练信号太弱；温度太大，模型一旦答错就会极度自信，loss 变得很大。所以我们在 5、10、15、20、30、40、60、80 这 8 个候选里，用 10,000 个训练特征上的交叉熵挑出最合适的一个，结果是 40。注意：挑温度只是从 8 个数里选一个，不是对 head 做梯度训练。

下面的三个数字卡片

- **0**：建 head 用了 0 步梯度下降。它从来没有“拟合”过任何一张训练图片，所以没有东西可以死记。
- **44%**：还没开始训练，原型 head 在 holdout 上就有 44% 的准确率。10,000 类里随便猜只有 0.01%，它高了约 4,400 倍。说明预训练特征已经很好，“比一比和每类平均方向的相似度”就能分对不少。
- **4.39**：训练刚开始时（LoRA 前 100 步平均）的训练 loss。对比：线性探针 head 是 2.29，随机 head

是 9.28。

怎么理解 2.29、4.39、9.28

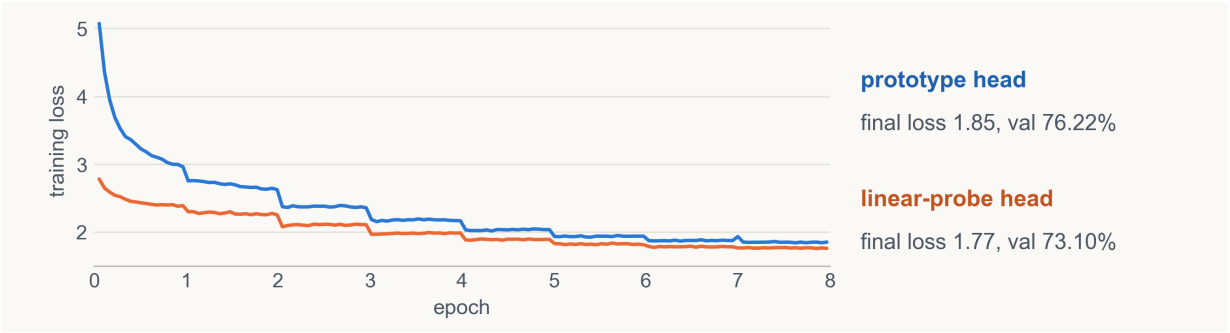
这三个数是训练 loss，里面包含 label smoothing 0.1 和数据增强的影响，只适合互相比较，不要换算成“正确类的概率”。

- **9.28（随机 head）**：和 $\ln 10{,}000 = 4 \ln 10 \approx 4 \times 2.303 = 9.21$ 几乎一样。$\ln 10{,}000$ 正是“在 10,000 类里平均瞎猜”的 loss（加不加 label smoothing 都差不多是这个数）。所以随机 head 一开始就是在瞎猜，给 backbone 的梯度又大又乱。
- **4.39（原型 head）**：大致知道每一类长什么样，但远远谈不上自信。loss 不小，梯度信号就够强，能推动 backbone 去适应新类别。
- **2.29（线性探针 head）**：已经在这些训练图片上拟合过，loss 本来就低，梯度信号弱，backbone 得不到多少“需要改变”的推动。

容易搞错

起点的训练 loss 高，在这里是好事：说明还有很多东西可学。但 loss 太高（随机 head 的 9.28）又意味着一开始的梯度很乱。原型 head 正好在两者之间：它的起点 loss 够高，又不是瞎猜，所以学得快。第 1 个 epoch 后，它比随机 head 高 5 分（63.3% 对 58.4%）。

4.7　第 8 页：训练 loss 曲线、三步解释和对照实验



第 8 页的图：全量微调、全部数据、8 个 epoch，两次训练只有 head 的初始化不同

这张图怎么读

- **横轴**：epoch，从 0 到 8。**纵轴**：训练 loss。
- **蓝线**：原型 head，从 5 以上开始，最后是 1.85，val 76.22%。
- **橙线**：线性探针 head，从约 2.8 开始，最后是 1.77，val 73.10%。
- 两次训练只有 head 的初始化不同，其他全部一样：全量微调、每类全部 45 张、8 个 epoch。

第一个反常：loss 更低，准确率反而更低

橙线从头到尾都在蓝线下面：训练 loss 更低（1.77 < 1.85），val 却低了 3.12 分（73.10% 对 76.22%）。训练 loss 低，只说明模型对训练图片答得好；在没见过的图片上答得不好，说明它记住的是训练图片本身，而不是能用到新图片上的规律。这就是死记硬背（memorization）。

第二条线索：“台阶”

仔细看两条线：每到一个 epoch 的边界（横轴 1、2、3……），loss 就突然往下掉一截；在一个 epoch 内部，它几乎是平的。为什么？

- 在一个 epoch 内部，每张图都是这一轮第一次出现；
- 新的 epoch 一开始，每张图都是“又见面了”。

如果模型只学到了一般规律，loss 应该平滑地下降；在边界上突然掉一截，说明它认出了“这张图我见过”，也就是在记具体的图片。这是死记硬背的典型信号。

生活类比

学生提前拿到了练习题的答案：练习分越来越高（训练 loss 越来越低），换成新的考题（val）却考不好。线性探针 head 就像这份“答案”：它正是在这些训练图片上训练出来的。

下面的三步解释

1. **在同样的照片上拟合**：线性探针 head 是在训练图片的冻结特征上训练出来的，它已经“认识”这些图片。
2. **信号先弱、后变错**：起点 loss 低，backbone 收到的梯度信号弱；等特征开始变化，原来拟合好的 head 又对不上新的特征了。证据：LoRA + 线性探针 head 的训练 loss 在第 1 个 epoch 里从 2.22 升到 2.69（约 2.7），那时还没有任何一张图被看第二遍。
3. **死记**：最后模型拟合的是每类 45 张图，而不是能泛化的特征：训练 loss 更低（1.77 对 1.85），val 更低（73.10% 对 76.22%）。

最下面的对照实验

如果原型 head 好，只是因为“原型”这个想法本身很神奇，那么换成随机 head 应该明显更差。实际结果：在 LoRA 上，随机 head 71.95%，原型 head 72.00%，打平。所以真正的问题是“在训练图片上拟合 head”这件事，而不是原型有什么魔法。

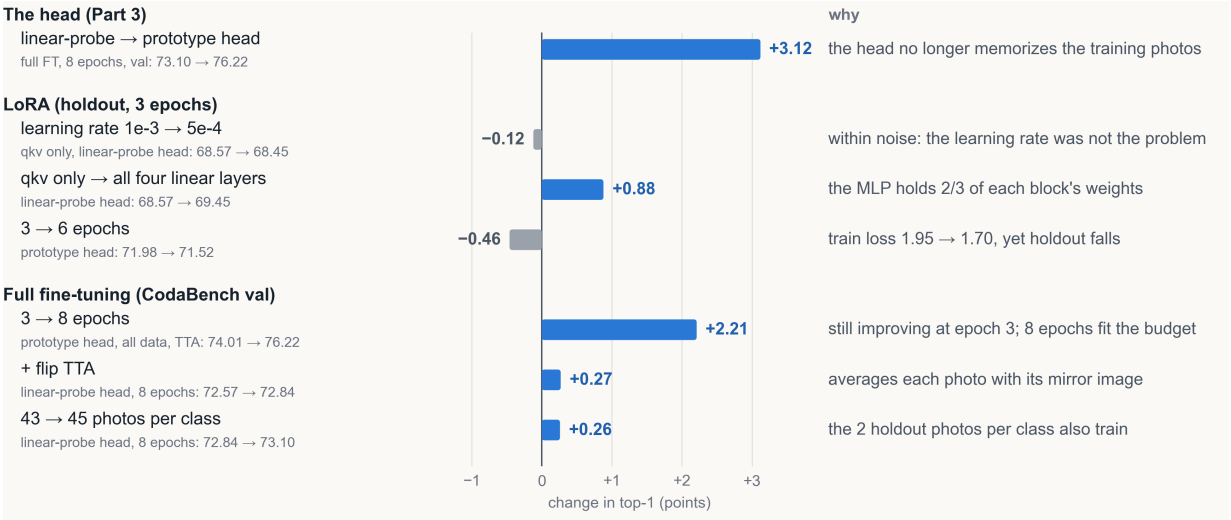
重点

对照实验（control）的意思是：专门设计一个实验，用来排除另一种解释。这里用随机 head 排除了“原型本身很神奇”这个解释，结论就更可信。

容易搞错

不要把这一页说成“我们证明了”。报告里的说法是：这个解释和 loss 曲线、对照实验都一致，但它不是严格的因果证明。答辩时说“our explanation is consistent with the loss curves and the control”更稳妥。

4.8　第 9 页：消融实验条形图



第 9 页的图：每一行只改一个设置；中间是准确率的变化，右边是原因

三栏怎么读

- **左栏**：改了什么；下面一行小字写着“在什么设置下、从多少变到多少”。
- **中栏**：条形的长度 = top-1 变化了多少分；蓝色是提升，灰色是下降；中间的竖线是 0。
- **右栏（why）**：为什么会这样。

三组用的数据不同：第一组和第三组是 CodaBench val，第二组是 holdout。所以只在组内比较，不要跨组比较分数的高低。

逐行解释

改了什么（设置）	前 → 后	变化	原因
head：线性探针 → 原型（全量、8 epoch、val）	73.10 → 76.22	+3.12	head 不再死记训练图片。最大的一个改动
LoRA 学习率 1e-3 → 5e-4（只在 qkv、线性探针 head）	68.57 → 68.45	−0.12	在噪声以内：学习率不是问题
LoRA 从 qkv 扩到四个线性层（线性探针 head）	68.57 → 69.45	+0.88	MLP 占每个 block 2/3 的参数，只改注意力漏掉了大部分
LoRA 3 → 6 个 epoch（原型 head）	71.98 → 71.52	−0.46	训练 loss 从 1.95 降到 1.70，holdout 却下降：过拟合，所以停在 3 个 epoch
全量 3 → 8 个 epoch（原型 head、全部数据、TTA）	74.01 → 76.22	+2.21	第 3 个 epoch 时还在进步；8 个 epoch 正好放进预算（1.03 GPU 小时）
加 flip TTA（线性探针 head、8 epoch）	72.57 → 72.84	+0.27	把每张图和它的镜像图平均
每类 43 → 45 张（线性探针 head、8 epoch）	72.84 → 73.10	+0.26	留出的 2 张也拿来训练

计算：为什么说“MLP 占 2/3”

一个 block 里，fc1 有 $768 \times 3{,}072 + 3{,}072 = 2{,}362{,}368$ 个参数，fc2 有 $3{,}072 \times 768 + 768 = 2{,}360{,}064$ 个。两者相加是 4,722,432，除以整个 block 的 7,087,872，约 66.6%，也就是 2/3。只在 qkv 上加

LoRA，等于完全没碰这 2/3 的权重。

哪些差别是噪声

- −0.12：远小于 0.3，是噪声。所以说“学习率不是原因”。
- +0.27 和 +0.26：只比噪声大一点，方向符合预期，但不要过度解读。
- +3.12、+2.21、+0.88：明显超过噪声。−0.46 也超过了噪声，而且和训练 loss 下降同时出现，所以可以判断是过拟合。

重点

消融的规则：每次只改一个设置，其他全部不变。这样，变化只能归因于这一个设置。第 9 页的每一行都遵守这个规则，所以最终配方里的每一个设置都有数据支撑。

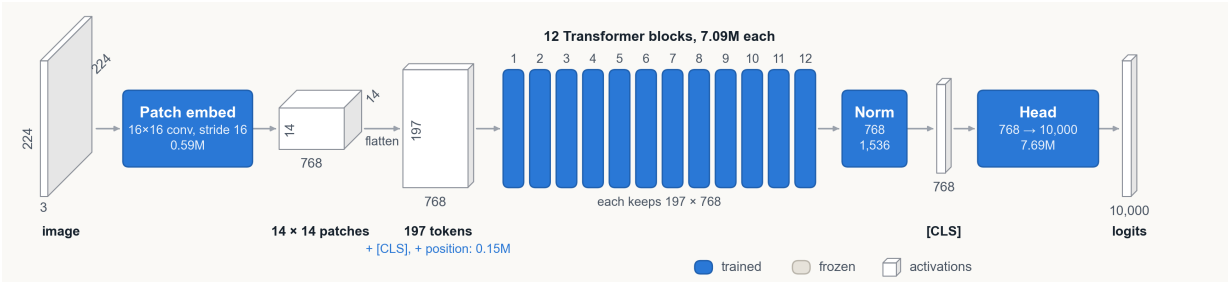
4.9　架构图逐个讲解（第 10、11、16 到 20 页）

这些结构图有两种视角：

- **整网图**：从输入图片一直画到输出的分数（全量微调、线性探针、部分微调）；
- **block 内部图**：把 12 个 block 中的一个放大来看（LoRA、BitFit、Adapter、AdaptFormer）。

图例统一：**蓝色** = 这个方法训练的部分（trained）；**米灰色** = 冻结（frozen）；**白色方框** = 只有计算、没有参数（no weights）；**白色立体方块** = 中间结果（activations，张量），旁边的数字是它的形状。

4.9.1　整网图：全量微调（第 10 页）



第 10 页：全量微调的整网图，所有部分都是蓝色（都训练）

数据从左往右流：

1. **image（224 × 224 × 3）**：输入图片，宽 224、高 224 像素，3 个颜色通道（红、绿、蓝）。
2. **Patch embed（16 × 16 conv, stride 16，0.59M）**：用一个 16 × 16、步长 16 的卷积把图切成不重叠的小块，每块变成 768 个数。
3. **14 × 14 × 768**：224 ÷ 16 = 14，所以得到 14 × 14 个图块，每块 768 维。
4. **flatten → 197 tokens（197 × 768）**：把 14 × 14 的网格排成一行（196 个），前面再加一个 [CLS] token，共 197 个；每个 token 再加上自己的位置嵌入。[CLS] 和位置嵌入一共 0.15M 参数。
5. **12 Transformer blocks（每个 7.09M）**：12 个一模一样的 block 依次处理。每个 block 进去是 197 × 768，出来还是 197 × 768（图上“each keeps 197 × 768”）：形状不变，内容越来越“懂”这张图。
6. **Norm（768，1,536）**：最后一个 LayerNorm。

7. **[CLS]（768）**：只取 [CLS] 这一个 token 的 768 维向量，它代表整张图。
8. **Head（768 → 10,000，7.69M）**：一个线性层，输出 10,000 个分数。
9. **logits（10,000）**：分数最高的类就是预测结果。

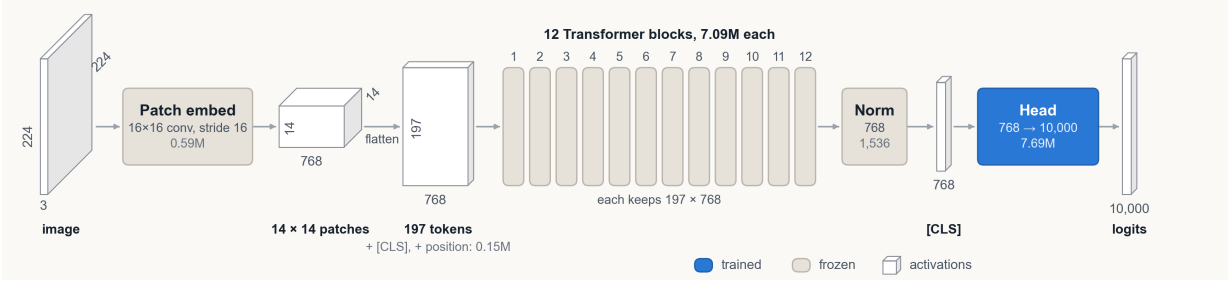
计算：85,798,656 个参数是怎么加出来的

- Patch embedding：$16 \times 16 \times 3 \times 768 + 768 = 590{,}592$（图上 0.59M）。

CLS + 位置嵌入：$768 + 197 \times 768 = 152{,}064$（图上 0.15M）。

- 一个 block：qkv $768 \times 2{,}304 + 2{,}304 = 1{,}771{,}776$；proj $768 \times 768 + 768 = 590{,}592$；fc1 $768 \times 3{,}072 + 3{,}072 = 2{,}362{,}368$；fc2 $3{,}072 \times 768 + 768 = 2{,}360{,}064$；两个 LayerNorm $2 \times 1{,}536 = 3{,}072$。合计 7,087,872（图上 7.09M）。
- 12 个 block：$12 \times 7{,}087{,}872 = 85{,}054{,}464$；最后的 LayerNorm：1,536。
- 总计：$590{,}592 + 152{,}064 + 85{,}054{,}464 + 1{,}536 = 85{,}798{,}656$。
- 新 head（不算在 backbone 里）：$768 \times 10{,}000 + 10{,}000 = 7{,}690{,}000$（图上 7.69M）。

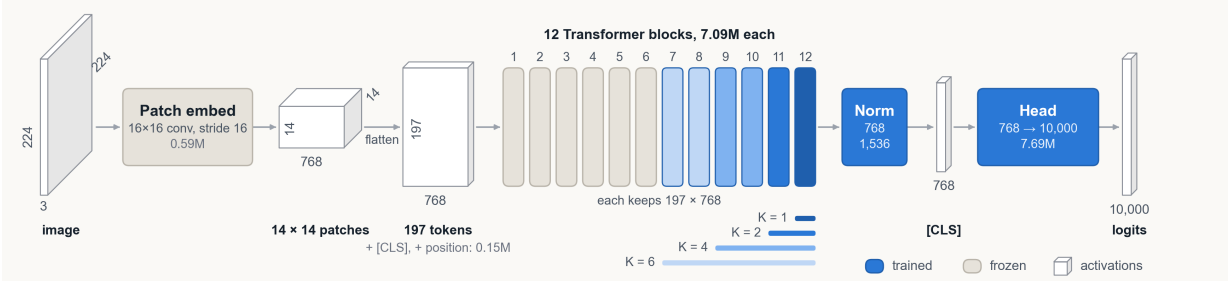
4.9.2　整网图：线性探针（第 16 页）



第 16 页（备用）：线性探针，只有 head 是蓝色

- 除了 head，全部是米灰色（冻结），backbone 从头到尾不变。
- 因为 backbone 不变，每张图的特征只需要算一次，存起来反复用（cached features）。之后在 768 维特征上训练 head 10 个 epoch，只要几秒钟。所以它最便宜：0.10 GPU 小时（大部分是开头提取特征的时间）、3.2 GB。
- 缺点：特征永远不会为新类别调整，准确率的上限就是“冻结特征有多好”：61.94%。

4.9.3　整网图：部分微调（第 17 页）



第 17 页（备用）：部分微调，只训练最后 K 个 block

- 第 7 到 12 个 block 用深浅不同的蓝色；下面 $K = 1, 2, 4, 6$ 的横条，标出每个 K 训练哪些 block：$K = 1$ 只训练第 12 个，$K = 6$ 训练第 7 到 12 个。

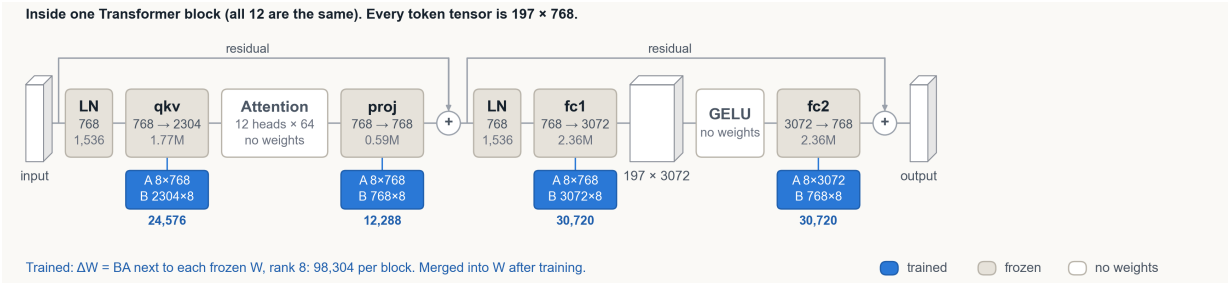
- 最后的 Norm 和 head 也训练（蓝色）；patch embedding 和下面的 block 冻结。
- 参数：$K \times 7{,}087{,}872 + 1{,}536$（最后的 LayerNorm）。
- 显存：梯度只需要传过顶部 K 个 block，所以显存随 K 增长，每个 block 约 1.5 GB（3.8 到 10.0 GB）。

4.9.4　block 内部的公共底图

LoRA、BitFit、Adapter、AdaptFormer 的图都是“放大一个 block”，底图是同一张。图上方写着：12 个 block 都一样，每个 token 张量都是 197 × 768。从左到右：

1. **input**（197 × 768）。
2. **LN**（768，1,536 个参数）：第一个 LayerNorm。
3. **qkv**（768 → 2,304，1.77M）：一次算出 Q、K、V，各 768 维。
4. **Attention**（12 heads × 64，no weights）：多头注意力，按 Q 和 K 的匹配程度把 V 加权汇总。这一步只有计算，没有参数，所以是白色方框。
5. **proj**（768 → 768，0.59M）：把 12 个头的结果整合回 768 维。
6. **第一个“+”** 残差连接，把 block 的输入加回来。上方标着 residual 的线就是这条“近路”。
7. **LN**（768，1,536）：第二个 LayerNorm。
8. **fc1**（768 → 3,072，2.36M）：放大到 3,072 维，得到 197 × 3,072 的中间结果。
9. **GELU**（no weights）：激活函数。
10. **fc2**（3,072 → 768，2.36M）：缩回 768 维。
11. **第二个“+”** 第二条残差连接。
12. **output**（197 × 768）：送进下一个 block。

4.9.5　LoRA（第 11 页）



第 11 页：LoRA，每个冻结的线性层下面挂一对蓝色的小矩阵 A 和 B

- 四个线性层（qkv、proj、fc1、fc2）本身都是米灰色（冻结），每个下面挂一对蓝色小矩阵：A（8 × 输入维度）和 B（输出维度 × 8）。
- **数据怎么走**：输入 x 同时走两条路。主路是冻结的 Wx；旁路先用 A 把 x 压成 8 个数，再用 B 放大回输出维度，乘以 $\alpha/r = 16/8 = 2$。两条路的结果相加：$y = Wx + 2 \cdot BAx$。
- B 一开始全是 0，所以训练开始时旁路输出为 0，模型和预训练模型完全一样，不会一上来就把好特征改乱。
- 图左下角写着：$\Delta W = BA$，秩 8，每个 block 98,304 个参数；训练后合并进 W。

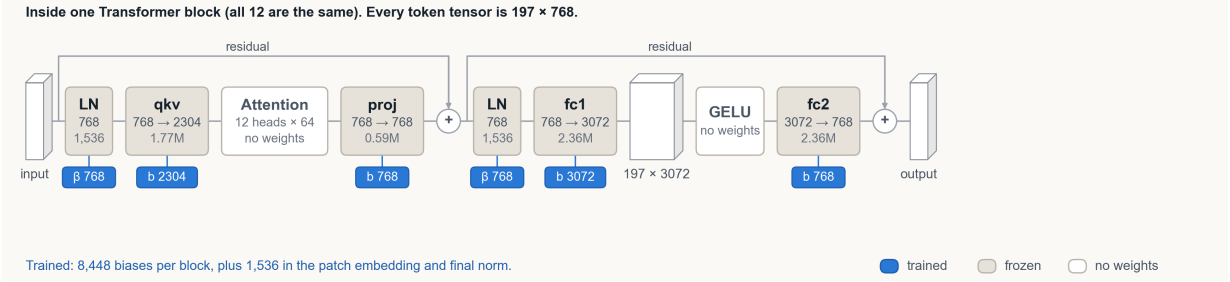
计算：LoRA 的参数

每对 A、B 的参数 = $r \times$ (输入维度 + 输出维度)，$r = 8$：

- qkv：$8 \times (768 + 2{,}304) = 24{,}576$；proj：$8 \times (768 + 768) = 12{,}288$；
- fc1：$8 \times (768 + 3{,}072) = 30{,}720$；fc2：$8 \times (3{,}072 + 768) = 30{,}720$。

每个 block 合计 98,304；12 个 block 共 1,179,648，占 backbone 的 1.375%。对比一下：proj 这一层自己有 $768 \times 768 = 589{,}824$ 个权重，LoRA 只用 12,288 个数来修改它，约 2%。

4.9.6　BitFit（第 18 页）



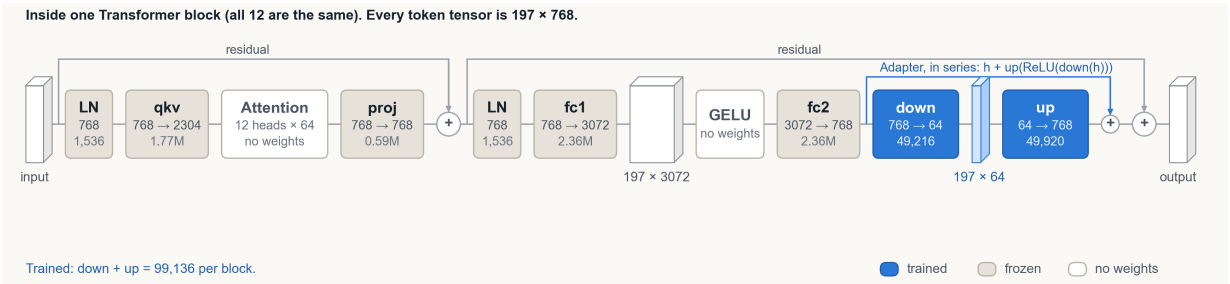
第 18 页（备用）：BitFit，只训练每层下面那些蓝色的偏置小标签

- 所有权重矩阵都冻结。每层下面挂一个蓝色小标签，就是这一层的偏置：第一个 LN 的平移参数 β（768）、qkv 的 b（2,304）、proj 的 b（768）、第二个 LN 的 β（768）、fc1 的 b（3,072）、fc2 的 b（768）。
- LayerNorm 有两组参数：缩放（γ）和平移（β）。BitFit 只训练平移这一组，因为它的作用就像偏置。

计算：每个 block 8,448 个偏置

$768 + 2{,}304 + 768 + 768 + 3{,}072 + 768 = 8{,}448$。12 个 block 共 101,376；再加上 patch embedding 的偏置 768 和最后 LayerNorm 的 β 768（共 1,536），总计 102,912，占 0.120%。

4.9.7　Adapter（第 19 页）



第 19 页（备用）：Adapter 串联在 fc2 后面

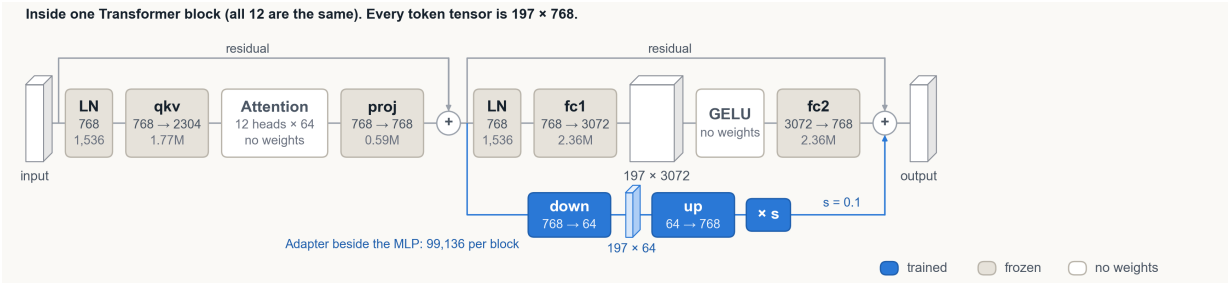
- 在 fc2 之后、第二条残差之前，串联两个蓝色模块：down（768 → 64，49,216 个参数）和 up（64 → 768，49,920 个参数），中间是 197 × 64 的“瓶颈”。
- 公式：$h + \mathrm{up}(\mathrm{ReLU}(\mathrm{down}(h)))$，$h$ 是 MLP 的输出。也就是说，Adapter 自己带一条小残差（图上第一个蓝色的“+”），之后才是主残差（第二个“+”）。
- up 一开始是 0，所以训练刚开始时 Adapter 什么都不改。

- 因为是串联，推理时每个 token 都要多算两层，中间还有 ReLU，所以没法合并：延迟从约 7.3 ms 变成 9.74 ms。

计算：每个 block 99,136 个参数

down：$768\times64+64=49{,}216$；up：$64\times768+768=49{,}920$；合计 99,136。12 个 block 共 1,189,632，占 1.387%。

4.9.8　AdaptFormer（第 20 页）



第 20 页（备用）：AdaptFormer 是一条并联在 MLP 旁边的支路

- 支路从第一个“+”之后（也就是 MLP 子层的输入）分出来，经过 down（$768\to64$）、197×64 的瓶颈、up（$64\to768$），再乘一个小系数 $s=0.1$，在第二个“+”处和 MLP 的输出一起加回主路。
- 公式：$x_{l+1}=x'+\mathrm{MLP}(\mathrm{LN}(x'))+s\cdot\mathrm{up}(\mathrm{ReLU}(\mathrm{down}(x')))$，其中 x' 是注意力子层之后的结果。
- 和 Adapter 的区别：Adapter 是“串联，改 MLP 的输出”；AdaptFormer 是“并联，在旁边加一点修正”，原来 MLP 的输出保持不变。$s=0.1$ 让修正一开始很温和。参数量一样，每个 block 99,136。
- 准确率 71.17%，Adapter 是 71.54%，差 0.37，接近噪声；两者的缩放系数也不同（1.0 对 0.1）。所以不能说“并联更好”或“串联更好”。
- 同样不能合并，延迟 10.23 ms。

4.9.9　为什么 LoRA 能合并，Adapter 不能

一个 2 × 2 的小例子

设冻结权重 $W=\begin{pmatrix}1&0\\0&1\end{pmatrix}$，LoRA 的 $A=(1\ \ 1)$（1×2，这里 $r=1$），$B=\begin{pmatrix}0.5\\0\end{pmatrix}$（$2\times1$），缩放 $\alpha/r=2$，输入 $x=(1,2)$。

- **不合并**：主路 $Wx=(1,2)$；旁路 $Ax=1+2=3$，$B\times3=(1.5,0)$，再乘 2 得 $(3,0)$；两路相加得 $(4,2)$。
- **合并**：$\Delta W=2BA=\begin{pmatrix}1&1\\0&0\end{pmatrix}$，$W'=W+\Delta W=\begin{pmatrix}2&1\\0&1\end{pmatrix}$，$W'x=(2+2,\ 0+2)=(4,2)$。

两种算法结果完全一样，但合并后只做一次矩阵乘法，结构和原模型一模一样，所以不增加延迟。Adapter 的 down 和 up 中间有 ReLU（负数变成 0）。这一步不是矩阵乘法，没法把“down、ReLU、up”三步乘成一个矩阵，所以合并不了。

4.9.10　七张架构图放在一起看

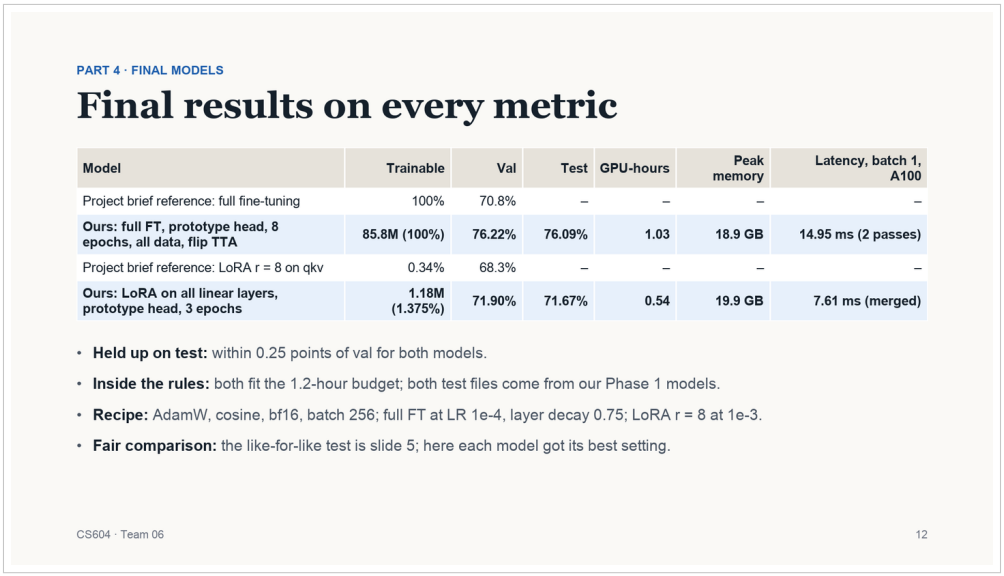
看完七张图，用一张表把它们的区别放在一起。“位置”一栏回答“蓝色在图上的哪里”，“推理”一栏回答“训练完以后，部署的模型和原来的 ViT-B/16 一样吗”。

方法	训练的部分（蓝色）	位置	推理（batch 1 延迟）	训练比例
线性探针	只有 head	网络最顶端	结构不变，6.96 ms	0%
部分微调	最后 K 个 block 和最后的 LN	顶部 K 层	结构不变，约 7.4 ms	8.26% 到 49.6%
全量微调	全部参数	每一层	结构不变，7.78 ms	100%
BitFit	所有偏置和 LN 的 β	每一层，但只有一点点	结构不变，7.33 ms	0.12%
Adapter	每个 block 里串联的 down 和 up	每一层，MLP 之后（串联）	多两层，不能合并，9.74 ms	1.39%
AdaptFormer	每个 block 旁边的支路	每一层，MLP 旁边（并联）	多一条支路，不能合并，10.23 ms	1.39%
LoRA	4 个线性层旁边的 A 和 B	每一层，线性层旁边	合并后结构不变，7.61 ms	1.38%

一句话记住

第一族（线性探针、部分微调、全量微调）从顶部往下，整块整块地训练；第二族（BitFit、Adapter、AdaptFormer、LoRA）在每一层都动一点点。第二族里，BitFit 和合并后的 LoRA 不增加推理延迟，两种 Adapter 会增加；而 LoRA 在第二族里准确率最高（71.98%）。

4.10　第 12 页：最终结果表



第 12 页缩略图

每一列是什么

- **Model**：模型。两行是课程参考（灰色，只有 val 分数和训练比例），两行是我们的最终模型（蓝色加粗）。
- **Trainable**：训练的 backbone 参数量和占比。全量 85.8M（100%）；LoRA 1.18M（1.375%）。两

者都还训练了 7.69M 的 head，按课程参考的惯例不计入占比。

- **Val / Test**：CodaBench 验证集和测试集上的 top-1。
- **GPU-hours**：一次训练用的 GPU 时间。
- **Peak memory**：训练时显存的最高值。
- **Latency, batch 1, A100**：在 A100 上一次处理一张图的时间。

计算：比参考高多少，test 和 val 差多少

- 全量：$76.22 - 70.8 = 5.42 \approx +5.4$；LoRA：$71.90 - 68.3 = 3.60 \approx +3.6$。
- test 减 val：全量 $76.09 - 76.22 = -0.13$；LoRA $71.67 - 71.90 = -0.23$。都在 0.25 以内，说明模型没有过度适应 val，在全新的 100,000 张图上表现一样稳定。
- 预算：1.03 和 0.54 GPU 小时，都不超过 1.2 小时。

两个括号的意思

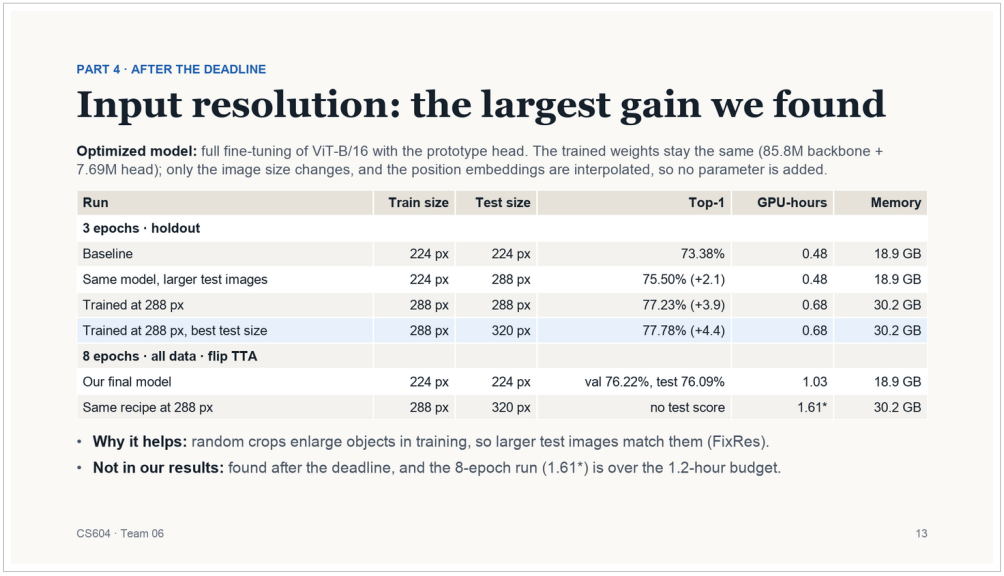
- **14.95 ms (2 passes)**：全量模型用了 flip TTA，每张图要算两遍（原图和镜像图），所以延迟约翻倍；同样的结构不做 TTA 是 7.35 ms。
- **7.61 ms (merged)**：LoRA 合并之后就是普通的 ViT-B/16，延迟和原模型一样。

公平比较在第 5 页，不在这一页

这一页是两种方法“各自最好的配置”：全量用了 8 个 epoch、每类全部 45 张、flip TTA；LoRA 用 3 个 epoch、每类 43 张、不做 TTA。所以 76.22 和 71.90 的差距不全是方法本身造成的。同一套协议下的公平比较在第 5 页：73.38% 对 71.98%。

幻灯片底部的“Inside the rules”也很重要：两个模型都在 1.2 小时预算内；两个 test 文件都来自 Phase 1 的同一个模型，符合课程规定。

4.11　第 13 页：分辨率表



PART 4 · AFTER THE DEADLINE

Input resolution: the largest gain we found

Optimized model: full fine-tuning of ViT-B/16 with the prototype head. The trained weights stay the same (85.8M backbone + 7.69M head); only the image size changes, and the position embeddings are interpolated, so no parameter is added.

Run	Train size	Test size	Top-1	GPU-hours	Memory
3 epochs · holdout					
Baseline	224 px	224 px	73.38%	0.48	18.9 GB
Same model, larger test images	224 px	288 px	75.50% (+2.1)	0.48	18.9 GB
Trained at 288 px	288 px	288 px	77.23% (+3.9)	0.68	30.2 GB
Trained at 288 px, best test size	288 px	320 px	77.78% (+4.4)	0.68	30.2 GB
8 epochs · all data · flip TTA					
Our final model	224 px	224 px	val 76.22%, test 76.09%	1.03	18.9 GB
Same recipe at 288 px	288 px	320 px	no test score	1.61*	30.2 GB

- **Why it helps:** random crops enlarge objects in training, so larger test images match them (FixRes).
- **Not in our results:** found after the deadline, and the 8-epoch run (1.61*) is over the 1.2-hour budget.

CS604 · Team 06　13

第 13 页缩略图

表怎么读

表上面的一句话先说清楚：被优化的模型是“全量微调 + 原型 head”。训练的参数完全一样（85.8M backbone + 7.69M head），只改输入图片的尺寸；位置嵌入用插值适配，不增加参数。

列：Run（哪个实验）、Train size（训练时的图片边长）、Test size（测试时的边长）、Top-1、GPU-hours、Memory。

- **上半部分（3 个 epoch，holdout）**：
  - Baseline：224 训练、224 测试，73.38%；
  - 同一个模型，只把测试图片放大到 288：75.50%（+2.1），一点额外训练都没有；
  - 用 288 训练、288 测试：77.23%（+3.9），0.68 GPU 小时，30.2 GB；
  - 288 训练、320 测试：77.78%（+4.4），蓝色高亮，是最好的测试尺寸。
- **下半部分（8 个 epoch、全部数据、flip TTA）**：
  - 我们的最终模型：224 训练、224 测试，val 76.22%，test 76.09%，1.03 GPU 小时；
  - 同一个配方改成 288 训练、320 测试：没有 test 分数，1.61* GPU 小时（星号表示超出 1.2 小时预算）。

计算：token 数和计算量

token 数 = (边长 ÷ 16)^2 + 1（多出来的 1 是 [CLS]）：

$$224：\ 14^2+1=197,\qquad 288：\ 18^2+1=325,\qquad 320：\ 20^2+1=401.$$

计算量怎么变？ViT 的计算大部分在线性层（qkv、proj、fc1、fc2），它们和 token 数成正比：325÷197 ≈ 1.65 倍。注意力里“每个 token 看所有 token”那一步和 token 数的平方成正比：$(325 \div 197)^2 \approx 2.7$ 倍，但它在 ViT-B 的总计算里只占很小一部分。合起来约 1.7 倍，这就是报告里说的“贵约 1.7 倍”。显存也跟着上升：18.9 GB → 30.2 GB。

位置嵌入怎么适配新尺寸

预训练时，位置嵌入是为 14 × 14 的网格学的。换成 288 像素，网格变成 18 × 18。timm 把原来 14 × 14 的“位置图”像放大照片一样插值成 18 × 18（320 像素时是 20 × 20）。这样每个新位置都有合理的位置向量，而且没有新增任何参数。

为什么放大测试图片就能涨分：FixRes

训练时我们用 RandomResizedCrop：随机裁下原图面积的 25% 到 100%，再缩放到 224 × 224。裁得越小，物体被放得越大。

一个极端例子

如果裁下面积的 25%，也就是边长的一半，再放大到 224 × 224，那么物体在训练图里就被放大了 2 倍。测试时我们不裁小块，而是把整张图缩小后取中间（先缩到 248，再裁出中间的 224）。于是测试图里的物体“看起来比训练时小”。把测试图放大到 288，物体变大，更接近模型训练时习惯的大小，准确率就上升。

这个现象叫“训练和测试的分辨率不一致”，FixRes 论文（Touvron et al., 2019）专门研究过。用 288 训练之后，最好的测试尺寸也跟着往上移到 320，是同一个道理。

完整的测试尺寸扫描（来自报告里的分辨率表）

幻灯片只放了几个关键点。报告里把两个 3 个 epoch 的模型，在 6 种测试尺寸上都测了一遍

（holdout top-1，%，加粗的是每一行的最好值）：

训练尺寸 \ 测试尺寸	224	256	288	320	352	384
224 像素	73.38	75.21	**75.50**	75.28	74.33	72.94
288 像素	73.10	75.74	77.23	**77.78**	77.63	77.41

怎么读这张表：

- **每一行都是先升后降。**测试图比训练图稍大一点最好（224 训练的模型最好在 288，288 训练的模型最好在 320）；再大下去，物体就比模型训练时见过的还大，准确率又往下掉（224 训练的模型在 384 只有 72.94%）。
- **测试图越大，用 288 训练的好处越明显。**同一个测试尺寸下，288 训练的模型比 224 训练的高：256 时高 0.5 分，288 时高 1.7 分，320 时高 2.5 分，384 时高 4.5 分。图片更大，细节更多，用大图训练的模型也学会了用这些细节。
- **一个小例外。**在 224 上测，288 训练的模型（73.10%）反而比 224 训练的（73.38%）略低：它已经习惯了更大的输入。这个差别在噪声范围内。

为什么最后一行不算我们的结果

- **发现得太晚**：分辨率实验是比赛截止后做的分析；
- **超出预算**：8 个 epoch 的配方在 288 像素下要 1.61 GPU 小时，超过 1.2 小时（就算加上 10% 也只到 1.32）；
- **没有 test 分数**：截止后不能再提交。

所以它只作为“教训和下一步”出现。上半部分 3 个 epoch 的 288 像素实验只用了 0.68 GPU 小时，在预算以内；但它也只在 holdout 上测过，没有 test 分数。

重点

这一页的教训：我们很早就研究“怎么微调”，却很晚才研究“输入长什么样”。分辨率带来的 +4.4 分，比任何单个微调设置都大。下一步：在预算内用“少几个 epoch”换“更大的图片”。

4.12　公式汇总

下面每个公式都按同样的顺序讲：先写公式，再解释每个符号和它想表达什么，最后配一个小计算。看不懂符号时，先读“它想表达什么”那一句。

4.12.1　softmax：把分数变成概率

softmax

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

- z_k：第 k 类的分数（logit）；K：类别数，我们是 10,000。
- e^{z_k}：取指数，保证是正数，并且放大差距。
- 分母：所有类的指数加起来，保证全部 p_k 加起来等于 1。

它想表达：分数越高，概率越大；分数只高一点点，也会被指数放大成明显更大的概率。

例子

三个类的分数 $z = (2, 1, 0)$：$e^2 = 7.389$，$e^1 = 2.718$，$e^0 = 1$，总和 11.107。概率 $= (0.665, 0.245, 0.090)$，加起来是 1。

4.12.2　交叉熵：错得有多离谱

交叉熵 cross-entropy

$$L = -\log p_y$$

y 是正确的类别，p_y 是模型给正确类别的概率，log 是自然对数。
它想表达：正确类的概率越接近 1，loss 越接近 0；概率越接近 0，loss 越大，而且是急剧变大。

例子

接上例：如果正确的是第 1 类，$L = -\ln 0.665 = 0.41$；如果正确的是第 3 类，$L = -\ln 0.090 = 2.41$。如果在 10,000 类里平均分配概率（每类 0.0001），$L = -\ln 0.0001 = \ln 10{,}000 = 9.21$，这就是“完全瞎猜”的 loss。

4.12.3　label smoothing：不要太自信

标签平滑

$$q_k = (1-\varepsilon)\,\mathbf{1}[k=y] + \frac{\varepsilon}{K}, \qquad L = -\sum_{k=1}^{K} q_k \log p_k$$

- q_k：训练目标里第 k 类“应得”的概率；$\varepsilon = 0.1$：拿出来平均分给所有类的份额；
- $\mathbf{1}[k=y]$：k 是正确类时等于 1，否则等于 0。

我们的数字：$K = 10{,}000$，正确类的目标是 $0.9 + 0.1/10{,}000 = 0.90001$，其他每类 0.00001。
它想表达：不要求模型对正确答案百分之百确定，留一点余地，防止过度自信、减少死记。

3 类的小例子（$\varepsilon = 0.1$）

目标 $q = (0.933, 0.033, 0.033)$。预测 $p = (0.665, 0.245, 0.090)$ 时：

$$L = -(0.933 \ln 0.665 + 0.033 \ln 0.245 + 0.033 \ln 0.090) \approx 0.381 + 0.047 + 0.080 \approx 0.51,$$

普通交叉熵是 0.41。如果模型给正确类 100% 的概率，其他类的概率就是 0，$\log 0$ 是负无穷，loss 反而变得无穷大。所以加了平滑之后，最好的预测是“很有把握，但不绝对”。

4.12.4　准确率，以及噪声有多大

top-1、top-5 和标准误

$$\text{top-1} = \frac{\text{第一名答案正确的图片数}}{\text{图片总数}}, \qquad \text{top-5} = \frac{\text{正确答案在前 5 名里的图片数}}{\text{图片总数}}$$

$$\text{标准误} = \sqrt{\frac{p\,(1-p)}{N}}$$

p 是准确率，N 是测试图片数。标准误表示“只因为测的是这 N 张图而不是别的图”，准确率会有多大的随机波动。

例子

- 5 张图，正确答案在模型排名里的位置分别是第 1、1、3、1、8 名：top-1 = 3/5 = 60%，top-5 = 4/5 = 80%。
- 我们的 holdout：$N = 20{,}000$，$p \approx 0.72$：$\sqrt{0.72 \times 0.28/20{,}000} \approx 0.0032$，也就是约 0.32 个百分点。所以 holdout 上 0.3 分以内的差别不能当真。

4.12.5　L2 归一化和余弦相似度

公式

$$\hat{z} = \frac{z}{\|z\|}, \quad \|z\| = \sqrt{z_1^2 + z_2^2 + \cdots + z_d^2}, \qquad \cos(a,b) = \frac{a \cdot b}{\|a\|\,\|b\|} = \frac{\sum_i a_i b_i}{\|a\|\,\|b\|}$$

$\|z\|$ 是向量的长度；$a \cdot b$ 是点积（对应位置相乘再相加）。
它想表达：归一化去掉“长短”，只留“方向”；余弦相似度只比较方向，1 是同方向，0 是互不相关，-1 是正好相反。

例子

$z = (3,4)$：长度 $\sqrt{9+16} = 5$，归一化后是 (0.6, 0.8)。$\cos\big((0.6,0.8),(0.707,0.707)\big) = 0.990$，方向很接近；$\cos\big((1,0),(0,1)\big) = 0$，互相垂直；$\cos\big((1,0),(-1,0)\big) = -1$，方向相反。

4.12.6　原型 head 的分数

公式

$$\text{logit}_c \approx \tau \cos(z, \mu_c), \qquad \tau = 40$$

μ_c 是第 c 类的原型（归一化后的平均方向），z 是一张图的特征。完整的推导和 3 类小例子，见本章“第 7 页”一节。

例子

一张图和某类原型的余弦是 0.990，分数就是 $40 \times 0.990 = 39.6$；和另一类的余弦是 0.569，分数是 22.8。分数差 16.8，经过 softmax 后几乎全部概率都给了第一类。

4.12.7　自注意力

self-attention

$$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

- Q、K、V：由 qkv 层从同一批 token 算出来的三组向量（197 个 token，每个头 64 维）。
- $QK^\top$：每个 token 的 Q 和每个 token 的 K 做点积，得到一张 197 × 197 的“相关程度表”。
- $\sqrt{d}$：除以 $\sqrt{64}=8$，防止点积太大，让 softmax 变得过于尖锐。
- softmax：把每一行变成加起来为 1 的权重；最后乘 V：按权重把所有 token 的内容加权平均。

它想表达：每个 token 的新表示 = 所有 token 内容的加权平均，越相关的 token 权重越大。那张 197 × 197 的表也解释了为什么 token 变多时，注意力的计算按平方增长。

2 个 token、2 维的小例子

token 1 的 $q=(1,0)$；两个 token 的 k 分别是 (1, 0) 和 (0, 1)，v 分别是 (10, 0) 和 (0, 10)。点积分别是 1 和 0；除以 $\sqrt{2}$ 得 0.707 和 0；softmax：$e^{0.707}=2.03$，$e^0=1$，权重是 0.67 和 0.33。输出 $=0.67\times(10,0)+0.33\times(0,10)=(6.7,3.3)$。token 1 主要“听”和自己匹配的 token 1，但也吸收了一部分 token 2 的信息。

4.12.8　LoRA：前向计算、参数量和合并

LoRA

$$y=W_0x+\frac{\alpha}{r}BAx,\qquad \text{参数量}=r\,(d_{\text{in}}+d_{\text{out}}),\qquad W'=W_0+\frac{\alpha}{r}BA$$

- W_0：冻结的原权重，形状 $d_{\text{out}}\times d_{\text{in}}$。
- A（$r\times d_{\text{in}}$）和 B（$d_{\text{out}}\times r$）：要训练的两个细长矩阵；B 初始化为 0。
- $r=8$：秩；$\alpha=16$：缩放，实际乘 $\alpha/r=2$。
- W'：训练完合并后的新权重，形状和 W_0 一样。

它想表达：对一个大矩阵的修改，可以只用两个细长矩阵的乘积来表示，参数少得多；训练完加回去，推理时和原模型一样快。

例子

fc1：$d_{\text{in}}=768$，$d_{\text{out}}=3{,}072$。LoRA 参数 $8\times(768+3{,}072)=30{,}720$；而 fc1 的整个权重矩阵有 $768\times3{,}072=2{,}359{,}296$ 个数，LoRA 只用了约 1.3%。合并的数值例子见上面“为什么 LoRA 能合并”一小节。

4.12.9　Adapter 和 AdaptFormer

公式

$$\text{Adapter:}\quad h\leftarrow h+W_{\text{up}}\,\text{ReLU}(W_{\text{down}}\,h)$$

$$\text{AdaptFormer:}\quad x_{l+1}=x'+\text{MLP}(\text{LN}(x'))+s\cdot W_{\text{up}}\,\text{ReLU}(W_{\text{down}}\,x')$$

- W_{down}：$768\to64$；W_{up}：$64\to768$，初始化为 0。

- ReLU：负数变成 0，正数不变。
- h：MLP 的输出；x'：注意力子层之后的结果；$s = 0.1$：AdaptFormer 的缩放系数。

它想表达：用一个“压缩 → 非线性 → 还原”的小模块学一点修正，再加回主路。Adapter 改 MLP 的输出（串联）；AdaptFormer 在 MLP 旁边加一条支路（并联）。

例子

ReLU(−2) = 0，ReLU(3) = 3。如果支路某个位置输出 5.0，AdaptFormer 乘 $s = 0.1$ 后只加 0.5，是一个温和的修正；Adapter 不乘系数，直接加 5.0。

4.12.10　逐层学习率衰减

layer-wise LR decay

$$\mathrm{lr}_l = \mathrm{lr} \times 0.75^{13-l}$$

- l：层号。最底层（patch embedding、[CLS]、位置嵌入）算第 0 层，12 个 block 是第 1 到 12 层，最后的 LayerNorm 和 head 算第 13 层。
- lr = 1e-4：全量微调的基础学习率；0.75：每往下一层乘的系数。

它想表达：越靠近输入，学习率越小；顶部的 head 学得最快，底层几乎不动，保护预训练学到的通用特征。只用在全量微调，LoRA 不用。

例子

层	系数	学习率
第 13 层（head、最后的 LN）	$0.75^0 = 1$	1.0e-4
第 12 个 block	$0.75^1 = 0.75$	7.5e-5
第 6 个 block	$0.75^7 \approx 0.133$	1.3e-5
第 1 个 block	$0.75^{12} \approx 0.032$	3.2e-6（约 3%）
第 0 层（patch embedding 等）	$0.75^{13} \approx 0.024$	2.4e-6

4.12.11　预热加余弦退火

学习率随步数的变化

$$f(s) = \begin{cases} \dfrac{s+1}{S_w}, & s < S_w \\ \dfrac{1}{2}\left(1 + \cos\dfrac{\pi\,(s - S_w)}{S - S_w}\right), & s \geq S_w \end{cases} \qquad \mathrm{lr}(s) = \mathrm{lr} \times f(s)$$

s 是当前第几步，S 是总步数，S_w 是预热的步数。

它想表达：前 S_w 步学习率从接近 0 直线升到最高；之后沿着余弦曲线慢慢降到 0。

全量最终模型：$S = 14{,}056$（8 个 epoch，每个 1,757 步），$S_w = 1{,}757$（1 个 epoch）

- 第 878 步（预热到一半）：$f = 879/1{,}757 \approx 0.5$，学习率约 5e-5；
- 第 1,757 步（预热结束）：$f = \frac{1}{2}(1 + \cos 0) = 1$，学习率 1e-4，最高；

- 约第 7,907 步（余弦段走到一半）：$\cos(\pi/2) = 0$，$f = 0.5$，学习率 5e-5；
- 最后一步：$\cos\pi = -1$，$f \approx 0$，学习率接近 0。

LoRA 是 3 个 epoch、每个 1,679 步、共 5,037 步，预热 0.3 个 epoch（约 503 步），形状一样。

4.12.12　AdamW 的一步（简化版）

AdamW

$$m \leftarrow \beta_1 m + (1-\beta_1)\, g, \qquad v \leftarrow \beta_2 v + (1-\beta_2)\, g^2$$

$$\theta \leftarrow \theta - \mathrm{lr} \cdot \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} - \mathrm{lr} \cdot \lambda\, \theta$$

- θ：一个参数；g：它的梯度。
- m：梯度的滑动平均，表示“方向”，带一点惯性；v：梯度平方的滑动平均，表示“这个参数的梯度一般有多大”。
- $\hat{m}$、$\hat{v}$：最开始几步的偏差修正版本；β_1, β_2：平均的“记忆长度”（PyTorch 默认 0.9 和 0.999）；ϵ：防止除以 0 的很小的数。
- λ：权重衰减系数。最后一项把参数往 0 拉，而且和梯度分开处理，这就是 AdamW 里的“W”（decoupled weight decay）。

它想表达：每个参数的步长自动调整：用 $\hat{m}/\sqrt{\hat{v}}$ 归一化之后，梯度一直很大的参数步子不会太大，梯度很小的参数步子也不会太小。

第一步：$\theta = 0.5$，$g = 0.2$，lr = 1e-4，$\lambda = 0.05$

偏差修正后 $\hat{m} = 0.2$，$\hat{v} = 0.04$，所以 $\hat{m}/\sqrt{\hat{v}} = 0.2/0.2 = 1$。

$$\theta \leftarrow 0.5 - 0.0001 \times 1 - 0.0001 \times 0.05 \times 0.5 = 0.5 - 0.0001 - 0.0000025 = 0.4998975.$$

如果梯度是 2.0（大 10 倍），第一步的改动还是约 0.0001：Adam 对梯度的整体大小不敏感，步长主要由学习率决定。

4.12.13　GPU 小时和预算

公式

$$\text{GPU 小时} = \text{GPU 数量} \times \text{小时数}, \qquad \text{预算} = \frac{2\ \text{h} \times 48\ \text{GB}}{80\ \text{GB}} = 1.2\ \text{h} \quad (\pm 10\%，\text{最多} 1.32\ \text{h})$$

例子

我们只用 1 块 A100，所以 GPU 小时就等于小时数。检查一下：全量最终模型 1.03，在预算内；LoRA 0.54，在预算内；288 像素、3 个 epoch 0.68，在预算内；288 像素、8 个 epoch 的配方 1.61，超出预算。

4.12.14　参数占比

公式

$$\text{占比} = \frac{\text{训练的 backbone 参数}}{85{,}798{,}656} \times 100\%$$

按课程参考的惯例，7.69M 的 head 不计入占比（课程给的 LoRA 0.34% 也不算 head）。

例子

LoRA：$1{,}179{,}648 \div 85{,}798{,}656 \approx 1.375\%$；BitFit：$102{,}912 \div 85{,}798{,}656 \approx 0.120\%$。如果把 head 也算上，LoRA 实际训练 $1{,}179{,}648 + 7{,}690{,}000 = 8{,}869{,}648$ 个参数，其中 head 占了约 87%。

第 5 章　Presentation Q&A

这一章是一个“题库”。我们按老师最可能问的方向，整理了 37 个问题。每道题有四部分：

- **英文原题**：老师大概会这样问；
- **中文解释**：这个问题其实在问什么、背后的原理是什么、要用到哪些数字。读懂这一部分，老师换个问法你也能答；
- **English answer**：一段可以直接说的英文回答，2–5 句；
- **可能的追问**：容易被继续追问的地方，以及怎么接着回答。

每道题开头的“主讲”表示建议由谁回答。A、B、C、D 的分工见下面的表。

5.1　回答问题的方法

回答三步法

1. **先给结论**：第一句话直接回答问题，不要先讲背景。
2. **再给一个关键数字**：比如“+2.5 个点”“1.03 GPU 小时”。数字让回答显得有根据。
3. **最后说原因**：一两句话解释为什么。

如果不知道答案，或者我们没做过这个实验，就诚实地说“我们没有测试”，然后说出你的猜想和你会怎么验证它。老师在邮件里说过：清楚、诚实的过程，比单纯追高分更重要。

谁来回答哪类问题

讲者	负责的问题	对应的页
A	预算、GPU 小时、基础模型（backbone）、数据、holdout、诚信	第 2、4、12 页
B	方法细节、参数量、延迟、LoRA 和 Adapter 的对比、实验过程	第 3、5、10、11 页，备用页 16–20
C	原型 head 和线性探针 head 的对比，以及证据	第 6–8 页
D	PEFT 和全量微调的差距、比较是否公平、局限、分辨率、训练配方	第 10–14 页

好用的英文句子

English	什么时候用
Good question. In short, …	先给结论，再展开
The key number here is …	引出一个关键数字
You can see this on slide …	把老师带回某一页
We did not test that, but our guess is …, and we would test it by …	没做过的实验，诚实回答
That is a fair point, and it is one of our limits.	老师指出了问题，先承认
The difference is within our noise of about 0.3 points.	差距很小时
Just to make sure I understand: are you asking about …?	没听清或没听懂问题
My teammate worked on this part, so let me hand it over.	把问题交给负责的队友

回答时千万不要

- **不要编数字**。记不清就说大概，比如“about 2 points”，然后指向对应的页。
- **不要说 288 px 的模型有 test 分数**。它只在 holdout 上评估过，没有 test 分数，也不是我们的正式结果。
- **不要猜别的队用了什么方法**。
- **不要把三种分数混在一起**：holdout（我们自己留出的 2 万张）、val（CodaBench 验证集）、test（CodaBench 测试集）。说数字时最好带上是哪一种。

5.2　基础概念

Q1. What is transfer learning, and what does fine-tuning mean in your project?

中文意思：什么是迁移学习？你们项目里的“微调”指什么？

主讲：A

迁移学习（transfer learning）就是“先在大数据上学会通用本领，再搬到新任务上用”。我们的 ViT-B/16 先在 ImageNet-21k 上训练，再在 ImageNet-1k 上训练，已经学会了看图的通用能力，比如边缘、纹理、形状。

微调（fine-tuning）就是拿这个学过的模型，在我们的新数据上继续训练一小会儿。新数据有 10,000 个类别，每类 45 张照片。微调有两大类做法：

- **全量微调**（full fine-tuning）：8,580 万个参数全部更新；
- **参数高效微调**（PEFT）：冻结大部分参数，只训练很少一部分。

我们两类都做了，一共比较了 7 种方法。

生活类比

一个已经学会画画的人，再去学画某一类鸟：不需要从拿笔开始学，只需要针对新内容多练几次。

English answer（可以直接说）

Transfer learning means we start from a model that already learned general visual features on large datasets, here ImageNet-21k and then ImageNet-1k. Fine-tuning means we keep training this model on our own task: 10,000 classes with 45 photos each. We compared full fine-tuning, which updates all 85.8 million weights, with parameter-efficient methods that update only a small part.

Q2. What is the difference between the backbone and the head?

中文意思：backbone 和 head 有什么区别？

主讲：B

backbone（主干网络）是负责“看图、提取特征”的大部分网络。我们的 backbone 是 ViT-B/16，有 8,580 万个参数，它把一张图变成一个 768 维的特征向量（768 个数字）。

head（分类头）是最后一层，把这 768 个数字变成每个类别的分数。我们的 head 是 $768 \to 10{,}000$ 的线性层，参数量是 $768 \times 10{,}000 + 10{,}000 = 7{,}690{,}000$，也就是 769 万。

预训练模型原来的 head 是给 ImageNet 的 1,000 类用的，不能直接拿来用，所以必须换一个新的 10,000 类 head。这个新 head 一开始“什么都不会”，怎么初始化它，就是我们项目最重要的发现（见第 6–8 页）。

生活类比

backbone 像人的眼睛和视觉大脑，负责看清楚；head 像最后“说出名字”的那一步。换任务时，眼睛可以继续用，但“要说哪些名字”必须重新学。

English answer（可以直接说）

The backbone is the big pretrained network that turns an image into a feature vector: for ViT-B/16, that is 85.8 million weights and a feature of 768 numbers. The head is the last layer that turns this feature into one score per class. Our head is new, 768 to 10,000, with 7.69 million weights, because the pretrained head was made for ImageNet's 1,000 classes. How this new head starts turned out to be our most important design choice.

可能的追问

追问：Why is the head so large?
回答：Because there are 10,000 classes. Each class needs its own 768 weights plus a bias, so 7.69 million in total, about 9% of the backbone. And each class row is learned from only 43 to 45 photos.

Q3. What does parameter-efficient fine-tuning mean, and why would anyone use it?

中文意思：什么是参数高效微调（PEFT）？为什么要用它？

主讲：B

参数高效微调（parameter-efficient fine-tuning，简称 PEFT）：冻结（不改）预训练模型的绝大部分参数，只训练一小部分新加的或选出来的参数。我们比较了 4 种：

- **BitFit**：只训练偏置（bias），占 backbone 的 0.12%；
- **Adapter**、**AdaptFormer**：在每个 block 里加一个小"瓶颈"模块，约 1.39%；
- **LoRA**：在线性层旁边加两个很小的低秩矩阵，1.38%。

注意：这些百分比不包括 head。head 在每种方法里都要训练。

PEFT 的主要好处是**省存储**：很多任务可以共用一个 backbone，每个任务只存一个小文件。

容易搞错

PEFT 省的是"要训练、要保存的参数"，不一定省训练显存和时间。我们的 LoRA 训练用了 19.9 GB 显存，比全量微调的 18.9 GB 还多（原因见本章"方法比较"一节）。

English answer（可以直接说）

PEFT means we freeze almost all pretrained weights and train only a small number of new or selected ones. We tried four: BitFit trains only the biases, 0.12% of the backbone; Adapter and AdaptFormer add small bottleneck modules, about 1.4%; LoRA adds low-rank matrices, 1.38%. The head is trained in every method. The main benefit is storage: one shared backbone plus a small file per task.

可能的追问

追问：Does PEFT also save training memory?
回答：Not always. In our runs LoRA needed 19.9 GB, more than full fine-tuning at 18.9 GB, because the gradients still flow back through all 12 blocks.

Q4. What exactly does your model take in, and what does it output?

中文意思：你们的模型输入是什么、输出是什么？

主讲：A

输入：一张彩色照片，缩放到 224 × 224 像素，有红、绿、蓝 3 个颜色通道。训练时还会随机裁剪和左右翻转（数据增强）。

输出：10,000 个分数（logits），每个类别一个。softmax 把这些分数变成概率（全部加起来等于 1），概率最大的那个类就是预测结果。

- **top-1 准确率**：概率最大的那个类刚好是正确答案的比例。我们报告的主要就是这个。
- **top-5 准确率**：正确答案在概率最大的 5 个类里面，就算对。

softmax 小例子（只有 3 类）

分数是 [2.0, 1.0, 0.1]。先取指数：$e^{2.0} = 7.39$，$e^{1.0} = 2.72$，$e^{0.1} = 1.11$，合计 11.22。再除以合计，得到概率约 [0.66, 0.24, 0.10]，所以预测是第 1 类。

English answer（可以直接说）

The input is one RGB image, resized to 224 by 224 pixels. The output is 10,000 scores, one per class. A softmax turns the scores into probabilities, and the class with the highest probability is our prediction. Top-1 accuracy is the share of images where this prediction is correct, and that is the main metric we report.

Q5. How do you measure performance, and why should we trust your holdout set?

中文意思：你们怎么评估模型？为什么要用 holdout，它可信吗？

主讲：A

val（5 万张）和 test（10 万张）的标签是隐藏的，只有上传到 CodaBench 才能知道分数，而且提交次数有限。所以我们从训练集里每类拿出 2 张照片，一共 20,000 张，当作自己的“模拟考卷”，叫 **holdout**（留出集）。这些照片：

- 从来不用来训练，只用来测准确率；
- 用固定的随机种子（seed 0）划分，所以每个实验用的是同一份。

为什么可信：它和 CodaBench 的分数非常接近。全量微调是 72.31% 对 72.57%，LoRA 是 71.98% 对 71.90%，差距都在 0.3 个点以内。

生活类比

考前自己留一套没做过的真题当模拟考。如果模拟考分数和正式考试分数差不多，就说明这套模拟考靠谱，可以放心用它来比较不同的复习方法。

English answer（可以直接说）

The validation and test labels are hidden, and submissions are limited, so we held out 2 training photos per class, 20,000 photos, as our own test set. They are never used for training, and the split is fixed. It tracks CodaBench closely: for LoRA, 71.98 on the holdout against 71.90 on validation, and full fine-tuning is also within 0.3 points. So all our comparisons use the holdout.

可能的追问

追问：Doesn't holding out 2 photos per class hurt the final model?

回答：A little. For full fine-tuning, training on all 45 photos instead of 43 added 0.26 points, so our final full model uses all 45.

5.3　技术原理

Q6. How does a Vision Transformer turn an image into a prediction?

中文意思：ViT 是怎么把一张图变成预测结果的?

主讲：B

ViT（Vision Transformer）的流程分五步：

1. **切块**：224 × 224 的图按 16 × 16 切开，得到 14 × 14 = 196 个小块（patch）。"B/16"里的 16 就是指 16 像素的小块。
2. **变成向量**：每个小块变成一个 768 维的向量，叫 token。再加一个特殊的 [CLS] token，一共 197 个。
3. **加位置信息**：每个 token 加上位置编码（position embedding），告诉模型这一块在图的哪里。
4. **12 个 Transformer block**：每个 block 先做自注意力（self-attention），让每个 token 参考其他所有 token 的信息；再用一个小网络 MLP 对每个 token 单独加工。
5. **输出**：取 [CLS] token 最后的 768 个数字作为整张图的特征，交给 head 打分。

生活类比

196 个人各拿一小块拼图，一起开 12 轮会。每轮会上，每个人先听听其他人手里是什么（attention），再自己想一想（MLP）。最后由主持人 [CLS] 总结：整张图是什么。

English answer（可以直接说）

ViT-B/16 cuts the 224-pixel image into 16-by-16 patches, which gives 196 patches. Each patch becomes a token of 768 numbers, and a special class token is added, so 197 tokens. Twelve Transformer blocks then mix information between all tokens with self-attention, and process each token with a small MLP. The class token's final 768 numbers are the image feature, and the head turns them into 10,000 class scores.

可能的追问

追问：What is self-attention, in simple words?

回答：Each token asks which other tokens are relevant to it, gives them weights, and takes a weighted average of their information. So a patch of a bird's wing can use information from the bird's head and from the background.

追问：How many weights are in one block?

回答：About 7.09 million. The MLP holds two thirds of them, which is why adding LoRA to the MLP helped.

Q7. Which loss function do you use, and why label smoothing?

中文意思：你们用什么损失函数？为什么加标签平滑？

主讲：D

我们用**交叉熵**（cross-entropy）损失。它的意思是：模型给正确类别的概率越高，损失越小。

交叉熵

$$L = -\log p_y, \qquad p_y = \frac{e^{z_y}}{\sum_{j=1}^{10{,}000} e^{z_j}}$$

z_j 是第 j 类的分数（logit）；y 是正确的类；p_y 是 softmax 给正确类的概率。
例：$p_y = 0.9$ 时 $L = 0.105$；$p_y = 0.01$ 时 $L = 4.6$。答得越错，惩罚越重。

标签平滑（label smoothing，我们用 0.1）：不要求模型对正确答案 100% 确定，而是把目标改成“正确类拿 90%，剩下的 10% 平均分给所有类”。每类只有 45 张照片，模型很容易对训练图过分自信、死记硬背，标签平滑可以减轻这一点。

English answer（可以直接说）

We use cross-entropy, the standard loss for classification: it is small when the model gives a high probability to the correct class. We add label smoothing of 0.1, so the target is not 100 percent on one class but 90 percent, with the rest spread over all classes. With only 45 photos per class, this keeps the model from becoming over-confident on the training photos.

可能的追问

追问：What loss does a random guess give?
回答：About the natural log of 10,000, which is 9.21. Our randomly initialized head started at 9.28, very close to that.

Q8. How does your prototype head work, step by step?

中文意思：你们的原型 head 具体是怎么做的？

主讲：C

一共四步，全程**没有梯度更新**：

1. 用冻结的 backbone 把 45 万张训练图各算出一个 768 维特征。约 6 分钟，算进 GPU 预算。
2. 把每个特征归一化成长度为 1（只保留“方向”）。同一类的所有特征加起来，再归一化一次，得到这一类的“原型”（prototype），也就是这一类的平均方向。
3. 把原型乘以温度 40，直接当作 head 的权重，偏置设为 0。
4. 预测时，一张图对某个类的分数约等于 $40 \times \cos$(图片特征, 类原型)。越像哪一类的“平均样子”，分数越高。

二维小例子

某类有两张图，特征是 $(3, 4)$ 和 $(6, 8)$。归一化后都变成 $(0.6, 0.8)$；相加得 $(1.2, 1.6)$，再归一化还是 $(0.6, 0.8)$。这就是这一类的原型。
一张新图的特征归一化后是 $(0.8, 0.6)$。余弦相似度是 $0.6 \times 0.8 + 0.8 \times 0.6 = 0.96$，所以分数是 $40 \times 0.96 = 38.4$。

生活类比

认人的时候，先把每个人的照片“平均”成一张标准脸，再看新照片最像哪一张标准脸。

English answer（可以直接说）

First, we run the frozen backbone once over all training photos and keep each 768-number feature. Then, for each class, we average the normalized features and normalize again; that is the class prototype. We copy the prototypes, times a temperature of 40, into the head. So each score is about 40 times the cosine similarity between a photo and its class mean. No gradient step is needed, so the head never fits individual training photos.

可能的追问

追问：How long does it take?
回答：About six minutes on the A100, and we count that time in our GPU budget.

追问：How accurate is the prototype head before any training?
回答：44 percent on the holdout. The linear-probe head starts higher, at 62 percent, but ends lower after fine-tuning.

Q9. How did you choose the temperature of 40?

中文意思：温度 40 是怎么选出来的?

主讲：C

余弦相似度总是在 −1 到 1 之间。如果直接用它当分数，10,000 个类的分数都挤在一个很小的范围里，softmax 之后每个类的概率几乎一样，模型显得“没有主见”。乘一个温度 s 就能把分数拉开。

我们在 {5, 10, 15, 20, 30, 40, 60, 80} 这 8 个值里挑，标准是：在 1 万个训练特征上算交叉熵，哪个最小就选哪个，结果是 40。这只是挑一个数字，不是训练 head 的权重。

为什么要放大

两个类的余弦相似度是 0.5 和 0.3。

- 不放大：softmax 概率约 0.55 和 0.45，几乎分不出高低；
- 乘 40：分数变成 20 和 12，概率约 0.9997 和 0.0003，差别就很明显了。

English answer（可以直接说）

Cosine similarity is always between minus one and one, so without scaling, the softmax over 10,000 classes would be almost flat. We tried eight values from 5 to 80 and picked the one with the lowest cross-entropy on 10,000 training features, which was 40. It is one number chosen by a simple search, not a trained head.

Q10. Why is a higher starting loss a good thing for the prototype head?

中文意思：为什么原型 head 起始损失更高，反而是好事?

主讲：C

三种 head 在 LoRA 训练前 100 步的平均损失：

- 线性探针 head：2.29（最低）；
- 原型 head：4.39；
- 随机 head：9.28（最高，接近随便猜的水平 $\ln 10{,}000 = 9.21$）。

这些是训练损失的平均值，包含了标签平滑和数据增强，所以只能定性地比较高低，不要把它们换算成“正确类的概率”。三个数可以这样理解：

- 9.28：完全不知道答案，相当于在 10,000 类里随便猜；
- 4.39：大概知道每一类长什么样，但还远远不确定，所以学习信号很强；
- 2.29：已经在这些训练图上拟合过，所以学习信号很弱。

线性探针 head 是在**同一批训练图**上训练出来的，所以一开始就“答对了大部分训练题”，损失很低。损失低，传回 backbone 的梯度（学习信号）就小，而且只来自它答错的少数几张图，backbone 学不到多少东西。

原型 head 已经有了正确的方向（起点准确率 44%），但还有很多没学会，所以学习信号足够大，能推动 backbone 去适应新任务。随机 head 的信号最大但方向很乱，一开始学得最慢：第 1 个 epoch 后是 58.4%，原型 head 是 63.3%。不过训练 3 个 epoch 后两者打平（71.95% 对 72.00%）。

生活类比

老师批改作业：如果学生的答案是照着标准答案抄的（线性探针 head 在训练图上拟合过），几乎全对，老师就没什么可教的。原型 head 像学生凭自己的理解写的答案，错的地方多一些，但老师的批改能真正帮他进步。

English answer（可以直接说）

The linear-probe head was fitted on the same training photos, so it starts with a low loss, 2.29. That means small gradients, coming from the few photos it still gets wrong, so the backbone barely adapts. The prototype head starts at 4.39: it already points in the right direction, at 44 percent accuracy, but it leaves a strong learning signal for the backbone. That is what we want, because our classes need the features to change.

Q11. How does LoRA work, and why can it be merged after training?

中文意思：LoRA 是怎么工作的？为什么训练完能合并？

主讲：B

LoRA（Low-Rank Adaptation，低秩适配）的做法：

- 不改原来的权重矩阵 W，而是在旁边学一个“修正量” $\Delta W = BA$；
- A 的大小是 $r \times d_{\text{in}}$，B 的大小是 $d_{\text{out}} \times r$。$r = 8$ 很小，所以参数很少；
- 前向计算是 $y = Wx + \frac{\alpha}{r} BAx$。我们用 $\alpha = 16$，所以缩放系数是 2；
- B 初始化为 0，所以训练开始时模型和原模型完全一样。

训练完以后，直接把 $\frac{\alpha}{r} BA$ 加进 W：$W' = W + \frac{\alpha}{r} BA$。合并后模型结构和原来一模一样，推理不多花任何时间。这叫合并（merge）。

qkv 层的参数量

qkv 层：$d_{\text{in}} = 768$，$d_{\text{out}} = 2304$。

- 原矩阵：$768 \times 2304 = 1{,}769{,}472$ 个参数；
- LoRA：$8 \times 768 + 2304 \times 8 = 24{,}576$ 个，约为原矩阵的 1.4%。

四种线性层加起来，每个 block 有 98,304 个，12 个 block 一共 1,179,648 个，占 backbone 的 1.375%。

English answer（可以直接说）

LoRA keeps each pretrained weight matrix W frozen and learns a small update next to it, B times A, with rank 8. For example, next to qkv, A is 8 by 768 and B is 2304 by 8, which is only 24,576 weights. B starts at zero, so training starts from the original model. After training we add the update into W, so the deployed model has exactly the original structure and the same speed.

可能的追问

追问：Why can't the adapters be merged in the same way?
回答：Their bottleneck has a nonlinearity, a ReLU, between the two small layers. So the module is not a single matrix and cannot be added into W. LoRA's update is linear, so it can.

追问：What does low-rank mean?
回答：B times A is a 768-by-2304 matrix, but it is built from only 8 directions. So it can only make a simple, low-dimensional change, which is often enough for adapting a pretrained model.

Q12. What do the training tricks in your recipe do: AdamW, warmup and cosine, layer-wise decay, drop path?

中文意思：训练配方里的这些技巧分别有什么用？

主讲：D

下面是全量微调最终模型用的设置，每个一两句话：

- **AdamW**：最常用的优化器，会根据每个参数过去的梯度，自动调整每一步走多大。
- **权重衰减**（weight decay，0.05）：每一步把权重往 0 拉一点点，防止权重变得太大、过拟合。
- **预热 + 余弦**（warmup + cosine）：第 1 个 epoch，学习率从 0 慢慢升到 10^{-4}，防止一开始步子太大，把预训练学到的特征"打乱"；之后学习率按余弦曲线慢慢降到 0，后期小步微调。
- **分层学习率衰减**（layer-wise LR decay，0.75）：越靠近输入的层，学习率越小，每往下一层乘 0.75，第一个 block 只有大约 3%。底层学到的是边缘、纹理这类通用特征，不需要大改；顶层离具体任务更近，需要多改。
- **随机深度**（drop path，0.1）：训练时随机跳过一些 block 的分支，像"随机请假"，逼网络不过分依赖某一个 block，减少过拟合。
- **梯度裁剪**（gradient clipping，1.0）：梯度太大时把它缩小，防止某一步更新过猛。
- **bf16**：用 16 位浮点数计算，更快、更省显存，精度几乎不受影响。

English answer（可以直接说）

For full fine-tuning we use AdamW at a learning rate of 1e-4, with one warmup epoch and then a cosine decay to zero. The learning rate shrinks by 0.75 per block toward the input, so the first block gets only about 3 percent of it: low layers hold general features that should change little. Weight decay of 0.05 and drop path of 0.1 regularize the model, and gradient clipping at 1.0 keeps the updates stable.

可能的追问

追问：Why does LoRA not use layer decay or drop path?
回答：Drop path is mainly a regularizer for training many weights, and LoRA trains few, so we kept its setup simple: one learning rate of 1e-3 and weight decay of 1e-4. We now suspect that this weaker regularization is one reason LoRA overfits after 3 epochs.

5.4　模型与 backbone 选择

Q13. Why did you use ViT-B/16, and why not a bigger or different backbone?

中文意思：为什么用 ViT-B/16？为什么不换一个更大或别的模型？

主讲：A

四个理由：

1. **项目要求**：brief 第 1 页推荐的就是这个模型（先在 ImageNet-21k、再在 ImageNet-1k 上训练）。老师的邮件也说过：没有很强的理由，就不要换 backbone。
2. **流程验证过**：我们复现了 brief 里的三个参考结果，差距都在 0.3 以内（零样本 43.95 对 43.7，全量微调 70.88 对 70.8，LoRA 68.57 对 68.3）。
3. **检查过别的版本**：截止后我们试了两个只在 ImageNet-21k 上训练的版本。线性探针只有 61.05% 和 51.07%，都比我们的 61.94% 低。所以换版本没有好处。
4. **预算**：更大的模型每个 epoch 要贵好几倍，1.2 小时的预算里只能训很少几个 epoch。这一点我们没有实际测试。

所以我们把精力放在 adaptation 方法上，这也是老师说的重点。

English answer（可以直接说）

We kept ViT-B/16 because it is the backbone the brief recommends, and the instructor asked us not to change it without a strong reason. Our pipeline reproduces all three reference numbers within 0.3 points. We also checked two ImageNet-21k-only versions: their linear probes reach 61.05 and 51.07 percent, both below our 61.94. So we spent our effort on the adaptation methods instead.

可能的追问

追问：Why compare backbones with a linear probe and not zero-shot?
回答：Zero-shot only uses class means, so it is a weak test of the features. A linear probe trains a classifier on the frozen features, so it shows better how useful they are. One of the in21k-only versions even has a higher zero-shot score, 49.96 against our 43.95, but a lower linear probe.

追问：Would a bigger backbone help?
回答：Probably, but it costs several times more per epoch, so it would get only a few epochs in our 1.2-hour budget. We did not test it.

Q14. Why did you choose LoRA as your final parameter-efficient model?

中文意思：为什么最终的高效微调模型选 LoRA?

主讲：B

- **准确率**：同一套设置下，LoRA 是 4 种 PEFT 里最高的（71.98%），Adapter 71.54%，AdaptFormer 71.17%，BitFit 69.81%。
- **推理速度**：LoRA 能合并进原权重，推理和原模型一样快（7.61 ms），而两种 Adapter 要慢 2.4–2.9 ms。
- **参数少**：只训练 backbone 的 1.375%，远低于 brief 里举例的 5%。
- **缺点**：训练最慢（每秒 888 张），显存最多（19.9 GB）。

English answer（可以直接说）

LoRA was the most accurate of the four PEFT methods under the same protocol, 71.98 percent on the holdout, with only 1.375 percent of the backbone trained. And after training it merges into the weights, so inference is as fast as the original model, 7.61 milliseconds per image, while the two adapters add 2.4 to 2.9 milliseconds. Its downside is training cost: it was the slowest to train and used the most memory.

5.5　方法比较

Q15. What do you mean by “where you train beats how much”?

中文意思：“训练的位置比训练的数量更重要”是什么意思？

主讲：B

这句话比较的是两件事：训练的参数放在**哪里**，和训练**多少**参数。两条证据：

1. LoRA 只训练 1.38% 的参数，但每个 block 里都有，准确率 71.98%；只训练最后 4 个 block（33% 的参数），是 71.88%，两者差不多。
2. BitFit 只训练所有偏置，占 0.12%，准确率 69.81%，反而比只训练最后 1 个 block（8.3%）的 67.97% 高。

结论：在每一层都做一点小改动，比在顶部几层做大改动更有效。原因（我们的解释）：10,000 个细粒度类别需要整个网络从低层到高层都稍微调整；只改顶层的话，底层特征完全没有机会适应。

生活类比

装修房子：每个房间都换一下灯和窗帘（小改动遍布全屋），往往比只把一个房间彻底翻新（大改动集中在一处）整体效果更好。

English answer（可以直接说）

It means the location of the trained weights matters more than their number. LoRA trains 1.38 percent of the weights, spread over every block, and matches training the last four blocks, which is 33 percent. BitFit trains only the biases, 0.12 percent, and beats training the last block, 8 percent. So a little capacity in every block beats a lot of capacity in a few top blocks.

可能的追问

追问：Why would every block need to change?
回答：Our 10,000 classes are fine-grained, so even low-level features like textures and shapes need some adjustment. Training only the top blocks leaves the lower blocks exactly as they were pretrained.

Q16. Why does LoRA need more memory than full fine-tuning, although it trains far fewer weights?

中文意思：LoRA 训练的参数少得多，为什么显存反而比全量微调还多？

主讲：B

训练时，显存主要花在**激活值**（activations）上：前向计算时，每一层的中间结果都要存起来，反向传播时才能算梯度。所以显存主要看“梯度要往回传多深”，而不是“训练多少参数”。

- **部分微调 K=4**：梯度只需传回最后 4 个 block，只存这 4 个 block 的激活，所以只要 7.0 GB。
- **LoRA**：每个 block 都有 LoRA，梯度要一直传回第 1 个 block。而且为了算 A 的梯度，每个线性层的输入都要存下来（和全量微调一样），再加上 96 个额外小矩阵乘法的激活（12 个 block × 4 个线性层 × 2 个矩阵）。所以是 19.9 GB。
- **全量微调**：多出来的只是优化器状态。8,580 万个参数大约 1 GB，在批量 256 时不是大头，所以是 18.9 GB。

English answer（可以直接说）

Training memory is mostly the activations stored for the backward pass, so it depends on how far back the gradients must flow, not on how many weights we train. LoRA sits in every block, so the gradients go all the way back, and to train its A matrices it must keep every linear layer's input, just like full fine-tuning, plus its own small activations. Full fine-tuning adds only about 1 GB of optimizer state on top. Training only the last four blocks needs 7 GB, because the gradients stop after four blocks.

可能的追问

追问：Then why do BitFit and the adapters need less, 12.7 to 12.8 GB?
回答：Their gradients also flow through every block, so they need more than partial fine-tuning. We think they need less than full fine-tuning because they do not have to keep the inputs of the frozen linear layers to compute weight gradients.

Q17. Why are Adapter and AdaptFormer slower at inference, but LoRA is not?

中文意思：为什么 Adapter 和 AdaptFormer 推理更慢，LoRA 却不慢？

主讲：B

- **Adapter**：在每个 block 的 MLP 后面**串联**一个小瓶颈：768 → 64 → 768，中间有一个 ReLU。
- **AdaptFormer**：把同样的瓶颈**并联**在 MLP 旁边，输出乘以 0.1 再加回去。

这些模块在推理时都要额外计算，而且中间有非线性的 ReLU，所以没法合并进原来的权重矩阵。批量 1 时：原模型 7.33 ms，Adapter 9.74 ms，AdaptFormer 10.23 ms，多了 2.4–2.9 ms，约 33%–40%。

LoRA 的修正 BA 是纯线性的，可以直接加进 W。合并后是 7.61 ms，和其他结构相同的模型在测量误差范围内（同结构模型之间本来就会差 0.5 ms 左右）。

English answer（可以直接说）

The adapters add small extra layers to every block, and because of the ReLU inside them, they cannot be folded into the frozen weights. So at batch size 1 they take 9.74 and 10.23 milliseconds, against 7.33 for the plain model. LoRA's update is linear, B times A, so we add it into W after training, and the model keeps its original structure and speed: 7.61 milliseconds, within normal timing noise.

Q18. Is LoRA really better than the Adapter?

中文意思：LoRA 真的比 Adapter 好吗？

主讲：B

准确率上，LoRA 71.98%，Adapter 71.54%，只差 0.44。我们每个设置只跑了一次，holdout 的噪声大约是 ±0.3，所以 0.44 只比噪声大一点点，不能说 LoRA 一定更准。

两者各有真正的优势：

- **LoRA**：能合并，推理不变慢；
- **Adapter**：训练快 1.6–1.7 倍，显存少 36%。

诚实的说法是：准确率差不多，按部署需求选。

English answer（可以直接说）

Only by a small margin: 71.98 against 71.54, a gap of 0.44 points, which is close to our noise level of about 0.3. So we would not claim LoRA is clearly more accurate. Its clear advantage is merging: no extra latency, 7.61 against 9.74 milliseconds. The Adapter, on the other hand, trains 1.6 to 1.7 times faster and uses 36 percent less memory.

Q19. Did you try other LoRA ranks?

中文意思：你们试过其他的 LoRA rank 吗？

主讲：B

没有。我们一直用 brief 参考设置里的秩 $r = 8$。我们增加容量的方式是**换位置**：参考设置只把 LoRA 放在 qkv 层，我们扩展到 qkv、proj、fc1、fc2 四种线性层。结果：

- 准确率 +0.88（68.57 → 69.45，这组实验用的是线性探针 head）；
- 参数从 0.344% 变成 1.375%，仍然远低于 5%。

理由：每个 block 有三分之二的权重在 MLP（fc1、fc2）里，只改注意力层是不够的。

English answer（可以直接说）

No, we kept rank 8 from the reference setup. Instead, we added capacity by placement: the reference puts LoRA only on qkv, and we added it to proj, fc1 and fc2 as well. That gave plus 0.88 points, and it is still only 1.375 percent of the backbone. Two thirds of each block's weights sit in the MLP, so adapting the MLP matters. A rank sweep would be a natural next experiment.

Q20. Why is every parameter-efficient method below full fine-tuning here?

中文意思：为什么所有高效微调方法都比全量微调差?

主讲：D

同一套设置下（3 个 epoch，原型 head），全量微调 73.38%，最好的 PEFT（LoRA）71.98%，差 1.4。我们认为可能的原因有两个：

1. 虽然每类只有 45 张，但总共有 45 万张，数据总量足够支撑训练全部参数；
2. 10,000 个细粒度类别之间的差别很细，需要整个网络都改变，PEFT 能改动的空间有限。

另外，训练更久对两者的效果正好相反：全量微调从 3 到 8 个 epoch，val +2.21；LoRA 从 3 到 6 个 epoch，−0.46。所以最终差距更大（val 76.22 对 71.90）。

不过 LoRA 只训练 1.4% 的参数，就达到了全量微调 98% 的准确率。

English answer（可以直接说）

Under the same protocol, full fine-tuning reaches 73.38 and LoRA 71.98, so LoRA keeps 98 percent of the accuracy with 1.4 percent of the weights. We think full fine-tuning wins because 450,000 photos in total is enough data to train every weight, and 10,000 fine-grained classes need changes in every layer. The final gap is larger because longer training helped full fine-tuning but hurt LoRA.

Q21. Is your final comparison between full fine-tuning and LoRA fair?

中文意思：你们最终的全量微调和 LoRA 对比公平吗?

主讲：D

要分两种比较来说：

1. **同条件比较**在第 5 页：7 种方法都用原型 head、3 个 epoch、同一份 holdout、同一块 GPU。全量微调 73.38 对 LoRA 71.98。
2. **第 12 页的最终结果**是“各自最好的设置”：全量微调用 8 个 epoch、全部 45 张、翻转 TTA；LoRA 用 3 个 epoch（训久了反而变差）、43 张、不加 TTA。所以第 12 页不是同条件比较，我们在幻灯片上也写明了。

还要主动承认一个局限：几种 PEFT 方法共用一个学习率 10^{-3}，没有分别调。只检查过 LoRA 的：减半后 −0.12，没有变化。

English answer（可以直接说）

There are two comparisons. The like-for-like one is on slide 5: same head, 3 epochs, same holdout and GPU, and there full fine-tuning leads LoRA, 73.38 to 71.98. The final results on slide 12 give each model its best setting, for example 8 epochs for full fine-tuning, because longer training hurt LoRA. One limit

we are open about: the efficient methods share one learning rate that we did not tune per method.

5.6　实验结果与数据解读

Q22. Why does the linear-probe head, the LP-FT recipe, hurt in your setting?

中文意思：为什么线性探针 head（LP-FT）在你们的任务里反而有害?

主讲：C

常见做法 **LP-FT**（Kumar et al., ICLR 2022）分两步：先冻结 backbone，在特征上训练一个线性分类器（线性探针，linear probe），再用它当 head 继续微调。

在我们的任务里，它**起点高、终点低**：

- 训练前：62.1% 对原型 head 的 44.0%，领先 18 个点；
- 训练后：全量微调 70.88 对 73.38，LoRA 69.45 对 72.00，AdaptFormer 69.05 对 71.17；最终配方的 val 73.10 对 76.22。

我们的解释分三步：

1. 线性探针是在**同一批训练图**上训练的，已经“背”下了这些图；
2. 所以起始损失很低（2.29），传给 backbone 的信号弱，而且只来自少数答错的图。backbone 一开始变化，原来拟合好的 head 就对不上了：LoRA 的训练损失在第 1 个 epoch 从 2.22 升到 2.69；
3. 最后，模型去拟合每类 45 张图本身，而不是学到能泛化的特征：训练损失更低（1.77 对 1.85），但 val 更低（73.10 对 76.22）。这就是过拟合，也叫“死记”。

English answer（可以直接说）

The linear probe is fitted on the same photos we then fine-tune on, so it starts with a low loss and gives the backbone only a weak learning signal. When the features start to move, the fitted head no longer matches them; with LoRA, the training loss even rose in the first epoch. In the end the model memorizes the 45 photos per class: a lower training loss, 1.77 against 1.85, but a lower validation accuracy, 73.10 against 76.22.

可能的追问

追问：Is this proven?
回答：It is consistent with all our loss curves and with the random-head control, but it is not a formal proof. We say that in the report.

追问：Did it hurt every method?
回答：It hurt all three methods we tested it with, full fine-tuning, LoRA and AdaptFormer, by 2.1 to 3.1 points.

Q23. The LP-FT paper says a random head distorts the pretrained features. How does that fit your random-head result?

中文意思：LP-FT 论文说随机 head 会破坏预训练特征，这和你们随机 head 的结果怎么对得上?

主讲：C

LP-FT 论文的观点：随机初始化的 head 在微调初期会产生很大、很乱的梯度，把预训练学到的好特征“扭曲”（feature distortion）。这尤其会伤害模型在新数据域（out-of-distribution）上的表现，所以要先训练一个好的线性 head。

我们的对照实验：LoRA 用随机 head 训练 3 个 epoch，结果 71.95%，和原型 head 的 72.00% 打平，比线性探针 head 的 69.45% 高 2.5。

这说明：在我们的任务里，“特征被改变”不是坏事，因为 10,000 个细粒度类本来就需要特征改变。真正的问题是 head 在同一批训练图上拟合过。原型 head 的好处是起步更快：第 1 个 epoch 后是 63.3%，随机 head 是 58.4%。

English answer（可以直接说）

LP-FT protects the pretrained features, mainly to keep robustness on new domains. In our task the features need to change, because the classes are new and fine-grained. And our control shows that a random head does fine here: with LoRA it ends at 71.95, level with the prototype head at 72.00, and well above the linear-probe head. So the harm comes from fitting the head on the training photos, not from changing the features.

可能的追问

追问：Then why use prototypes at all, if a random head ties?
回答：Prototypes learn faster: after one epoch they lead a random head, 63.3 against 58.4 percent, and they cost only six minutes. With a tight budget, a faster start is useful.

Q24. Is the prototype head your own idea? What is actually new in your work?

中文意思：原型 head 是你们自己发明的吗？你们的工作到底新在哪里？

主讲：C

方法本身不是我们发明的：把每个类的平均特征直接当作分类器权重，叫“权重印记”（weight imprinting，Qi et al., CVPR 2018），和原型网络（prototypical networks）是同一个思路。

我们的新发现是：在“10,000 类、每类 45 张、训练时间很短”的设定下，大家推荐的 LP-FT 起点反而让我们测试的三种方法都变差了 2.1–3.1 个点；而随机 head 和原型 head 打平，说明问题出在“head 在同一批图上拟合过”。

所以我们声称的新意是**发现和解释**，不是技术本身。实话实说，反而更可信。

English answer（可以直接说）

No, the technique is known: it is weight imprinting, from Qi and colleagues at CVPR 2018. What is new is our finding: with 10,000 classes and 45 photos per class, the recommended LP-FT start hurt all three methods we tested, by 2.1 to 3.1 points, while prototypes and even a random head did better. So we claim the finding and its explanation, not the method.

Q25. Why did LoRA get worse after 3 epochs, while full fine-tuning kept improving?

中文意思：为什么 LoRA 训练超过 3 个 epoch 就变差，全量微调却越训越好？

主讲：D

现象：LoRA 训练 6 个 epoch 比 3 个 epoch 低 0.46（71.52 对 71.98），而训练损失从 1.95 降到 1.70。

训练集上更好、holdout 上更差，这就是过拟合。全量微调正好相反：从 3 到 8 个 epoch，val +2.21。

我们的猜想（还没有验证）：LoRA 训练的参数里，87% 其实是 head（887 万个里有 769 万个）。而 LoRA 的设置给 head 的条件和全量微调很不一样：

- 学习率 10^{-3}，全量微调是 10^{-4}，高 10 倍；
- 权重衰减 10^{-4}，全量微调是 0.05，小 500 倍；
- 没有 drop path，全量微调有 0.1。

所以训练越久，head 越容易死记每类 45 张图。这和第 3 部分线性探针 head 的问题是同一回事。

验证方法已经准备好（notebook 10）：只把 head 的学习率降到 10^{-4}，训练 6 个 epoch，看还会不会掉分。

English answer（可以直接说）

With LoRA, 6 epochs gave 71.52 against 71.98 for 3, while the training loss kept falling, so it overfits. Our hypothesis, not yet tested, is the head: 7.69 of LoRA's 8.87 million trainable weights are the head, and our LoRA recipe trains it with a ten times higher learning rate than full fine-tuning, much less weight decay and no drop path. We have prepared the test: 6 LoRA epochs with a head learning rate of 1e-4.

可能的追问

追问：What result would support your hypothesis?
回答：If the 6-epoch run with the slower head is clearly above 71.52, by more than our noise of 0.3, then longer LoRA training no longer hurts, and the head was the cause.

Q26. Why did you train full fine-tuning for exactly 8 epochs?

中文意思：全量微调为什么正好训练 8 个 epoch?

主讲：D

两个原因：

1. **准确率还在涨**：最终配方下，从 3 到 8 个 epoch，val 从 74.01 升到 76.22（+2.21）。
2. **预算刚好够**：8 个 epoch 用了 1.03 GPU 小时，在 1.2 小时的预算内。一个 epoch 大约 6.5–7 分钟，再多就超预算了。

English answer（可以直接说）

Two reasons. Accuracy kept improving: going from 3 to 8 epochs added 2.21 points on validation, from 74.01 to 76.22. And 8 epochs fit our budget: the full run took 1.03 GPU-hours, below the 1.2-hour limit, with one epoch taking about 6.5 minutes.

Q27. Why does your final LoRA model use no test-time augmentation and only 43 photos per class?

中文意思：为什么最终 LoRA 模型不用 TTA，而且每类只用 43 张?

主讲：D

TTA（test-time augmentation，测试时增强）：测试时把每张图和它的左右翻转版本都算一遍，把两次的分数相加，结果更稳。代价是推理时间翻倍：全量微调最终模型的 14.95 ms 就是两次前向的时

间。对全量微调，它带来了 +0.27。

- **不加 TTA**：LoRA 最大的卖点就是“合并后零额外延迟”，加 TTA 会让延迟翻倍。
- **43 张**：LoRA 最终模型保留了每类 2 张 holdout，这样可以在本地验证它（holdout 71.98 对 val 71.90）。

对全量微调来说，用满 45 张只多了 0.26，接近噪声。这两项各只有约 0.25 分，不会改变两个模型的排序。

English answer（可以直接说）

For LoRA, the selling point is zero extra latency after merging, and flip TTA would double the inference time, so we kept it single-pass. We also kept its 2 holdout photos per class, so we could check it locally: 71.98 on the holdout against 71.90 on validation. For full fine-tuning, these two changes added only about 0.27 and 0.26 points, near our noise level.

Q28. Why are your test scores so close to your validation scores?

中文意思：为什么你们的 test 分数和 val 分数这么接近?

主讲：D

- val 有 5 万张，test 有 10 万张，都很大，而且来自同一个分布；
- 我们选模型、调参数主要看自己的 holdout，从来没有用 test 调过任何东西；
- Phase 2 用的就是 Phase 1 的同一个模型。

所以 val 和 test 很接近（76.22 对 76.09，71.90 对 71.67，差距都在 0.25 以内），说明分数反映的是真实的泛化能力，不是“碰巧在 val 上表现好”。

English answer（可以直接说）

Both sets are large, 50,000 and 100,000 photos, from the same distribution, and we never tuned anything on the test set. We chose our settings on our own holdout, and the test predictions come from the same Phase 1 models. So the small gap, 76.22 against 76.09 and 71.90 against 71.67, shows that the scores are real and not a lucky fit to validation.

Q29. What is your resolution result, and why is it not in your final results?

中文意思：分辨率的结果是什么？为什么没有放进最终结果?

主讲：D

截止日期后我们发现：同一个全量微调模型，只改输入图片的大小，准确率就能明显提高。位置编码用插值适配新的网格，不增加任何参数。下面都是 holdout 上 3 个 epoch 的结果：

训练尺寸 / 测试尺寸	top-1	代价
224 / 224（基线）	73.38%	0.48 GPU 小时，18.9 GB
224 / 288（不用重新训练）	75.50%（+2.1）	同上
288 / 288	77.23%（+3.9）	0.68 GPU 小时，30.2 GB
288 / 320	77.78%（+4.4）	同上

图变大，token 就变多：224 时 197 个，288 时 325 个，320 时 401 个。288 时计算量约为原来的 1.7 倍。

为什么不放进最终结果：

1. 截止日期之后才发现；
2. 用最终配方（8 个 epoch、全部数据）在 288 上训练要 1.61 GPU 小时，超出 1.2 小时的预算；
3. Phase 2 必须用 Phase 1 的同一个模型，所以这个模型没有 test 分数。

它是一个“教训”，不是“成绩”。

English answer（可以直接说）

After the deadline we found that image size is a big lever. The same 3-epoch full fine-tuning model, trained at 288 pixels and tested at 320, reaches 77.78 percent on the holdout, 4.4 points above the 224-pixel baseline, in 0.68 GPU-hours, but with 30 GB of memory instead of 19. We did not add it to our results: we found it after the deadline, the full 8-epoch recipe at 288 pixels takes 1.61 GPU-hours, which is over the budget, and it has no test score.

可能的追问

追问： Why does a larger test image help even without retraining?
回答： Random cropping during training zooms in on objects, so the model gets used to seeing them large. At test time, a slightly larger image makes objects look about as large again. This is known as FixRes, from Touvron and colleagues, 2019.

追问： Does the model need new weights for a larger image?
回答： No. A larger image just gives more patches, 18 by 18 instead of 14 by 14, and we interpolate the position embeddings to the new grid. No parameter is added.

Q30. How large is the noise in your numbers? Can we trust a gain of 0.26 points?

中文意思：你们的数字误差有多大？0.26 个点的提升可信吗？

主讲：A

holdout 有 20,000 张。准确率约 72% 时，抽样误差（标准误）大约是 0.3 个百分点：

标准误

$$\mathrm{SE} = \sqrt{\frac{p(1-p)}{n}} = \sqrt{\frac{0.72 \times 0.28}{20{,}000}} \approx 0.0032 = 0.32 \text{ 个百分点}$$

p 是准确率，n 是照片数。意思是：换一批同样大小的照片来测，准确率大约会上下浮动 0.3 个点。

我们每个设置只训练了一次（一个随机种子），没有重复。所以小的差距要谨慎看待，比如多 2 张照片的 +0.26、LoRA 学习率减半的 −0.12、LoRA 和 Adapter 之间的 0.44。

大的差距远大于噪声，是可靠的：原型 head 的 +2.1–3.1，分辨率的 +4.4。

English answer（可以直接说）

Each setting was trained once, and with 20,000 holdout photos at about 72 percent accuracy, the sampling error is about 0.3 points. So we treat smaller differences, like the 0.26-point gain from two extra photos

per class, as noise. Our main findings are far larger: the prototype head adds 2.1 to 3.1 points, and the resolution change adds 4.4.

可能的追问

追问：Why not run several seeds?

回答：Each run costs up to an hour of our budget, so we spent it on new questions instead. Repeating the key head comparison with more seeds is on our list.

5.7　实际应用

Q31. If you had to deploy this, would you use full fine-tuning or LoRA?

中文意思：如果真的要上线使用，你们会用全量微调还是 LoRA?

主讲：D

看需求：

- **只有一个任务、追求最高准确率**：用全量微调，test 76.09%。
- **要支持很多任务**（比如很多客户各有自己的类别）：用 LoRA。所有任务共用一个 backbone，每个任务只存一个小文件，35.5 MB（其中 30.8 MB 是 head）；而全量微调每个任务都要存一个 374 MB 的完整模型。部署时把 LoRA 合并进权重，速度和原模型一样。

English answer（可以直接说）

It depends on the use case. For one task where accuracy matters most, full fine-tuning: 76.09 percent on test. For many tasks, LoRA: one shared backbone and a small file per task, 35.5 megabytes, of which 30.8 is the head, instead of a 374-megabyte copy of the whole model. And after merging, LoRA runs at the original speed.

Q32. How did you measure latency, and why does it matter?

中文意思：你们怎么测量延迟？为什么它重要?

主讲：B

延迟（latency）是处理一张图要多久。我们在一块 A100 上测：用随机输入，只测模型的前向计算（不包括读图、解码），用 bf16。批量 1 跑 100 次取平均，得到延迟；批量 256 跑 20 次，得到**吞吐量**（throughput，每秒能处理多少张）。

全量微调最终模型是 14.95 ms，因为翻转 TTA 要算两次。为什么重要：实际应用里，用户在等结果；brief 也要求报告这一项。

English answer（可以直接说）

We measured on one A100 with random inputs, forward pass only, in bfloat16. Latency is the average over 100 runs at batch size 1; throughput uses batch size 256. Our full model takes 14.95 milliseconds because flip TTA runs two passes; LoRA after merging takes 7.61.

Q33. What would you do next with more time or compute?

中文意思：如果有更多时间或算力，你们下一步会做什么？

主讲：D

1. 在预算内，用更少的 epoch 换更大的图片（288 px），因为分辨率在 holdout 上带来了 +4.4；
2. 跑已经准备好的 head 学习率实验，验证 LoRA 过拟合的猜想；
3. 给每种 PEFT 方法单独调学习率；
4. 多跑几个随机种子，给主要结论加上误差范围；
5. 试 LoRA 的其他 rank，以及基于提示的方法，比如 VPT。

English answer（可以直接说）

First, trade some epochs for larger images within the budget, since resolution gave 4.4 holdout points. Second, run our prepared test of the LoRA head learning rate. Third, tune the learning rate for each efficient method and repeat the key comparisons with more seeds. We would also like to try other LoRA ranks and prompt-based methods like VPT.

5.8　过程、预算与诚信

Q34. How do you count GPU-hours, and do your models fit the budget?

中文意思：你们怎么计算 GPU 小时？模型在预算内吗？

主讲：A

规则是：每次训练最多用 2 小时 × 48 GB 显存 = 96 GB·h。我们用的是 80 GB 的 A100，所以 $96 \div 80 = 1.2$ 小时（加上允许的 ±10%，最多 1.32 小时）。

计入预算的：给 head 提取特征（约 6 分钟）、训练、训练过程中的 holdout 评估。**不计入的**：下载和解压数据。

结果：最终全量微调用了 1.03 GPU 小时，LoRA 用了 0.54 GPU 小时，都在预算内。

English answer（可以直接说）

The rule is 2 hours on a 48-gigabyte GPU, which is 96 gigabyte-hours. On our 80-gigabyte A100 that is 1.2 hours per run. We count everything the GPU does for a run: extracting features for the head, about 6 minutes, the training, and the holdout evaluation during training. Our final runs took 1.03 GPU-hours for full fine-tuning and 0.54 for LoRA, both inside the budget.

Q35. How did you decide what to try next, and what did the failed ideas teach you?

中文意思：你们是怎么决定下一步做什么的？失败的想法教会了你们什么？

主讲：B

我们每一轮只问一个问题、只改一个设置（第 4 页的 8 轮）。失败的想法也很有用：

1. **怀疑学习率，结果不是**：LoRA 的损失在第 1 个 epoch 上升，第一个怀疑是学习率太高。减半后没有变化（−0.12），排除了优化器，于是转而怀疑 head，结果找到了最大的提升（+2.5）。
2. **LoRA 训得更久反而变差**：训练损失降了，holdout 也降了，说明 LoRA 的瓶颈是死记，而不是没学够。

3. **好的起点会骗人**：线性探针 head 训练前领先 18 个点，训练后却落后 2.5 个点。所以我们永远在训练之后用 holdout 来判断，不看训练损失，也不看起点。

English answer（可以直接说）

We asked one question per round and changed one setting at a time. The failed ideas pointed the way: when LoRA's loss rose, we first suspected the learning rate, but halving it changed nothing, which pointed us to the head and our biggest gain. Training LoRA longer lowered its training loss but also its accuracy, which showed memorization, not underfitting. And we learned that a good start can mislead, so we always judged on held-out photos after training.

Q36. The top team reached about 84 percent. Why is your score lower?

中文意思：排名第一的队伍有约 84%，为什么你们的分数更低?

主讲：D

不要猜别人用了什么方法，我们不知道。可以这样回答：

- 老师的邮件说过，过程比排名更重要；
- 我们遵守了推荐的 backbone 和预算，把实验次数花在理解 adaptation 方法上：7 种方法的同条件比较，以及找到并解释 head 的问题；
- 我们截止后发现，只改输入分辨率就能在 holdout 上多 4.4 个点。这是一个我们发现得太晚的杠杆。

English answer（可以直接说）

We do not know what the top team did, so we would not guess. We kept the recommended backbone and the budget, and we spent our runs on understanding the adaptation methods rather than only chasing accuracy. After the deadline we found that input resolution alone adds 4.4 points on our holdout, so that is one lever we found too late.

Q37. Which models made your test predictions, and did you use the test set in any way?

中文意思：test 预测是哪个模型做的？你们有没有用过 test 集?

主讲：A

课程规则：Phase 2 必须用 Phase 1 的同一个模型来预测 test。我们的两个 test 结果都来自 Phase 1 的模型：

- 全量微调 1019_1319_1_full：76.09%；
- LoRA 1019_1319_3_lora：71.67%。

我们从来没有用 test 集来选模型或调参数，所有选择都在自己的 holdout 上做。截止后的 288 px 模型只在 holdout 上评估过，没有 test 分数。

English answer（可以直接说）

Both test files come from our Phase 1 models, as the course rule requires: 76.09 percent for full fine-tuning and 71.67 for LoRA. We never used the test set to choose settings; all our choices were made on our own holdout. The post-deadline 288-pixel model was only evaluated on the holdout and has no test score.

5.9 最可能被问到的 10 个问题

考前最后看一遍：10 个最可能的问题和一句话回答

1. **Why ViT-B/16?** The brief recommends it; we reproduce the references within 0.3; ImageNet-21k-only versions probe worse (61.05 and 51.07 against 61.94).
2. **Why does the prototype head beat the linear-probe head?** The probe is fitted on the same photos, so it gives a weak signal and the model memorizes; prototypes add 2.1 to 3.1 points.
3. **Is the prototype head new?** No, it is weight imprinting (Qi et al., 2018); our finding about LP-FT in this setting is what is new.
4. **Why LoRA as the efficient model?** Most accurate PEFT method (71.98) with 1.375% of the weights, and it merges, so there is no extra latency.
5. **Why does LoRA need more memory than full fine-tuning?** Memory follows how far the gradients flow; LoRA keeps every linear layer's input, plus its own activations.
6. **Why is PEFT below full fine-tuning?** 450,000 photos are enough to train every weight, and fine-grained classes need every layer to change; LoRA still keeps 98%.
7. **Is the comparison fair?** The like-for-like test is slide 5 (73.38 against 71.98); slide 12 gives each model its best setting; one shared, untuned learning rate is a limit.
8. **Why trust the holdout?** It matches CodaBench within 0.3; it is never used for training.
9. **What about resolution?** +4.4 on the holdout after the deadline; the 8-epoch 288 px run is over budget (1.61 h) and has no test score.
10. **Why does LoRA get worse after 3 epochs?** It overfits; we suspect its fast, weakly regularized head, and the test is prepared.

第 6 章　最后总结

6.1　整个 Presentation 的核心逻辑

整场演讲是一个“发现问题 → 提出假设 → 验证 → 得出结论”的故事。每一步都回答上一步留下的问题。

页码	这一步在做什么	一句话和关键数字
1	开场	7 种方法、一个预算；最重要的是 head 怎么开始；测试集 76.09% 和 71.67%
2	分析问题	10,000 类、每类 45 张、1.2 小时、推荐的 backbone；每个难点对应一个设计决定
3	确认流程正确	三个参考基线都复现在 0.3 分以内；介绍两类共 7 种方法
4	展示过程	8 轮实验，每轮一个问题；失败的想法排除了错误解释
5	比较方法	在哪里训练比训练多少更重要；显存跟梯度传多深有关；LoRA 能合并
6	发现问题	线性探针 head 起点高 18 分，终点低 2.5 分
7	解决问题	原型 head：0 步训练、44% 起点、起始 loss 4.39
8	解释原因	训练 loss 更低但准确率更低 = 背答案；随机 head 打平原型 head
9	检验每个设置	head +3.12；LoRA 加到四层 +0.88；LoRA 6 轮 −0.46；全量 8 轮 +2.21
10–11	两个最终模型	全量微调（8 轮、逐层衰减、TTA）；LoRA（1.375%、3 轮、合并）
12	最终成绩	76.22/76.09 和 71.90/71.67；测试和验证差不到 0.25；都在预算内
13	截止后的发现	更大的图片 +4.4（holdout，无测试成绩）
14	总结	三个发现 + 局限 + 下一步

用三句话讲完整个项目

1. 我们在同一个预算、同一个 backbone 下比较了 7 种适配方法：全量微调最准，LoRA 是最好的高效方法，而且“在每一层做小改动”比“集中改顶层”划算得多。
2. 最大的提升来自 head 的初始化：常用的线性探针 head 因为在同一批训练照片上拟合过，会导致模型背答案；用类别原型初始化，三种方法都多出 2.1–3.1 分。
3. 最终全量微调测试集 76.09%（参考 70.8%），LoRA 71.67%（参考 68.3%）；截止后还发现更大的输入图片能再多 4.4 分。

6.2　最重要的 10 个知识点

1. **预训练 + 微调**：不从零训练，而是拿学过看图的 ViT-B/16 来适应新任务。backbone 提供特征，新的 head 负责 10,000 类打分。
2. **ViT 怎么看图**：224 像素的图切成 196 个 16×16 的小块，加一个 [CLS] 共 197 个 token，经过 12 个 Transformer block，用 [CLS] 的 768 个数字分类。
3. **七种方法的本质区别**：训练哪些参数、训练多少参数。线性探针只训 head；部分微调训顶部 K 个 block；全量微调训全部；BitFit 只训偏置；Adapter / AdaptFormer 加小瓶颈模块；LoRA 学一个低秩更新 $\Delta W = BA$。
4. **在哪里训练比训练多少更重要**：LoRA 1.38% $\approx$ 部分微调 33%；BitFit 0.12% $>$ 部分微调 8%。
5. **LoRA 的优点**：参数少、可以合并（推理不变慢）；**缺点**：训练时显存不省（19.9 GB），速度最慢。
6. **线性探针 head 为什么有害**：它在同一批训练照片上拟合过 $\rightarrow$ 起始 loss 低、学习信号弱 $\rightarrow$ 特征一变就不匹配 $\rightarrow$ 背答案。
7. **原型 head 为什么有效**：用每类特征的平均方向 $\times$ 40 当权重，0 步训练、不背答案，训练前就有 44%；随机 head 对照实验证明伤害来自“拟合训练照片”。
8. **判断好坏用 holdout，不用训练 loss**：holdout 和 CodaBench 差 0.3 以内；差异小于约 0.3 分算噪声；每个设置只跑一次。
9. **训练配方里每个设置都有理由**：全量微调 8 轮（还在进步、预算内）、逐层学习率衰减、标签平滑、drop path；LoRA 3 轮（6 轮过拟合）、四个线性层（+0.88）。
10. **截止后的分辨率发现**：训练 288、测试 320，holdout +4.4；原因是 FixRes（训练时随机裁剪放大了物体）；只有 holdout 成绩，没有测试成绩。

术语	一句话解释

6.3　必须理解的专业术语

下面这些词在演讲和提问里几乎一定会出现。第 3 章有完整的术语表。

术语	一句话解释
backbone / head	模型主体（把图片变成特征）/ 最后的分类层（把特征变成 10,000 个分数）
pretraining / fine-tuning	在大数据上先学 / 在新任务上继续学
PEFT	参数高效微调：只训练很少的参数
full fine-tuning	全量微调：训练所有参数
linear probe	冻结 backbone，只训练 head
LP-FT	先训练线性探针 head，再微调整个模型的流行做法
prototype / imprinting	每类特征的平均方向；直接把原型“印”成 head 的权重
LoRA, rank r, α	学一个低秩更新 $\Delta W = BA$；r 是中间维度（8）；实际更新乘 $\alpha/r = 2$
merge	训练完把 LoRA 的更新加回原权重，推理不变慢
Adapter / AdaptFormer	串联 / 并联的瓶颈小模块，不能合并
BitFit	只训练偏置
token / patch / [CLS]	图片小块变成的向量 / 16×16 的图片小块 / 负责总结全图的特殊 token
self-attention	让每个小块参考其他所有小块的信息
logits / softmax	模型输出的原始分数 / 把分数变成加起来等于 1 的概率
cross-entropy / label smoothing	衡量“正确答案的概率有多低”的 loss / 不要求 100% 有把握，正确类目标设为约 0.9
overfitting / memorization	在训练数据上好、新数据上差 / 把训练照片背下来
holdout / validation / test	自己留的模拟考 / CodaBench 月考 / 期末考
top-1 accuracy	第一个答案就对的比例
epoch / step / batch	看完一遍所有数据 / 更新一次参数 / 一次用多少张图（256）
learning rate / warmup / cosine	每一步走多大 / 开始时慢慢加大 / 后面按余弦曲线慢慢减小
weight decay / drop path	把参数往 0 拉一点 / 训练时随机跳过一些 block；都是防过拟合
layer-wise LR decay	越靠近输入的层学习率越小（每层乘 0.75）
TTA（flip）	测试时用原图和镜像各预测一次再合起来
GPU-hours / latency / throughput	用了多少 GPU 小时 / 一张图要多少毫秒 / 每秒处理多少张
FixRes	训练时随机裁剪放大了物体，测试时用更大的图更匹配
ablation	一次只改一个设置，看它单独的影响

6.4　每个模型和方法的核心区别

方法	训练什么	参数 / holdout	核心区别（一句话）
线性探针	只训练 head	0% / 61.94%	特征完全不变；最快最省，但上限低
部分微调 K=4	最后 4 个 block	33% / 71.88%	只改顶层，底层不变；训练显存最小（7.0 GB）
全量微调	全部参数	100% / 73.38%	最准；每个任务存一整份模型（374 MB）
BitFit	只训练偏置	0.12% / 69.81%	参数最少，推理不变慢，但容量太小
Adapter	串联瓶颈模块	1.39% / 71.54%	加在 MLP 之后的主路上；推理慢 2.4 ms
AdaptFormer	并联瓶颈旁路	1.39% / 71.17%	和 MLP 并排，输出乘 0.1；推理慢约 2.9 ms
LoRA	低秩矩阵 A、B	1.38% / 71.98%	高效方法里最准；能合并，推理不变慢；训练显存最大
随机 head	（初始化方式）	–	什么都不知道；起始 loss 9.28；最后和原型 head 打平（LoRA）
线性探针 head	（初始化方式）	–	训练前 62.1%，但在训练照片上拟合过，最后低 2.1–3.1 分
原型 head	（初始化方式）	–	训练前 44.0%，0 步训练，不背答案；我们的选择

6.5　最容易被老师问到的技术问题

完整的回答见第 5 章。这里是一句话版本：

1. **为什么原型 head 比线性探针 head 好？** 线性探针 head 在同一批训练照片上拟合过，导致学习信号弱、模型背答案；原型 head 没有拟合任何照片。证据：训练 loss 更低但验证更低（1.77 对 1.85，73.10 对 76.22），随机 head 和原型 head 打平。
2. **原型 head 是你们发明的吗？** 不是，它叫 weight imprinting（Qi 等人，2018）。我们的新发现是：在类别多、每类照片少的设置下，它比推荐的 LP-FT 起点好。
3. **为什么不换更大或更新的 backbone？** 课程推荐这个；我们检查过只在 ImageNet-21k 上训练的版本，线性探针更差（61.05%、51.07% 对 61.94%）；而且预算只有 1.2 小时。
4. **为什么 LoRA 比全量微调还占显存？** 它和全量微调一样要保存每个线性层的输入来算梯度，还多了 96 个小矩阵乘法的中间结果；全量微调多出的优化器状态只有约 1 GB。
5. **为什么 LoRA 推理不变慢，Adapter 会变慢？** LoRA 的更新是线性的，可以加回原权重；Adapter 是额外的非线性层，必须单独计算。
6. **为什么 PEFT 比全量微调低？** 450,000 张照片总量其实不少，足够让全量微调发挥；10,000 个细粒度类别需要每一层都有较大改变。同等条件下差 1.4 分（73.38 对 71.98）。

7. **为什么 LoRA 训 6 轮反而变差？**训练 loss 下降但 holdout 下降，是过拟合。我们的猜想（未验证）：LoRA 训练的参数 87% 是 head，而 head 的学习率是全量微调的 10 倍、几乎没有权重衰减。
8. **你们的结论可靠吗？**每个设置只跑一次，差异小于约 0.3 分算噪声；但主要结论（head +2.1 到 +3.1、方法之间的大差距）都远大于噪声。holdout 和 CodaBench 一致，测试和验证一致。
9. **你们遵守预算了吗？**是。两个最终模型分别 1.03 和 0.54 GPU 小时，都低于 1.2；计时包括特征提取、训练和训练中的评估。
10. **分辨率那个结果为什么不用？**截止后才发现；只有 holdout 成绩；8 轮版本要 1.61 小时，超预算；Phase 2 规则要求用 Phase 1 的同一个模型。

6.6　Presentation 前应该重点复习什么

6.6.1　所有人都要会的

- 第 6 章第 1 节的三句话总结，能用英文说出来。
- 附录 A 里的核心数字：76.22 / 76.09、71.90 / 71.67、70.8、68.3、73.38 / 71.98、1.2 小时、1.375%、+2.1–3.1、+4.4。
- 区分三种分数：holdout（我们自己的 20,000 张）、验证集（CodaBench）、测试集（最终排名）。说数字时要说清楚是哪一个。
- 区分两种“线性探针”：一种适配方法（第 16 页），一种 head 初始化方式（第 6 页）。

6.6.2　按讲者分工

讲者	重点复习
A（第 1–3 页）	预算怎么算（96/80 = 1.2）；为什么不换 backbone（推荐 + 21k 版本更差 + 预算）；holdout 怎么建、为什么可信；三个基线的数字；七种方法各一句话
B（第 4–6 页）	8 轮实验的逻辑链；第 5 页的图怎么读；显存为什么跟梯度深度有关；LoRA 能合并 / adapter 不能；LP-FT 是什么；LoRA 参数怎么算（备用页 16–20 也归你）
C（第 7–9 页）	原型 head 的四步；余弦相似度和温度 40 的含义；4.39 / 2.29 / 9.28 三个起始 loss；“训练 loss 低但准确率低”的三步解释；随机 head 对照；第 9 页每一行的原因
D（第 10–15 页）	两个最终模型的所有设置（loss、学习率、轮数、参数量、GPU 小时、显存、延迟）；第 12 页的公平性说明；分辨率结果为什么不算；第 14 页的局限和下一步；Q&A 分配

6.6.3　几个容易读错的词

- LoRA：读作“LOR-uh”（重音在前），不是逐个字母念。
- ViT：可以读“vit”或“V-I-T”，组里统一一种即可。
- AdamW：读作“Adam W”。GELU：读作“gee-loo”。bf16：读作“B-F-sixteen”。
- CodaBench：“co-da-bench”。FixRes：“fix-res”。

6.6.4　最后提醒

容易搞错

- 封面和结束页上还是 [Name A]–[Name D] 占位符，讲之前要换成真名。
- 288 像素的模型**没有测试成绩**，不要说成正式结果。
- 不要猜测排行榜第一名用了什么方法；可以说我们更关注过程，并且找到了一个截止后才发现的提升方向（分辨率）。
- 不知道的就承认：“We did not test that, but our guess is …”

附录 A　关键数字速查表

所有数字都来自幻灯片、讲稿备注和报告。括号里注明是哪一种分数：holdout（我们留出的 20,000 张）、验证集（CodaBench val）、测试集（CodaBench test）。

A.1　任务、数据和预算

项目	数字
类别数	10,000（细粒度，类名隐藏）
训练集	450,000 张，每类 45 张
验证集 / 测试集	50,000 张（每类 5 张）/ 100,000 张（每类 10 张），标签隐藏
Holdout	每类 2 张，共 20,000 张；做 holdout 时用每类 43 张训练
Holdout 和 CodaBench 的一致性	72.31 对 72.57（全量微调），71.98 对 71.90（LoRA）
噪声	约 ±0.3 分（20,000 张、准确率 72% 时标准误约 0.32 分）；每个设置跑一次
预算	2 小时 × 48 GB = 96 GB·h；A100-80GB 上 1.2 小时（加 10% 是 1.32 小时）

A.2　模型（ViT-B/16）

项目	数字
版本	`vit_base_patch16_224.augreg2_in21k_ft_in1k`（ImageNet-21k 预训练 → ImageNet-1k 微调）
backbone 参数	85,798,656（约 8,580 万）
结构	12 个 block，每个 7,087,872 个参数；宽度 768；12 个注意力头 × 64；MLP 768 → 3072 → 768
输入	224 × 224；16 × 16 的块 → 196 块 + [CLS] = 197 个 token
新 head	768 → 10,000，7,690,000 个参数（backbone 的 9.0%）
21k 版本对比（线性探针）	augreg_in21k 61.05%，orig_in21k 51.07%，我们的 61.94%

A.3　参考基线（holdout）

基线	参考	我们
Zero-shot（原型 head，不训练）	43.7%	43.95%
全量微调 3 轮（线性探针 head）	70.8%	70.88%
LoRA r = 8 只在 qkv（294,912 个参数，0.34%），3 轮（线性探针 head）	68.3%	68.57%

复现基线时用的是线性探针 head；换成原型 head 后，同样 3 轮的全量微调是 73.38%（第 5 页）。

A.4　七种方法（原型 head、3 轮、holdout）

方法	训练比例 / 参数	holdout	训练显存	延迟（bs 1）
线性探针	0%	61.94%	3.2 GB	6.96 ms
部分微调 K = 4	33%	71.88%	7.0 GB	7.41 ms
全量微调	100%（85.8M）	73.38%	18.9 GB	7.78 ms
BitFit	0.12%（102,912）	69.81%	12.7 GB	7.33 ms
Adapter	1.39%（1,189,632）	71.54%	12.8 GB	9.74 ms
AdaptFormer	1.39%（1,189,632）	71.17%	12.8 GB	10.23 ms
LoRA（四个线性层）	1.38%（1,179,648）	71.98%	19.9 GB	7.61 ms

部分微调的完整结果：K = 1、2、4、6 分别训练 8.26%、16.5%、33%、49.6% 的参数，holdout 67.97%、70.99%、71.88%、72.89%，训练显存 3.8–10.0 GB。

A.5　Head 初始化

设置	线性探针 head	原型 head	随机 head
训练前（只有 head）	62.1%	44.0%	约 0
起始训练 loss（LoRA 前 100 步平均）	2.29	4.39	9.28
全量微调 3 轮（holdout）	70.88%	73.38%	–
LoRA 3 轮（holdout）	69.45%	72.00%	71.95%
AdaptFormer 3 轮（holdout）	69.05%	71.17%	–
全量微调最终配方（验证集）	73.10%	76.22%	–
最终配方的训练 loss（最后）	1.77	1.85	–

线性探针 head 在 LoRA 训练第 1 轮里 loss 从 2.22 升到 2.69。原型 head 第 1 轮后领先随机 head：63.3% 对 58.4%。

A.6　消融实验（一次改一个）

改动	前 → 后	变化
Head：线性探针 → 原型（全量 8 轮，验证集）	73.10 → 76.22	+3.12
LoRA 学习率减半（holdout）	68.57 → 68.45	−0.12
LoRA：qkv → 四个线性层（holdout）	68.57 → 69.45	+0.88
LoRA：3 轮 → 6 轮（holdout；训练 loss 1.95 → 1.70）	71.98 → 71.52	−0.46
全量微调：3 轮 → 8 轮（验证集）	74.01 → 76.22	+2.21
Flip TTA（验证集）	72.57 → 72.84	+0.27
每类 43 → 45 张（验证集）	72.84 → 73.10	+0.26

A.7　两个最终模型

项目	全量微调（1019_1319_1_full）	LoRA（1019_1319_3_lora）
训练的参数	全部 85.8M + head	LoRA 1,179,648（1.375%）+ head
数据	每类 45 张	每类 43 张
Loss	交叉熵 + 标签平滑 0.1	同左
优化器	AdamW，权重衰减 0.05，梯度裁剪 1.0	AdamW，权重衰减 10^{-4}，梯度裁剪 1.0
学习率	10^{-4}，逐层乘 0.75	10^{-3}，无逐层衰减
其他	drop path 0.1，bf16，batch 256	无 drop path，bf16，batch 256，r = 8，α = 16
轮数 / 步数	8 轮（1 轮预热 + 余弦）/ 14,056 步	3 轮（0.3 轮预热 + 余弦）/ 5,037 步
GPU 小时 / 显存	1.03 / 18.9 GB	0.54 / 19.9 GB
推理	flip TTA：14.95 ms，1,919 张/秒	合并后：7.61 ms，3,869 张/秒
验证集 / 测试集	76.22% / **76.09%**（参考 70.8%，+5.4）	71.90% / **71.67%**（参考 68.3%，+3.6）

A.8　分辨率（截止后，holdout，3 轮）

设置（训练 / 测试像素）	token 数	holdout	GPU 小时	显存
224 / 224	197	73.38%	0.48	18.9 GB
224 / 288	325	75.50%（+2.1）	0.48	18.9 GB
288 / 288	325	77.23%（+3.9）	0.68	30.2 GB
288 / 320	401	77.78%（+4.4）	0.68	30.2 GB
最终配方 8 轮用 288 / 320	401	没有测试成绩	1.61（超预算）	30.2 GB